Universitetet i Stavanger

DET TEKNISK-NATURVITENSKAPELIGE FAKULTET

BACHELOROPPGAVE

Studieprogram/spesialisering: Bachelor i ingeniørfag / Datateknologi	Vårsemesteret 2022 **Åpen** / Konfidensiell
Forfatter(e): Vebjørn Lia Riiser, Åse Jortveit Sagebakken	
Fagansvarlig: Erlend Tøssebro Veileder(e): Erlend Tøssebro	
Tittel på bacheloroppgaven: Operatørgrensesnitt og topside-kommunikasjon for fjernstyrt undervannsfartøy Engelsk tittel: Operator User Interface and Topside Communication for Remotely Operated Vehicle	
Studiepoeng: 2 x 20	
Emneord: Operatørgrensesnitt, GUI, ROV, kommunikasjon, Python, PyQt, UiS Subsea	Sidetall: 92 + vedlegg/annet: 5 Stavanger 15. mai 2021

Sammendrag

Denne bacheloroppgaven har gått ut på å lage *topside*-kontrollsystemet for en undervannsrobot (ROV), som en tverrfaglig gruppe i studentorganisasjonen UiS Subsea har lagd i samarbeid. ROV-en er laget med utgangspunkt i oppgavene som skal utføres i *MATE ROV World Championship*, som skal arrangeres i Los Angeles, Long Beach City College mellom 21. og 25. juni 2022 [1].

Oppgaven beskriver hvordan kommunikasjonen mellom *topside* og ROV-en foregår over TCP, hvordan inndata fra kontrolleren er blitt håndtert med Pygame, og hvordan brukergrensesnittet er blitt lagd ved å benytte Python og Qt.

Det har blitt laget et flertrådet program med som er delt opp i moduler. Videre har det blitt implementert en velfungerende klasse for sending og mottak av data. Programmet er modulært, noe som gjør det lett å skifte ut deler.

Det ble definert klare oppgaver og alle de viktigste målene ble nådd. Kommunikasjonen til ROV-en fungerer. Funksjonene i operatørgrensesnittet som kreves for at ROV-en skal kunne styres, fungerer. Resultatet er en velfungerende ROV. Neste steg er å delta i ROV MATE-konkurransen, som UiS Subsea har kvalifisert seg til.

Det er planlagt å utføre forbedringer frem mot konkurransen, som ikke kunne bli utført før bachelorfristen siden informasjonen om andre gruppers implementasjon ikke var klar.

Link til *topside*-kontrollsystemet på GitHub:
`https://github.com/VebjornRiiser/uis-subsea/tree/8df34d2329690fc835bab00274981f09dc4399d2`

Forord

Denne oppgaven markerer siste del av bachelorgraden i Datateknologi på Universitetet i Stavanger. Vi ønsker først og fremst takke alle medlemmene som har bidratt i UiS Subsea. Det har vært et godt samarbeid og uten dem hadde ikke semesteret blitt det samme. Vi har sammen fått til mye på kort tid.

En takk går også til UiS Subsea sine sponsorer og Universitetet i Stavanger som legger til rette for studentorganisasjoner.

Vi ønsker også å takke veileder og fagansvarlig Erlend Tøssebro for oppfølging og samtaler.

Forkortelser og uttrykk

Liste med forkortelser eller uttrykk og medfølgende forklaringer som er relevante til oppgaven.

Forkortelse - Hele ordet - Forklaring

- **CAN** - Controller Area Network - Et nettverk for å kommunisere mellom mikrokontrollere. [2.1.1]
- **GUI** - Graphical User Interface - Brukergrensesnittet som benyttes for å styre undervannsroboten. [3.8]
- **HTTP** - HyperText Transfer Protocol - Protokoll for å overføre HTML-dokumenter mellom tjener og klient. Bruker klient-server-modellen.
- **JSON** - JavaScript Object Notation - Et tekstbasert format for datautveksling. [2.2]
- **JPEG** - Joint Photographic Expert Group - Bildeformat som bruker en komprimeringsmetode med kvalitetstap (*lossy compression* på engelsk).
- **MATE ROV Competition** - Marine Advanced Technology Education Competition - Internasjonal konkurranse for ingeniørstudenter, hvor målet er å designe, utvikle og bygge en fjernstyrt undervannsfarkost. [1.3]
- **MJPEG** - Motion JPG - Videoformat hvor hver video-bilderute er lagret som et JPEG-bilde. [2.2]
- **LAN** - Local Area Network - Et lokalt nettverk for sending av data. Avgrenset til et lite geografisk område.
- **OSI-modellen** - Open System Interconnection Model - Referansemodell for nettverkskommunikasjon. [2.1.1]
- **ROV** - Remotely Operated Vehicle - Roboten som skal fjernstyres for å løse oppgavene. [1.2.1]
- **SSH** - Secure Shell - Nettverksprotokoll for koble til en terminal på en annen datamaskin. [2.1.1]
- **TCP** - Transmission Control Protocol - Nettverksprotokoll som brukes for å pålitelig sende informasjon mellom to enheter. [2.1.1]
- **UDP** - User Datagram Protocol - Nettverksprotokoll for forbindelsesløs overføring av informasjon. [2.1.1]
- **USB** - Universal Serial Bus - Universiell standard for å koble enheter til en datamaskin.

Uttrykk - Forklaring

- ***Topside*** - Delen av ROV-ens kontrollsystem som befinner seg over vann.
- ***Subside*** - Delen av ROV-systemet som befinner seg under vann.

Innhold

Sammendrag **i**

Forord **ii**

Forkortelser og uttrykk **iv**

1 Introduksjon **1**

1.1 UiS Subsea 2

1.2 Farkostene 3

1.3 MATE – Marine Advanced Tehchnology Education 5

1.4 Overordnet system 15

1.5 Prosjektinndeling 16

1.6 Operatørgrensesnitt og *topside*-kommunikasjon 18

2 Teori og bakgrunn **22**

2.1 Teknologier og protokoller 23

2.2 Formater 27

2.3 Vektorgrafikk 27

2.4 Utviklingsmetodikk og arbeidsmetode 29

2.5 Datasikkerhet 31

2.6 Programmeringsspråk 31

2.7 Rammeverk brukt i operatørgrensesnittet . . . 34
2.8 Datamaskin og oppbevaringsløsning . . . 40
2.9 Styrespak . . . 41
2.10 Styring av ROV-en . . . 43
2.11 Xbox-kontrolleren . . . 44
2.12 Kommunikasjon mellom ROV og *topside* . . . 46
2.13 ROV-ens kamera . . . 48
2.14 Logging av sensordata . . . 50
2.15 Brukergrensesnitt i subsea-industrien . . . 50
2.16 Fjorårets brukergrensesnitt . . . 50

3 Implementasjon **52**

3.1 Oversikt over kommunikasjon og programflyt . . . 53
3.2 Programflyt . . . 54
3.3 Informasjonsoverføring mellom topside og mini-PC . . . 55
3.4 Oversikt over data i GUI . . . 56
3.5 Kontrolleren . . . 57
3.6 Kontroll over et flertrådet program . . . 60
3.7 Sanntidsvisning av videostrøm . . . 61
3.8 Brukergrensesnittet . . . 63

4 Økonomisk oversikt og miljørapport **76**

4.1 Miljøregnskap . . . 77

5 Resultater og diskusjon **78**

5.1 Testing av ROV . . . 78
5.2 Resultat av ROV . . . 82

5.3 Overflatesystemet 82
5.4 Utfordringer 84
5.5 Forbedringer i forhold til tidligere år 84
5.6 Veien videre fremover 84
5.7 Forbedringer til neste år 85

6 Konklusjon **86**

Bibliografi **88**

Vedlegg **92**

A Fremdriftsplan **93**

B Prosjektfiler **95**

Kapittel 1

Introduksjon

Kapitteloversikt

1.1 UiS Subsea . . . 2
1.2 Farkostene . . . 3
1.2.1 ROV . . . 3
1.2.2 Fenris . . . 3
1.2.3 Flyter . . . 3
1.2.4 Frøya . . . 4
1.3 MATE – Marine Advanced Tehchnology Education . . . 5
1.4 Overordnet system . . . 15
1.5 Prosjektinndeling . . . 16
1.5.1 Elektro . . . 16
1.5.2 Maskin . . . 17
1.5.3 Data . . . 17
1.5.4 Økonomi . . . 17
1.6 Operatørgrensesnitt og *topside*-kommunikasjon . . . 18
1.6.1 Problemstilling . . . 18
1.6.2 Samarbeid mellom gruppene . . . 18
1.6.3 Motivasjon . . . 19
1.6.4 Hva slags system skal vi lage? . . . 19
1.6.5 Oversikt over hva som skal gjøres . . . 20

Denne bacheloroppgaven er en del av et større prosjekt, som baserer seg på design, utvikling og bygging av undervannsfarkoster. Utgangspunktet for prosjektet er deltakelse i den internasjonale konkurransen, *MATE ROV Competition* [2]. Kapittelet vil presentere organisasjonen UiS Subsea, hvilke typer undervannsfarkoster som skal utvikles og oppgavene de designes etter. Deretter vil det overordnede systemet presenteres med inndeling av ansvarsområder for de ulike bachelorgruppene. **Avslutningsvis, i kapittel 1.6, gis et mer detaljert innblikk i systemet denne bacheloroppgaven omhandler.**

Introduksjonskapittelet er skrevet av gruppen med ansvar for styrings- og reguleringssystemet, og er delt med alle gruppene i prosjektet. Kapittelet er felles for alle gruppene ettersom det presenterer prosjektet i sin helhet.

1.1 UiS Subsea

UiS Subsea er en studentorganisasjon ved Universitetet i Stavanger, som siden 2013 har engasjert studenter i undervannsteknologi. UiS Subsea har som mål å gi studentene erfaring med å jobbe i et større prosjekt på tvers av fagområder.

Organisasjonen har gjennom flere år bygget ROV-er for å delta i MATE-konkurransen. Dette gir grunnlag for et prosjekt som krever at de involverte løser utfordrende tverrfaglige oppgaver. Formålet med dette er å skape et miljø hvor studentene får utviklet sine tekniske ferdigheter, samt evne til å samarbeide med andre studenter. UiS Subsea er en organisasjon som muliggjør et tett samarbeid mellom studenter og næringslivet. En rekke bedrifter er involvert og svært interessert i dette prosjektet, og bidrar med utstyr og andre ressurser gjennom sponsoravtaler. For å dyrke dette samarbeidet, vil det i 2022, for første gang, arrangeres *Subsea-dagen*. Her vil næringslivet inviteres til universitetet for å presentere sine bedrifter.

Figur 1.1: UiS Subsea sin logo.

I tidligere år har det manglet kontinuitet i organisasjonen, da den hvert år har blitt overtatt av et helt nytt kull med bachelorstudenter. I fjor ble organisasjonen restrukturert ved at styret ble adskilt fra selve prosjektet, med formål om å muliggjøre drift uavhengig av prosjektdeltakere. Dette åpner også mer opp for deltakelse fra studenter i andre perioder av utdanningsløpet. Fjorårets leder og nestleder har valgt å sitte en periode til, for å redusere belastningen på bachelorstudentene, og for å videreføre tidligere erfaringer.

Nytt for årets prosjekt, er at også en bachelorgruppe av økonomistudenter innehar en rolle i driften av organisasjonen. Dette bidrar med kunnskap innenfor økonomi og markedsføring i styret.

Årets styre består av følgende roller med tilhørende innehavere:

- **Styreleder:** Daniel Vasshus
- **Nestleder:** Geir Arne Solland Kindingstad
- **Økonomiansvarlig:** Sina Brunes
- **Markedsføring:** Sanna Sørskår og Åse Jortveit Sagebakken
- **Sponsoransvarlig:** Maren Lovise Jåsund og Otto Nessa Ljosdal
- **Styremedlem:** Tage Mellemstrand

Grunnet restruktureringen, er det opprettet en egen prosjektledelse med ansvar for det tekniske. Prosjektledelsen fungerer som et bindeledd mellom organisasjonen og prosjektdeltakerene.

Prosjektledelsen med roller og innehavere er:

- **Prosjektleder:** Tomas Royal Choat
- **Teknisk ansvarlig programvare:** Christoffer Næss
- **Teknisk ansvarlig maskinvare:** Mats Røste

1.2 Farkostene

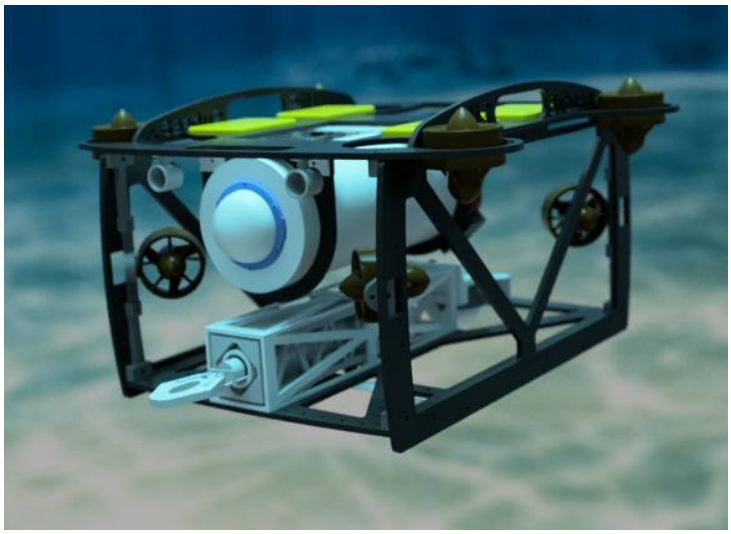
1.2.1 ROV

En ***R**emotely **O**perated Underwater **V**ehicle* (ROV) er, som navnet indikerer, en fjernstyrt undervannsfarkost. ROV-en er et hjelpemiddel som muliggjør undervannsoperasjoner uten behov for dykkere. Utviklingen av denne typen undervannsroboter har åpnet dørene for en rekke operasjoner under vann som ikke var mulig før, da man ikke lenger trenger å risikere menneskeliv. Dette utvider rekkevidden under vann, og forenkler operasjonene, siden de kan gjennomføres i mindre skala. Nyere ROV-er er også mulig å operere fra land, som reduserer kostnader av å utføre operasjoner *offshore*.

Utviklingen av ROV-er har på grunn av *offshore*-næringen vokst kraftig siden midten av 1980-tallet [3], da dette medførte utplassering av store installasjoner og behov for operasjoner på dybder man ikke kunne nå med dykkere. Eksempler på dette fremheves av årets konkurranse, med oppgaver som inspeksjon av fiskemerder og havvindmøller.

1.2.2 Fenris

ROV-en som utvikles i år, *Fenris*, er en frittsvømmende ROV med navlestreng, vist i figur 1.2.



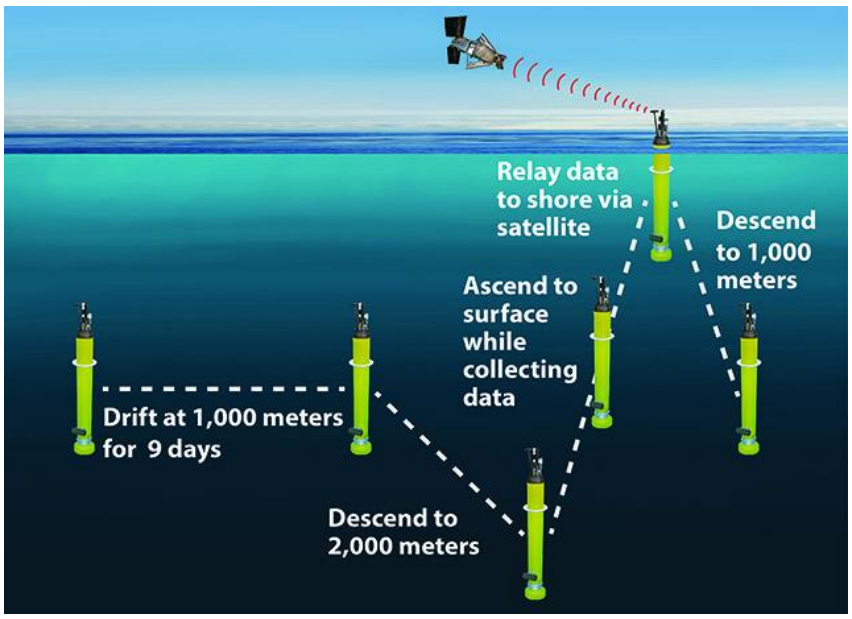
Figur 1.2: 3D-modell av Fenris.

Fenris styres fra et operatørgrensesnitt på land, og kommuniserer gjennom navlestrengen. Navlestrengen utgjør også kraftforsyningen til ROV-en. Målet er at ROV-en skal være i stand til å nå en dybde på 50 meter. Fenris er designet etter oppgavene i *MATE ROV Competition*, og er derfor utstyrt med en manipulatorarm for å løse disse. For å tilrettelegge for videreutvikling på ROV-en, er den designet modulært, slik at alt enkelt kan byttes ut. Dette gjør at Fenris kan brukes som en base for eventuell fremtidig utvikling.

1.2.3 Flyter

Nytt for årets prosjekt er at det skal bygges en *flyter*, inspirert av *float*-teknologi utviklet av forskningsprosjektet GO-BGC [4]. En flyter er en autonom undervannsfarkost som har i hoved-

oppgave å overvåke områder under vann. For å utføre dette, er den utstyrt med biologiske og kjemiske sensorer. Den har også en innebygd mekanisme som endrer volumet til farkosten, og dermed kan oppdriften reguleres. Et eksempel på syklusen til en flyter er vist i figur 1.3.

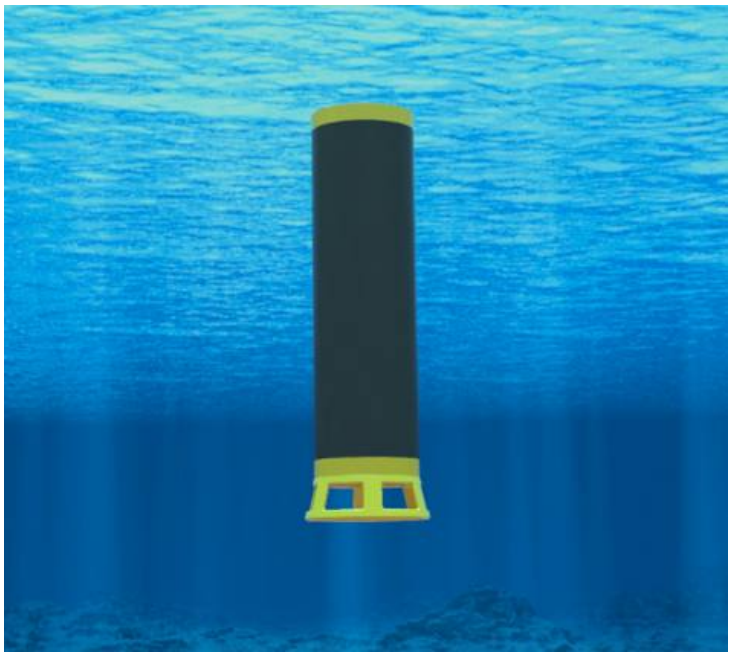


Figur 1.3: Eksempel på syklusen til en flyter. Hentet fra [5].

Flyteren kan dykke ned til flere tusen meters dybde, og returnere til overflaten på egenhånd. Når flyteren når overflaten, sendes innhentet data via satellitt, og slik skal den kunne fortsette ubemannet i opptil 5 år.

1.2.4 Frøya

Flyteren som utvikles i år, *Frøya*, er vist i figur 1.4.



Figur 1.4: 3D-modell av Frøya.

Dette er en prototype av en flyter, hvor fokuset er rettet mot den dynamiske oppdriftsmekanismen, og den er ikke utstyrt med biologiske eller kjemiske sensorer. Dybden den befinner seg på måles med en trykksensor, og reguleres gjennom en oppdriftsmotor. Oppdriftsmotoren er realisert ved hjelp av en pumpe som forflytter luft ut i en ballong på undersiden av flyteren. Av sikkerhetsmessige årsaker, må batteriene i flyteren være alkaliske. Dette begrenser hvor lenge den kan operere. Målet er at flyteren skal være i stand til å gjennomføre 2 vertikale profiler innenfor en tidsramme, etter at den er blitt plassert i vannet av ROV-en. I konkurransen tilsvarer en vertikal profil et dykk fra overflaten, ned til bunnen og tilbake.

1.3 MATE – Marine Advanced Tehchnology Education

Informasjonen under er hentet fra arrangørens hjemmeside [6].

Prosjektet gjennomføres, som tidligere nevnt, med hovedfokus rettet mot en internasjonal konkurranse innen utvikling av robotiserte undervannsfarkoster. Arrangøren bak konkurransen er forskningssenteret *The Marine Advanced Tehchnology Education (MATE) Center*, som er et samarbeid mellom flere organisasjoner i USA, med et felles mål om å forbedre utdanning innen marin teknikk. Senterets logo er vist i figur 1.5. Forskningssenteret jobber i tett samarbeid med flere skoler og universiteter, men også med bedrifter. Gjennom dette samarbeidet kan de tilby utdanning av studenter, basert på etablerte retningslinjer fra bedriftene, og sørge for direkte kontakt mellom student og arbeidsgiver under utdanningen. Et av hovedformålene med The MATE Center er å gi studenter et læringsutbytte, som senere kan være med på å utvikle hele næringen. I den sammenheng arrangeres konkurransen *MATE ROV Competition* hvert år. UiS Subsea stiller i *utforskerklassen*, som er klassen med høyest vanskelighetsgrad.



Figur 1.5: The MATE Center sin logo. Hentet fra hjemmesiden deres.

MATE ROV Competition

Informasjon om konkurransen er hentet fra konkurransemanualen [2]. Konkurransen har også sin egen logo, vist i figuren under.



Figur 1.6: *MATE ROV Competition* logo. Hentet fra konkurransens nettside.

Årets utgave av konkurransen vender oppmerksomheten mot FNs havforskningstiår (2021-2030), som er et initiativ fra FN med bakgrunn i de 17 bærekraftsmålene. Motivasjonen bak MATEs engasjement er å snu den nedadgående trenden til havets helse, og i den forbindelse vil de utfordre studenter globalt til å komme med nyskapende løsninger på problemet. Oppgaven går ut på å bygge en ROV som bidrar i arbeidet mot klimaendringer, legger til rette for ren energi, sørger for mat til en voksende populasjon, overvåker havets helse og bevarer maritim historie. MATE konstaterer at gjennom dette arbeid vil man skape «havet vi trenger, for fremtiden vi ønsker».

Poenggivning

Under konkurransen deles det ut poeng i tre hovedkategorier. I del 1 gjennomgår man en produktdemonstrasjon. Her kontrolleres vekt og fysiske mål. I denne delen skal også ROV-en og flyteren gjennom de praktiske oppgavene som tester deres operative egenskaper. I konkurransen er det tre separate oppgaver man skal løse, hvor hver oppgave har en tidsramme på 15 minutter. I tillegg belønnes struktur og effektivitet under demonstrasjonen. Man får 1 poeng ekstra for hvert minutt, og 0.01 poeng for hvert sekund man fullfører før den avsatte tiden i hver oppgave. I del 2 får man poeng for teknisk dokumentasjon av farkostene, samt kommunikasjon og markedsføring fra organisasjonens (UiS Subsea) side. I del 3 får man poeng for å ta hensyn til sikkerhet. Dette innebærer skriftlig sikkerhetsdokumentasjon, men også det å vise aktsomhet under operasjoner med ROV-en.

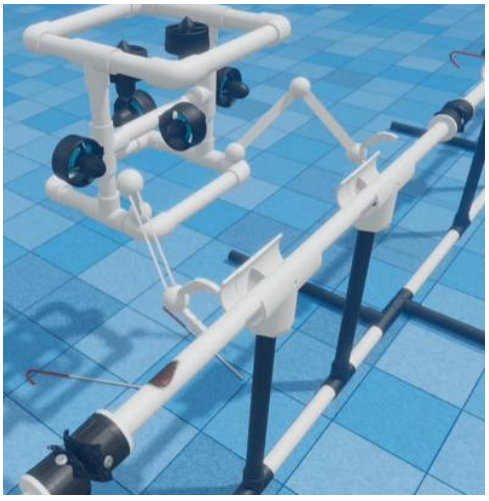
Under gjennomgås de praktiske oppgavene, med tilhørende poengfordeling. Oppgaver, poengfordeling og restriksjoner er gjengitt fra konkurransemanualen [7].

Oppgave 1: Maritim fornybar energi
FNs bærekraftsmål:
#7 Ren energi for alle
#12 Ansvarlig forbruk og produksjon

Oppgaven er utformet for å representere arbeidsoppgaver som er nødvendige for å utføre vedlikehold på *offshore* havvind. Man skal blant annet reparere en skadet strømkabel, erstatte en ødelagt oppdriftsmodul og fjerne et fiskegarn som har satt seg fast på en av vindturbinenes understell. Utover dette skal det også plasseres ut en hydrofon for å overvåke tilstedeværelsen av sjøpattedyr. Til slutt skal ROV-en kjøre autonomt inn i en dokking-stasjon. Alle arbeidsoppgavene er simulert ved hjelp av konstruksjoner laget av PVC-rør. Under er oppgavebeskrivelsene gjengitt, med tilhørende poengfordeling.

1.1 Erstatte en skadet del av en strømkabel

Figur 1.7 viser hvordan deler av oppgaven kan løses.

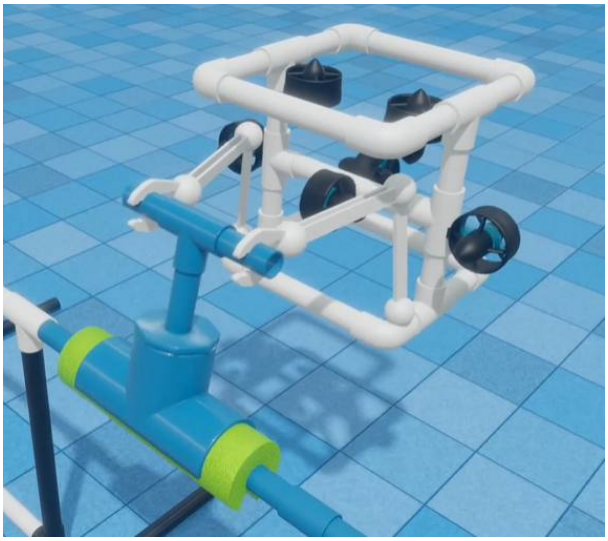


Figur 1.7: Fjerning av skadet del av kabel. Skaden er markert med en brun flekk. Låsepinnen vises rett til venstre for den brune flekken på røret. Hentet fra [8].

- Utføre en visuell inspeksjon av kabelen: **5 poeng**.
- Kutte kabelen på begge sider av den skadede delen. Simulert ved å dra ut en låsepinne på hver side: **10 poeng**.
- Fjerne den skadede delen av kabelen (PVC-rør), og bringe den til overflaten: **5 poeng**.
- Installere en ny kabelseksjon (PVC-rør), og feste denne med tilhørende festemekanisme: **5 poeng** for hver ende av røret, totalt **10 poeng**.

1.2 Erstatte en skadet oppdriftsmodul på en strømkabel tilhørende en havvindmølle

Figur 1.8 viser hvordan deler av oppgaven kan løses.

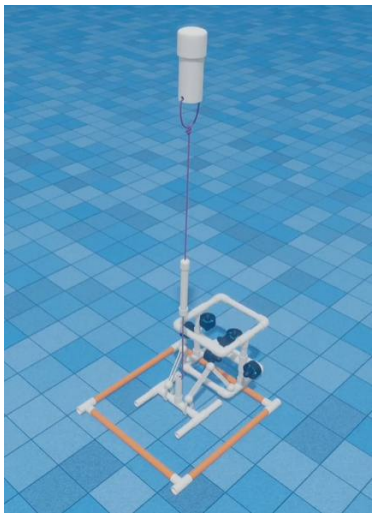


Figur 1.8: Fjerning av skadet oppdriftsmodul. Den nye oppdriftsmodulen er lik den som fjernes, men har borrelås som lar den festes på røret. Hentet fra [8].

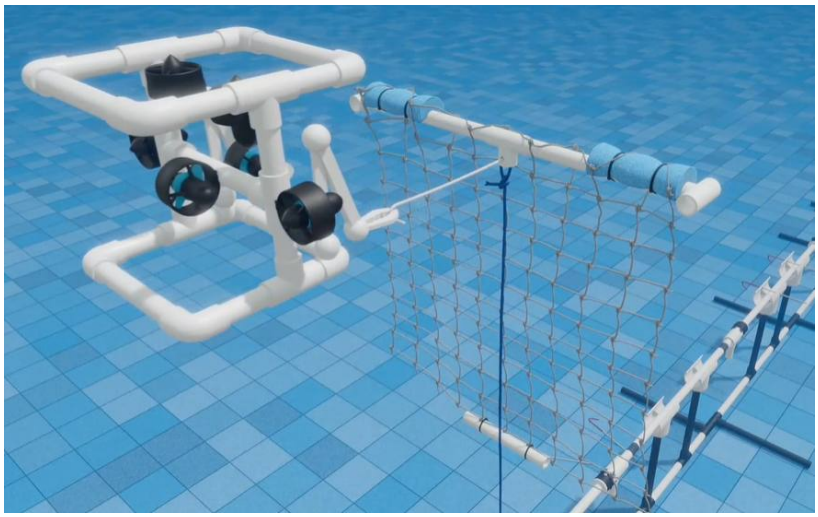
- Fjerne den skadede oppdriftsmodulen ved å rotere festet 180 grader: **5 poeng**.
- Frakte den skadede oppdriftsmodulen til kanten av bassenget, slik at den kan plukkes opp fra land: **5 poeng**.
- Plassere en ny oppdriftsmodul på kabelen: **5 poeng**.
- Feste oppdriftsmodulen ved hjelp av borrelås: **5 poeng**.

1.3 Overvåke miljøet

Figur 1.9a viser hvordan første del av oppgaven kan løses. Figur 1.9b viser hvordan andre del av oppgaven kan løses.



(a) Utplassering av hydrofon. Området på 40 cm × 40 cm er representert av de oransje PVC-rørene. Hentet fra [8].

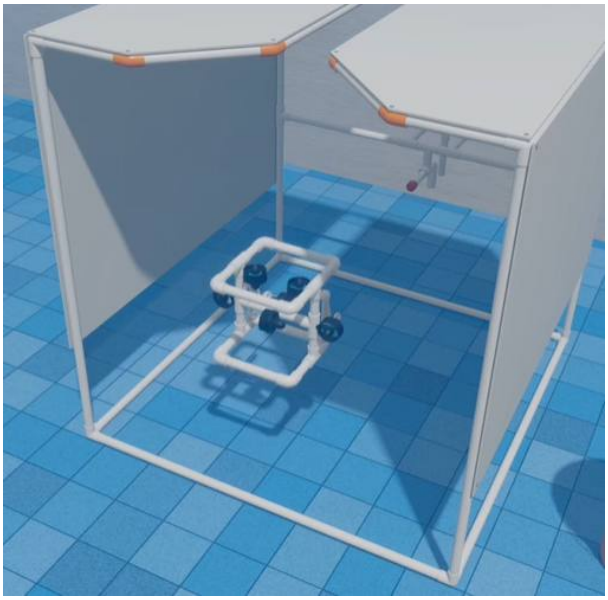


(b) Fjerning av fiskegarn. Garnet vil flyte opp til vannoverflaten når låsespinnen dras ut. Hentet fra [8].

- Utplassere en hydrofon, simulert av et PVC-rør, for å oppdage og registrere tilstedeværelsen av sjøpattedyr.
  - Plassere hydrofonen i et angitt område på 40 $cm \times 40\ cm$: **5 poeng**.
  - Plukke opp hydrofonen etter 5 minutter i vannet og frakte den til bassengkanten: **5 poeng**.
- Fjerne et fiskegarn som er fanget på vindturbinens understell.
  - Dra ut en låsepinne som holder en flytende ramme med fiskegarn under vann: **10 poeng**.
  - Fjerne fiskegarnet fra vannet ved å frakte det til bassengkanten, og plukke det opp fra land: **10 poeng**.

1.4 Styre ROV-en inn i en dokking-stasjon

Figur 1.10 viser hvordan oppgaven kan løses.



Figur 1.10: Parkering av ROV i dokking-stasjon. Hentet fra [8].

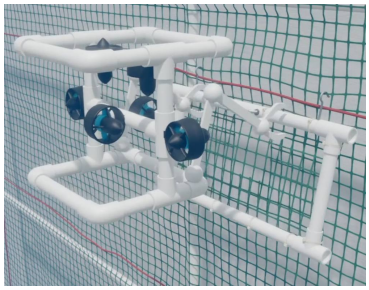
- Styre ROV-en autonomt inn i dokking-stasjonen: **15 poeng**.
- Styre ROV-en manuelt inn i dokking-stasjonen: **5 poeng**.

Oppgave 2: Offshore akvakultur og blå karbon
FNs bærekraftsmål:
#2 Utrydde sult
#13 Stoppe klimaendringene
#14 Liv under vann

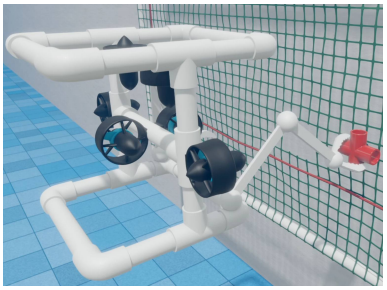
Oppgaven er utformet for å representere arbeidsoppgaver som er nødvendige for å utføre vedlikehold på en fiskemerd, og å bidra til en sunn havhelse. Man skal blant annet reparere en skadet del av en fiskemerd, og fjerne uønsket algevekst. Oppgaven innebærer også å autonomt skille døde fisk fra levende, og å kunne måle størrelsen på fisken. Utover dette skal dødt sjøgress fjernes, og det skal videre plantes nytt. Alle arbeidsoppgavene er simulert ved hjelp av konstruksjoner laget av PVC-rør. Under er oppgavebeskrivelsene gjengitt, med tilhørende poengfordeling.

2.1 Inspisere en offshore akvakultur fiskemerd

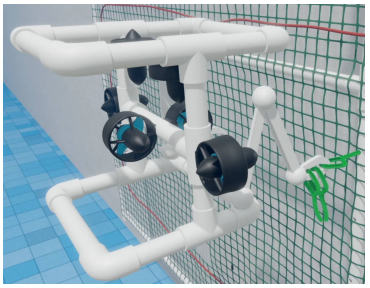
Figur 1.11 viser hvordan noen av deloppgavene kan løses.



(a) Reparasjon av skadet del av nettet.



(b) Fjerne innkapslende marin vekst.



(c) Fjerne algevekst.

Figur 1.11: Alle hentet fra [8].

- Inspisere nettet for å identifisere skadede områder.
  - Kjøre en transektlinje for å identifisere skadede områder: Autonomt: **25 poeng**. Manuelt: **10 poeng**.
  - Identifisere og telle skadede områder av nettet: **5 poeng**.
- Reparere en skadet del av nettet: **10 poeng**.
- Fjerne marin vekst.
  - Fjerne innkapslende marin vekst, simulert med et PVC-kryss: **5 poeng**.
  - Fjerne algevekst, simulert med 3 piperensere: **5 poeng**.

2.2 Opprettholde et sunt miljø

Oppgaven som innebærer oppsamling av død fisk er vist i figur 1.12.



Figur 1.12: Plassering av død fisk i oppsamlingsrør. Hentet fra [8].

- Håndtere fiskedød ved å fjerne død fisk, simulert av en gummifisk, fra bunnen av bassenget.

- Lage et program som kan skille døde fra levende fisk. Man får utdelt en video, og programmet skal merke døde fisk med røde rammer: **10 poeng**.
- Plukke opp en død fisk: **5 poeng**.
- Legge fisken i et oppsamlingsrør, simulert av en bøtte: **5 poeng**.

2.3 Måle fiskestørrelser

Et eksempel på hvordan man kan løse denne oppgaven er vist i figur 1.13.

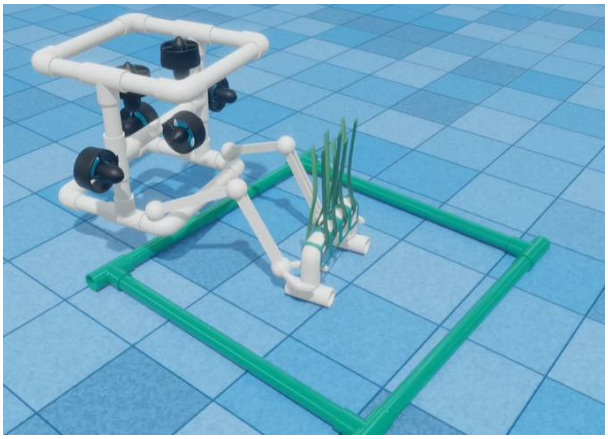


Figur 1.13: Måling av fiskelengde. Hentet fra [8].

- Beregne gjennomsnittsstørrelsen på en fiskekohort (tre gummifisker av ulik lengde) med maksimalt 2 cm feilmargin: **15 poeng**.
- Beregne biomassen av fiskekohorten: **5 poeng**.

2.4 Dyrke sjøgress

Planting av nytt sjøgress er vist i figur 1.14.



Figur 1.14: Planting av nytt sjøgress. Sjøgresset som skal lukes er simulert av en lignende konstruksjon, men denne skal fjernes fra det markerte området. Hentet fra [8].

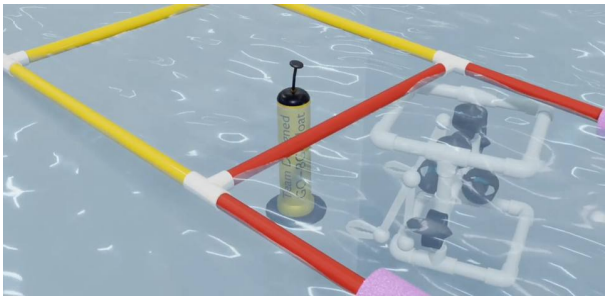
- Luke et eksisterende sjøgressbed: **5 poeng**.
- Plante et nytt sjøgressbed: **5 poeng**.

Oppgave 3: Da og nå – *Endurance22* og *MATE Floats*!
FNs bærekraftsmål:
#13 Stoppe klimaendringene

Første del av oppgaven representerer det å hente inn en «GO-BGC»-flyter, for så å plassere ut vår egenproduserte flyter i et angitt område. Flyteren vår skal deretter gjennomføre 2 vertikale profiler, altså traversere dybden av bassenget to ganger. Andre del av oppgaven handler om å kartlegge posisjonen til vraket av skipet *Endurance*, som sank i Antarktis. Deretter skal det autonomt lages en fotomosaikk av vraket, og videre skal lengden av det måles. Både «GO-BGC»-flyteren og vraket av Endurance er simulert ved hjelp av PVC-rør. Under er oppgavebeskrivelsene gjengitt, med tilhørende poengfordeling.

3.1 MATE Floats!

Utplassering av flyter i et angitt område er vist i figur 1.15.



Figur 1.15: Plassering av selvlagd flyter i angitt område. Hentet fra [8].

- Hente inn en *GO-BGC* flyter.
  - Beregne hvor flyteren dukker opp fra vannet: **5 poeng**.
  - Plukke opp flyteren, simulert av et PVC-rør: **10 poeng**.
- Design og konstruksjon av en operativ flyter for vertikal profilering.
  - Bygge en flyter før konkurransen starter: **5 poeng**.
  - Plassere flyteren i et angitt område ved hjelp av ROV-en: **5 poeng**.
  - Flyteren fullfører vertikal profilering.
    - To ganger: **25 poeng**.
    - Én gang: **15 poeng**.

3.2 Endurance22

Kartlegging av området der vraket av *Endurance* ble funnet, se figur 1.16.



Figur 1.16: Området der vraket av Endurance er plassert. Vraket er simulert med den brune PVC-strukturen. Hentet fra [9].

- Finne og kartlegge posisjonen til Endurance.
  - Kjøre en transektlinje over området der vraket er. Vraket er simulert av PVC-rør: **10 poeng**.
  - Indikere på et kart hvor vraket er: **5 poeng**.
- Lage en fotomosaikk av vraket.
  - Ta bilder av alle delene av vraket: **5 poeng**.
  - Autonomt sette bildene sammen til en fotomosaikk: **20 poeng**.
  - Manuelt sette bildene sammen til en fotomosaikk: **10 poeng**.
- Måle lengden av vraket fra baug til akterende.
  - Innen 10 cm av den sanne lengden: **10 poeng**.
  - Innen 10.1 til 20 cm av den sanne lengden: **5 poeng**.

Videre vil ROV-en veies, og belønnes med 10 ekstrapoeng dersom egenvekt i luft er mindre enn 20 kg, og 5 ekstrapoeng med egenvekt under 28 kg. Det deles ikke ut poeng dersom ROV-en er tyngre enn 28 kg, og veies den til over 35 kg resulterer det i diskvalifikasjon. Nytt for året er at det ikke tas størrelsesmål av farkosten. Likevel begrenses størrelsen på ROV-en av dokking-stasjonen på én kubikkmeter.

Oversikt over poengfordeling

Det gås ikke inn på detaljer rundt poenggivning i forhold til dokumentasjon og sikkerhet, da det ikke har betydning for den tekniske delen av prosjektet. Tabellen under viser en oppsummering av den totale poengfordelingen i konkurransen.

Produktdemonstrasjoner	
Oppgave 1-3	300 + tidsbonus
Vektrestriksjoner	10
Organisatorisk effektivitet	10
Prosjektering og kommunikasjon	
Teknisk dokumentasjon	100
Produktpresentasjon	100
Markedsføring	50
Bedriftens spesifikasjonsark	20
Bedriftsansvar	20
Bruk av VR-utstyr	25
Sikkerhet	
Gjennomgang av sikkerhet og dokumentasjon	20
Sikkerhetsinspeksjon	30
Sikker jobbanalyse	10
Totalt	695 + tidsbonus

Begrensninger og restriksjoner

Fysiske begrensninger for ROV-en omfatter ikke flyteren, da denne regnes som en egen enhet.

I utforskerklassen spesifiseres det at deltakere skal designe en ROV, uten eksterne komponenter, for å løse de praktiske oppgavene. Alle deler og komponenter skal være festet i ROV-ens ramme. Thrusterene[1] skal ha beskyttelse som ikke tillater fingre å berøre propellene. Farkosten skal operere i kloret ferskvann med temperaturer i området 15 ^{o}C-30 ^{o}C. Det nevnes også at uforutsette strømninger i vannet kan oppstå på grunn av bassengets filtreringssystem. Bassenget oppgis til å være maksimalt 6 meter dypt, og alle oppgaver utføres innenfor 10 meter fra bassengkanten. Stasjonen for skjermer og styresystem vil maksimalt være plassert 3 meter fra bassenget. Lengde på navlestreng må altså beregnes etter disse målene. Vinsj eller kran for sjøsetting er ikke tillatt. Dette utføres manuelt.

Arrangøren stiller med kraftforsyning, og dette er den eneste forsyningen man har tilgang til. Ekstra batterier og lignende tillates ikke. Kraftforsyningen leverer en nominell spenning på 48 VDC. I år er også deltakere pålagt å bruke en av MATE sine elektriske sikringer på 20 A, 25 A eller 30 A. Dette begrenser altså effektforbruket til 1440 W. All spenningsregulering skal foregå i elektronikkhuset til ROV-en, og kan ikke overgå 48 VDC. I tillegg er det oppgitt at koblinger og motorer som er eksponert i vann skal vanntettes, og ha en minimum resistans på 10 MΩ til vannet.

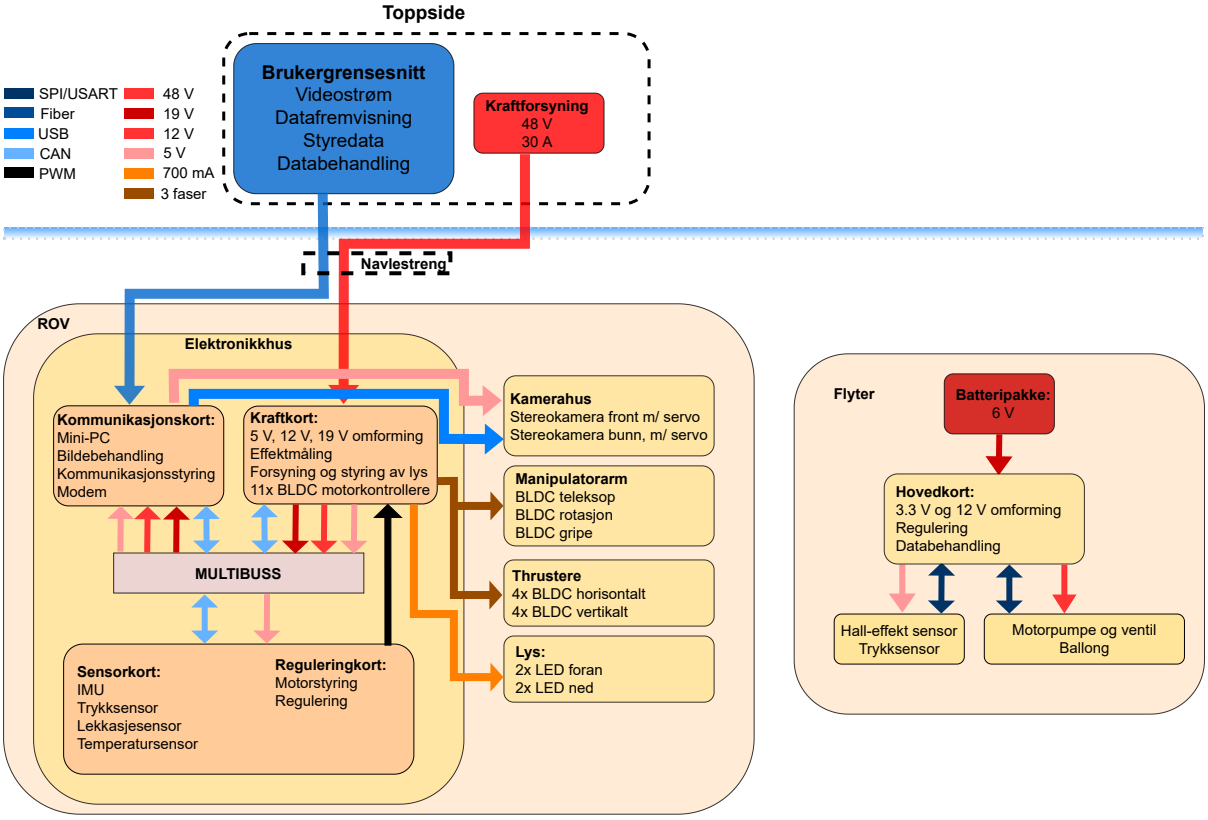
Flyteren skal være bygget fullstendig adskilt fra ROV-en, og skal fungere som en egen enhet. Størrelsen på flyteren er begrenset til 1 m i høyde, og 18 cm i diameter. Batterier ombord brukes som kraftforsyning. Her angir arrangøren en maksimal spenning på 12 VDC, og maksimalt strømtrekk på 6 A. Det kreves i tillegg en sikring på 7.5 A, som maksimalt kan plasseres 5 cm fra positiv pol på batteripakken.

[1] motorene på ROV-en som generer framdrift ved hjelp av propeller.

1.4 Overordnet system

Figur 1.17 viser et blokkskjema av det overordnede systemet. ROV-en styres fra et brukergrensesnitt på toppsiden. Herfra sendes det styredata, i tillegg til at det vises videostrøm og status fra ROV-en. Kraftforsyning og fiberkommunikasjon fra toppsiden går gjennom en navlestreng. Kraftforsyningen går direkte inn på kraftkortet. Her foregår omforming til nødvendige spenningsnivåer, og distribuering av disse. Lys, motorer og motorkontrollere forsynes direkte fra kraftkortet. Fiberkommunikasjonen går inn på en mini-PC, som behandler data og organiserer kommunikasjonen i systemet. Kommunikasjonssignalene går inn på et kommunikasjonskort, via USB, og sendes videre over den lokale CAN-bussen. Mini-PC-en er også koblet direkte til kameraene, og driver kontinuerlig bildebehandling. Både kommunikasjon og kraftforsyning sendes ut til det interne systemet i elektronikkhuset, via en felles multibuss. Det siste kortet er et kombinert sensor- og reguleringskort. Sensordelen av kortet leser og behandler data fra de ulike sensorene i systemet. Reguleringsdelen sørger for styring av thrustere og manipulatormotorer basert på styredata fra toppsiden. I tillegg utføres regulering av thrustere fra sensordataen. Pådragsdata sendes videre via motorkontrollerne på kraftkortet, og ut til aktuelle motorer.

Flyteren drives av en batteripakke på 6 V, som går direkte inn på et hovedkort. Her omformes spenningen fra 6 V til både 3.3 og 12 V. I tillegg gjennomføres databehandling og regulering i mikrokontrolleren på kortet. Sensordataen hentes fra to eksterne sensorkort, som inkluderer én hall-effekt-sensor, og én trykksensor. Mikrokontrolleren sender også styredata til motorpumpen og luftventilen, som sammen med en ballong utgjør oppdriftsmotoren.



Figur 1.17: Overordnet blokkskjema av ROV-en.

1.5 Prosjektinndeling

Prosjektet er delt opp i flere oppgaver, hvor hver bachelorgruppe er tildelt et eget ansvarsområde. Totalt utgjør dette 9 bacheloroppgaver, hvor 5 av disse er innen elektro, 2 innen maskin, 1 innen data og 1 innen økonomi. Nedenfor presenteres hver oppgave i korte trekk, med ansvarsområde og gruppemedlemmer.

1.5.1 Elektro

Kraftforsyningsmodul – Carl Henrik Preber Ettesvoll, Nicolai Jensen Narvesen og Jon Arve Andersen:

Oppgaven handler om å utvikle en kraftforsyningsmodul til ROV-en. I konkurransen skal ROV-en få elektrisk forsyning fra land via en navlestreng. Konkurransemanualen oppgir at det blir tilført 48 V spenning, og at strømbegrensningen er på 30 A. Kortet skal sørge for at all elektronikk blir tilført riktig spenningsnivå, og at effektforbruk overvåkes der det er nødvendig. Det skal også kunne prioritere kritiske prosesser, ved hjelp av digitalt styrte sikringskretser. Kraftkortet må ha rask responstid, kombinert med et robust design.

Maskinsyn og kommunikasjon – Christoffer Næss, Mats Røste og Tage Mellemstrand:

Oppgaven er todelt. Maskinsyn-delen går ut på å hente ut informasjon fra stereokameraene. Dette innebærer blant annet avstands- og størrelsesmåling, gjenkjenning av objekter og nødvendig informasjon for å løse de autonome oppgavene. Kommunikasjonsdelen går ut på å lage et robust kommunikasjonssystem i samarbeid med brukergrensesnittgruppen. Systemet skal kunne motta kommandoer fra toppsiden, behandle disse og sende dem videre til mikrokontrollere, som styrer de lokale prosessene i ROV-en. I tillegg skal systemet hente bilder fra kameraer, og sender dem tilbake til toppsiden, med så lite forsinkelse som mulig.

Sensorkort – Jørgen Hemnes Johannessen:

Oppgaven handler om å utvikle et sensorsystem til ROV-en. Dette innebærer å hente inn rådata fra sensorer, behandle dem og distribuere dem videre over kommunikasjonssystemet. Sensordata som behøves til reguleringen, er vinkelhastighet og -posisjon, samt dybden ROV-en befinnner seg på. Det må i tillegg være mulig å detektere kritiske feil, som for eksempel vannlekkasje eller overoppheting. For å realisere systemet, må det utvikles et kretskort med en mikrokontroller som sørger for tilfredsstillende databehandling. Vinkeldata leses fra en *Inertial Measurement Unit* (IMU) som settes på kortet. Resterende data leses fra eksterne sensorer. Dette innebærer trykk- og temperaturmåling, i tillegg til lekkasjedeteksjon.

Styrings- og reguleringssystem – Tomas Royal Choat, Kristian Birkeland og Otto Nessa Ljosdal:

Oppgaven går ut på å utvikle et motorsystem for ROV-en. Dette består av thrustere for manøvrering, samt elektriske motorer for bevegelse av manipulatorarmen. Det skal utvikles et robust styringssystem for alle motorer. Dette må kunne motta manuell styredata fra toppsiden, eller autonom styredata fra min-PC-en, og generere motorpådrag etter ønsket oppførsel. For å forenkle manøvreringen, skal det i tillegg utvikles et reguleringssystem, bestående av tre PID-regulatorer. Disse skal sørge for at ROV-en automatisk kan holdes ved en ønsket dybde, og at den ligger stabilt i vannet. Sistnevnte oppnås ved å motvirke rotasjon fremover, bakover og sideveis.

Utvikling av smart flyter – Malin Harr Overland og Hanne Lovise Berger:

Oppgaven går ut på å utvikle en trådløs flyter, som automatisk kan manøvreres i vertikal retning. Manøvreringen realiseres ved hjelp av et styresystem, og en oppdriftsmotor. Motoren består av en pumpe og en solenoidventil, som endrer oppdriften ved å forflytte luft fra innsiden av flyteren, og ut i en ballong. For at styresystemet skal vite når motoren må aktiveres, kreves et sensorsystem. Dette inkluderer en trykkmåler for avlesing av dybdeendring, og en hall-effektsensor for å detektere om flyteren holdes av ROV-en. Flyteren skal i tillegg drives på batterier, og systemet må derfor tilpasse spenningsnivå til de ulike formålene. Dette gjøres på et hovedkort, som også sørger for tilfredsstillende styring og databehandling.

1.5.2 Maskin

Design og montering av ROV og flyter - Christine Nordal og Sandra Nygård:

Oppgaven går ut på å designe og bygge rammen til ROV-en, samt den indre og ytre konstruksjonen til flyteren, ved å følge produktutviklingsprosessen. Den indre konstruksjonen til flyteren inkluderer oppdriftssystemet. Fokuset ved designet er å sikre at farkostene er godt egnet til å utføre oppgavene i konkurransen. I år rettes det også et økt fokus mot bærekraft og miljø. Det vil derfor bli lagt vekt på konseptet *Design for environment* (DfE) i utviklingsprosessen. Målet er å redusere miljøpåvirkningen i utviklingen av produktene. Effektiv bruk av DfE kan også bidra til reduserte kostnader og produksjonstid, i tillegg til å øke kvaliteten på produktet. Oppgaven inneholder også materialvalg, dimensjonering, strukturell analyse, flytanalyse og beregning av oppdrift og stabilitet. Dette for å sikre at produktene er stabile, har riktig oppdrift og er lette å manøvrere.

Design og produksjon av manipulator – Henrik Welde og Sindre Rød Torsteinsen:

Oppgaven går ut på å designe en manipulator som er i stand til å gripe tak i objekter. Fokuset for designet er å gjøre den så enkel som mulig, uten at dette går på bekostning av funksjonaliteten. Dette vil forenkle demontering, i tilfelle vedlikehold må utføres. Samtidig skal det tilstrebes å produsere armen så lett som mulig, da redusert vekt vil bidra til både bedre manøvreringsegenskaper, og poeng i konkurransen. Klypen må være i stand til å gripe objekter av forskjellige størrelser og former. I tillegg skal den ha mulighet for rotasjon og teleskopbevegelse.

1.5.3 Data

Operatørgrensesnitt og kommunikasjon – Vebjørn Lia Riiser og Åse Jortveit Sagebakken:

Oppgaven som skal løses, er å utvikle et system for å styre og overvåke ROV-en. Det må derfor implementeres et system for å sende kommandoer og styringsdata fra toppsiden til mini-PC-en i ROV-en. Dette gjøres i samarbeid med **Maskinsyn**-gruppen. I tillegg må det designes et brukergrensesnitt som presenterer videostrøm og sensordata i sanntid. Hovedfokuset for grensesnittet er brukervennlighet, som innebærer fremvisning av nødvendig informasjon på en oversiktlig måte.

1.5.4 Økonomi

Endringsprosess i UiS Subsea – Maren Lovise Jåsund, Sina Brunes og Sanna Sørskår:

UiS Subsea er i år utvidet ved å involvere en gruppe fra økonomi og administrasjon. Målene

fremover er blant annet å utvide organisasjonen til å involvere flere fagfelt, og ha flere pågående prosjekter samtidig. Ved hjelp av John P. Kotters 8-stegs modell, skal gruppen se på hvordan UiS Subsea på best mulig måte kan fortsette endringsprosessen av organisasjonen. Hovedsakelig innebærer dette organisering av de økonomiske og administrative oppgavene. UiS Subsea er hovedkilden. For å få et større perspektiv på økonomi og administrasjon, intervjues i tillegg tre andre bedrifter som organiserer disse oppgavene på forskjellige måter. Innhentet informasjon legger grunnlaget for videre analyser av hvordan UiS Subsea på best mulig måte kan organiseres med flere medlemmer og prosjekter. De strategiske analysene som skal gjennomføres, er *SWOT*-analyse og strategilerret. Analysemetodene vil gi god innsikt i bedriftene, og et godt grunnlag for å sammenligne dem.

1.6 Operatørgrensesnitt og *topside*-kommunikasjon

I dette kapittelet gis et mer detaljert innblikk i systemet denne bacheloroppgaven omhandler.

1.6.1 Problemstilling

Hovedmålet for oppgaven er å lage *topside*-systemet til ROV-en som UiS Subsea skal bygge for å delta i MATE ROV-konkurransen. Det går ut på at gruppen må ta seg av tre hovedansvarsområder:

- Lage en løsning for kommunikasjonen mellom *topside* og ROV-en.
- Håndtere inndata fra kontrolleren som skal styre ROV-en.
- Lage et brukergrensesnitt til ROV-en.

Det er ikke satt noe direkte krav til ROV-operatørgrensesnittet i MATE-konkurransemanualen [10], men siden det er en tidsbegrensning i konkurransen, er det et implisitt krav at operatørgrensesnittet er brukervennlig, og at ROV-operatøren ikke bruker ekstra tid på grunn av et dårlig system.

Gruppen må derfor lage et operatørgrensesnitt som er intuitivt og brukervennlig. Videre er det nødvendig at programmet og kommunikasjonen er stabil, og at programmet utfører handlingene operatøren ønsker, slik at tid i konkurransen ikke går med på problemer og feilsøking.

I tillegg må det lages en felles standard for sending av data mellom ROV-en og *topside* i samarbeid med **Maskinsyn**-gruppen [1.5.1], for at ROV-en skal få tilsendt kommandoene den skal utføre, som for eksempel å kjøre autonomt. Et annet problem som må løses er å oppnå kort forsinkelse ved framvising av video og sensordata, i tillegg til hurtig respons mellom trykk på kontrolleren og handling, som for eksempel bevegelse av ROV-en. Vårt ansvar for informasjonsflyt nedover, stopper etter at pakkene er sendt fra programmet vårt. Oppover har vår gruppe i tillegg ansvar for forsinkelsen av videostrøm.

1.6.2 Samarbeid mellom gruppene

Hva som vises fram i brukergrensesnittet er i store deler basert på hvilke sensordata som er tilgjengelig og ønskelig å vise fram fra ROV-en. Dette defineres sammen med de ulike gruppene

som har ansvar for sensorene på ROV-en. Derfor kan det være en utfordring å definere akkurat hvordan brukergrensesnittet skal se ut før de andre gruppene har bestemt seg for hvilke sensordata og funksjonalitet de ønsker å ha tilgjengelig. En stor del av oppgaven går derfor ut på å samarbeide og tilpasse seg hva de andre gruppene gjør, for å kunne realisere prosjektet.

1.6.3 Motivasjon

En motiverende faktor for prosjektet er muligheten til å kunne delta i MATE ROV-konkurransen med UiS Subsea sin egenlagde ROV. MATE-konkurransen går ut på at UiS Subsea-medlemmene skal se på seg selv som entreprenører der man jobber som en integrert del av en arbeidsgruppe, tenker kritisk og anvender teknisk kunnskap og ferdigheter på en ny og innovativ måte [10, side 3]. Med andre ord så får medlemmene erfaring med å jobbe i en større gruppe, der man må samarbeide med andre studieretninger. Dette vil være en god egenskap å kunne ta med seg inn i arbeidslivet etter endt bachelor. Det finnes mange bedrifter som driver med undervannsteknologi i Stavanger, og UiS Subsea-medlemmene kan bli sett på som attraktive på arbeidsmarkedet ettersom vi har fått erfaring med et slikt prosjekt.

I tillegg gir prosjektet et godt innblikk i hvordan det er å jobbe med elektronikk og hardware, og vil gi nyttig kunnskap om hvordan designe et godt brukergrensesnitt. Vi får erfart hvordan teorikunnskap lært i løpet av studiet fungerer i et praktisk prosjekt. Til slutt vil man ende opp med et fysisk produkt som belønning. Ønsket er å få laget et så godt resultat som mulig, for å sikre oss en god plassering i MATE ROV-konkurransen.

1.6.4 Hva slags system skal vi lage?

Gruppen skal lage *topside*-systemet for ROV-en. En del av dette går ut på å lage en løsning for datakommunikasjonen. Denne kommunikasjonen består av videostrømming, mottak av sensordata fra ROV-en og sending av instrukser fra kontrolleren og brukergrensesnittet. Videre må gruppen samarbeide med mottakeren av dataen for å lage en standard som begge sider forstår. Overføringen av videostrøm må skje med minst mulig forsinkelse fra den blir tatt opp og til den vises på skjermen.

Informasjonen som sendes opp fra ROV-en er blant annet data fra temperatur-, trykk, lekkasjesensor, stampvinkel, rullvinkel[2] og effektforbruk[3]. Pådraget som reguleringsgruppen sender til motorene inngår også i vår definisjon av sensordata.

Topside-systemet består også av et brukergrensesnitt som skal visualisere de forskjellige sensordataene, verdier fra motorpådraget og vise fram sanntidsvideo fra kameraene til ROV-en. Det skal også være mulig å sende ned kommandoer fra GUI-en til ROV-en. Verdier fra kontrolleren som skal styre fartøyet, dens forskjellige funksjoner må bli behandlet og sendt videre til ROV-en.

[2] stamp- og rullvinkel kommer fra et gyroskop og et akselerometer på en IMU-enhet.

[3] effektforbruket kommer fra hall-effekt-sensoren som måler magnetfeltet og dermed kan gi ut effektforbruk for de tre hovedkretsene i ROV-en.

1.6.5 Oversikt over hva som skal gjøres

Det blir gjennomgått hva som skal gjøres for at *topside*-systemet skal fungere som tiltenkt.

Overføring av sensordata og videostrøm

- En nettverksforbindelse skal lages mellom ROV-en og *topside*-serveren, i samarbeid med **Maskinsyn og kommunikasjon**-gruppen [1.5.1].
- Videostrøm skal fremvises i brukergrensesnittet, mens prosesseringen av videostrømmen skal gjøres av **Maskinsyn-gruppen**.
- Sensordata og motorpådragsverdier, som sendes fra ROV, skal mottas og visualiseres i brukergrensesnittet.
- Data som mottas skal logges i en tekstfil, slik at det er mulig å feilsøke under testing eller å analysere den i ettertid.

Styrekontroller

- Inndata fra kontrolleren skal hentes ut.
- Kontroller-oppsettet skal bestemmes.
- Styringsdata fra kontroller skal formateres til avtalt format og sendes ned til ROV. Styringsdata videresendes på CAN-bussen, men dette skal gjøres av **Maskinsyn**-gruppen.
- Et system skal lages for å ha mulighet til å velge forskjellige kontrollerprofiler, slik at man kan rekonfigurere knappene på kontrolleren i brukergrensesnittet.
- Kontrolleren skal kunne tilkobles på ny uten at det er nødvendig å restarte programmet.

Brukergrensesnitt

- Brukergrensesnittet skal lages visuelt og brukervennlig.
- Videostrøm skal fremvises i brukergrensesnittet, og videostrømmen kan gjerne ha relevant informasjon om hastighet, dybde og modus.
- Det skal være mulig å ta stillbilder av videostrøm, og starte videoopptak på de forskjellige kameraene. Bilder og video skal kunne lagres.
- Det skal være mulig å skifte mellom ulike kjøremodus. For eksempel manuell kjøring, autonom parkering eller bildebehandlingmodus for å detektere fisk.
- Feilmeldinger skal kunne vises fram.
- Skal være mulig å skru på eller av ROV-ens lys. Lysstyrken skal også kunne stilles på.

  Informasjon som skal vises fram i brukergrensesnittet:

  - Dybde fra overflaten av vannet til ROV. Sensorgruppen skal bruke en trykksensor til å måle dybde, og nullpunktet skal settes når *topside*-programmet sender ned melding til ROV-en.
  - Orientering til ROV. Det skal mottas 4 frihetsgrader; jag, hiv, svai og gir.
  - Temperatur inni ROV. Varsel dersom det blir overoppheting.

- Varsel i tilfelle vannlekkasje.
- Effektforbruk fra thrustere, manipulator og generell elektronikk i ROV-en. Advarsel vises dersom effektforbruket overstiges eller ligger høyt.
- På/av-status på strømregulatorer for elektronikk, thrustere og manipulator.
- ROV-ens orientering og dybdeverdier.
- Tidtaker fra man starter ROV-en.
- Pådragsverdier til ROV-ens thrustere.
- Motorpådrag til ROV-ens 4 vannrette og 4 loddrette thrustere.
- Motorpådrag til manipulatorens tre motorer.
- På/av-status til manipulatoren.
- Hvis tiden strekker til, kan det vises fram en 3D-modell av ROV-ens orientering i vannet.

Kapittel 2

Teori og bakgrunn

Kapitteloversikt

2.1 Teknologier og protokoller . . . 23
2.1.1 Kommunikasjon . . . 23
2.1.2 Operativsystem-begreper . . . 26
2.2 Formater . . . 27
2.3 Vektorgrafikk . . . 27
2.4 Utviklingsmetodikk og arbeidsmetode . . . 29
2.4.1 Git . . . 29
2.4.2 Utviklingsmetodikk . . . 29
2.4.3 Github-prosjekttavle . . . 29
2.4.4 Møter . . . 30
2.4.5 Modularitet i programmet . . . 30
2.5 Datasikkerhet . . . 31
2.6 Programmeringsspråk . . . 31
2.6.1 Python . . . 31
2.6.2 Go . . . 31
2.6.3 C# . . . 32
2.6.4 Programmeringsspråk brukt i subsea-industrien . . . 32
2.6.5 Valg av programmeringsspråk . . . 32
2.7 Rammeverk brukt i operatørgrensesnittet . . . 34
2.7.1 Qt . . . 34
2.7.2 PyQt . . . 34
2.8 Datamaskin og oppbevaringsløsning . . . 40
2.8.1 Valg av *topside*-datamaskin . . . 40
2.9 Styrespak . . . 41
2.9.1 Valg av styrespak . . . 41
2.9.2 Test av styrespak . . . 42
2.10 Styring av ROV-en . . . 43
2.10.1 Styring av manipulatoren . . . 44
2.11 Xbox-kontrolleren . . . 44
2.11.1 Kontroll-funksjoner . . . 44
2.11.2 Standardoppsett . . . 44

2.11.3 De analoge styrespakene 45
2.11.4 Forskjellen på ønsket pådrag fra kontrolleren og faktisk pådrag til motorene 46
2.11.5 Bibliotek til håndtering av kontrollerinndata 46
2.12 Kommunikasjon mellom ROV og *topside* 46
2.12.1 Kommunikasjonsmedium 46
2.12.2 Valg av kommunikasjonsmedium 47
2.12.3 Test av fiberkabelens overføringskapasitet 47
2.12.4 Mini-PC i ROV-en 48
2.13 ROV-ens kamera 48
2.13.1 Plassering og manøvrering av kameraene 49
2.13.2 Belysning 49
2.13.3 Lyd fra motorene 49
2.14 Logging av sensordata 50
2.15 Brukergrensesnitt i subsea-industrien 50
2.16 Fjorårets brukergrensesnitt 50

I dette kapittelet blir det presentert teori om teknologier og protokoller som er relevante for oppgaven. Vurderinger som er blitt gjort og valg som er blitt tatt blir også gjennomgått. Det har blitt gjort valg av programmeringsspråk, *topside*-datamaskin, styrespak, kontroller-oppsett og hvordan kommunikasjonen mellom ROV og *topside* skal foregå. Det blir innimellom sett på hva som er blitt gjort på tidligere ROV-prosjekt i UiS Subsea.

2.1 Teknologier og protokoller

Først blir det gjennomgått teknologier og protokoller innenfor kommunikasjon. Det blir også gått gjennom de ulike teknologiene brukt for oppbygning av systemet, og noen av formatene som blir benyttet.

For datastudenter som har hatt fagene DAT230 Kommunikasjonsteknologi 1 og DAT320 Operativsystemer og systemprogrammering er mesteparten av kapitlene **2.1.1 Kommunikasjon** og **2.1.2 Operativsystem-begreper** allerede er kjent. Vi velger likevel å gå gjennom noe av den grunnleggende teorien. Tidligere har det vært elektrostudenter i UiS Subsea, som har hatt ansvar for *topside*-kommunikasjonen og brukergrensesnittet til ROV-en. Dersom det blir elektrostudenter som har ansvaret for det i de kommende årene, kan det være relevant for dem å lese denne teorien.

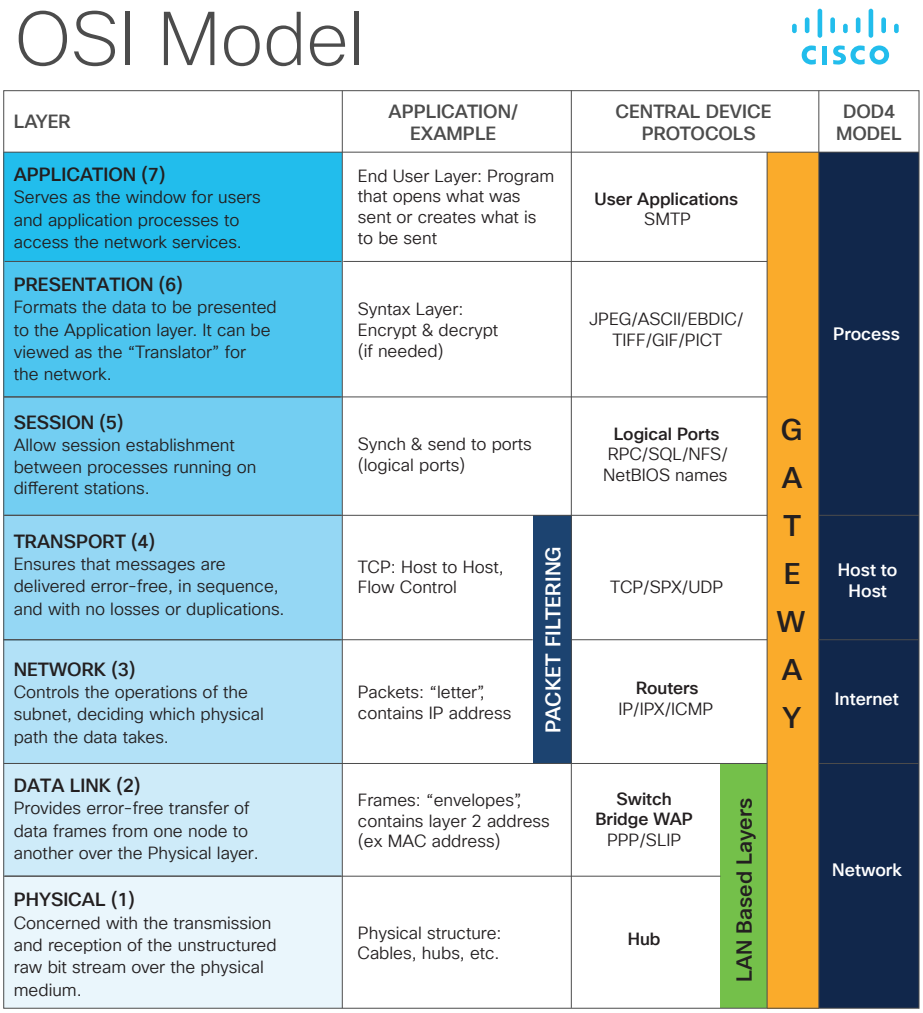
2.1.1 Kommunikasjon

Open Systems Interconnection model (OSI-modellen) OSI-modellen forklarer den logiske oppbygningen til et datanettverk [11]. Modellen består av syv lag, hvor de øvre lagene benytter abstraksjonen i lagene under i sin implementasjon.

Det er flere fordeler ved å abstrahere bort detaljene i lag. Man slipper for eksempel å lage implementasjoner på de øverste lagene for hvert kommunikasjonsmedium siden dette er fastsatt i det fysiske laget (lag 1). Det vil si at man kan bruke den samme implementasjonen av TCP, fra transportlaget (lag 4), uavhengig av om man bruker radiobølger, lys eller kobber for å overføre

informasjonen. Dette gjør at de som er eksperter på et felt kan jobbe med å optimalisere sin del uten å ha inngående kunnskap om de andre.

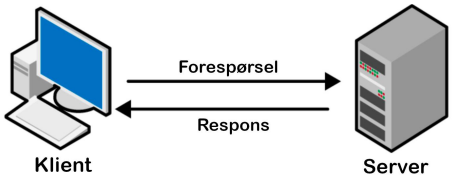
Vår oppgave går blant annet ut på å sette opp et datanettverk for kommunikasjon mellom ROV og *topside*, det er derfor en fordel å forstå hvordan kommunikasjon foregår på de ulike lagene, og forholdet mellom dem. Se figur 2.1 nedenfor for beskrivelse av hvert lag i modellen.



Figur 2.1: Figuren viser OSI-modellen som består av 7 lag. Oversikten er hentet fra [12]

.

Klient-server-modellen Klient-server-modellen vil si at en side spør om data (klienten) og en side svarer på denne forespørselen om data (serveren), illustrert i figur 2.2. Flere av teknologiene som brukes i oppgaven er representert i denne modellen, blant annet TCP.



Figur 2.2: Klient-server-modellen.

Transmission Control Protocol (TCP) TCP er den mest brukte protokollen i transportlaget (lag 4) i dag. TCP følger klient-server-modellen der serveren venter på at klienten skal ta kontakt slik at de kan opprette en kobling. Opprettelsen av denne koblingen kalles TCP-*handshake* [13].

Hovedfordelen med TCP over UDP er at den håndterer tap av pakker og sikrer at pakkene blir satt sammen igjen i riktig rekkefølge. Med pakker menes den grunnlegende dataenheten i nettverkslaget (lag 3). TCP er altså pålitelig i motsetning til UDP som er *best-effort*.

TCP er en stor fordel i situasjoner hvor man må være sikker på at all informasjon kommer fram og er i riktig rekkefølge. Hvis man ikke bruker TCP, men trenger dette må man enten bruke en annen protokoll som implementerer resending av pakker og sortering i riktig rekkefølge, eller gjøre dette selv i egen kode [14].

User Datagram Protocol (UDP) UDP er også en del av transportlaget. Men UDP er *best-effort* og ikke pålitelig. Det vil si at den ikke passer på at alle pakkene kommer fram til mottakeren. Den sørger heller ikke for at pakkene kommer fram i samme rekkefølge, så dette må derfor implementeres på toppen av UDP, hvis det skulle være nødvendig. Siden UDP ikke har de samme egenskapene som TCP er det ikke nødvendig å sende **ACK** for *acknowledgement* data, som er en bekreftelse på at man har mottatt pakkene siden sist **ACK** pakke. Og på grunn av det, bruker UDP mindre båndbredde. I noen applikasjoner som ved *live*-video og lyd, er det ikke noen poeng å sende data som ikke kom fram på nytt siden det da allerede er spilt av og man har ikke lenger noe behov for dette bildet eller en liten del av lyden som skulle vært spilt av for noen millisekunder siden.

UDP- vs. TCP-koblinger Ettersom UDP er *connectionless*, er det lettere å starte opp kommunikasjonen hvis en side skulle kræsje mens den andre siden fortsatt kjører, fordi de da ikke trenger å koble seg opp på nytt. TCP bruker *connections*, som vil si at begge sidene må bli enige og opprette en kobling før man kan sende informasjon. Med TCP kan et "kræsj" føre til problemer fordi den ene maskinen er usikker på om den andre faktisk er frakoblet og har derfor et tidsavbrudd før den gir opp på koblingen og kan bruke den igjen.

Vi benytter TCP i kommunikasjon mellom ROV-en og *topside*, fordi vi ønsket pålitelig kommunikasjon. Det at TCP har stor *overhead*, ble ikke sett på som noe problem siden vi har god båndbredde og raske maskiner.

Nettverks-*socket* Nettverks-*socket* er et API for sending og mottak av data over nettverket. Den tar tre parametere: **IP-adresse**, **transportprotokoll** og **port** den skal motta på. Det trengs kun en åpen port på den siden som venter på tilkobling. Den siden som tar kontakt for å opprette en kobling, åpner deretter en vilkårlig port utenfor de reserverte port-områdene.

Nettverks-*socket* blir brukt for sending og mottak av data i denne oppgaven.

Secure Shell (SSH) SSH er en protokoll for sikker oppkobling til en terminal på en annen datamaskin over nettverket. SSH ble laget for å erstatte telnet som også ble brukt til å koble til en terminal på en annen datamaskin, men telnet brukte ikke en kryptert linje. Dette gjorde det usikkert fordi passordet ble sendt over nettet i klartekst. SSH har også en mengde andre funksjoner som for eksempel overføring av filer, *port forwarding* og montering av filsystem over nettverket, som vil si at man kan få tilang til filer på en annen datamaskin som om det var en harddisk på din lokale datamaskin.

Vi bruker SSH for å koble opp til mini-PC-en plassert nede i ROV-en for å starte programmet til **Maskinsyn-gruppen** [1.5.1]. Se kapittel 2.12.4 om for mer informasjon om mini-PC-en.

Controller Area Network (CAN) CAN er et system som ble utviklet for å kommunisere mellom enheter i biler. CAN bruker kun to kabler som kobler sammen alle i en "buss" som vil si at alle har tilgang på informasjonen som sendes på kablene fordi de er koblet på den samme kabelen [15]. ROV-en bruker CAN for å kommunisere mellom mikrokontrollerne og mini-PC-en.

2.1.2 Operativsystem-begreper

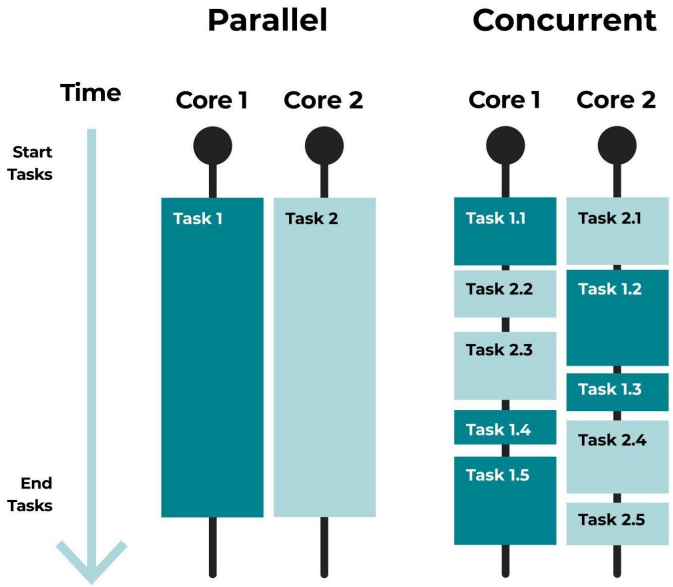
Parallellitet Parallellitet er når det kjøres to tråder eller prosesser helt samtidig, og ikke bare at de kjøres i levetiden av oppgavene, men at de kjøres helt samtidig på forskjellige kjerner og ikke bare bytter mellom hvilken prosess som kjøres på samme kjerne [16].

En tråd er den minste enheten som operativsystemet sin *scheduler* kan håndtere. I motsetning til en prosess som er en større enhet så har ikke tråder beskyttelse fra operativsystemet mot å påvirke hverandre. Operativsystemet deler ikke minnet og koden til en prosess med andre prosesser som et sikkerhetstiltak. Tråder har en egen stabel som betyr at man likevel må håndtere sending av data mellom de.

Et program som blir kjørt av et operativsystem blir gjort om til en prosess når koden skal kjøres. Prosessen er isolert og får ikke lov av operativsystemet til å se andre prosesser sitt minne som et sikkerhetstiltak [16].

Samtidighet (*Concurrency*) *Concurrency* betyr samtidighet, og skjer når det kjøres to eller flere tråder der den nye tråden kjører før den nåværende har fullført sin kjøring. Det er ikke nødvendig at koden kjøres samtidig for å oppnå ytelsesforbedringer, siden en tråd kan kjøre mens en annen tråd venter på noe, som data fra harddisk eller nettverket. Dette kan sees i figur 2.3, hvor hver kjerne bytter mellom hvilken prosess som kjøres, mens i parallell-kjøring så benyttes det samme kjerne til samme prosess hele tiden [16].

Samtidighet skal bli brukt i flere deler av programmet vårt. Blant annet bruker vi tråder til å ta i mot nettverksdata og å sende data mellom forskjellige deler av programmet som kjører samtidig som man henter data fra kontrolleren og sender på nettverket.



Figur 2.3: Forskjellen mellom samtidighet og parallellisme [17].

2.2 Formater

JavaScript Object Notation (JSON) JSON er et dataformat som er leselig for mennesker og gjør det lettere å debugge når man hvor som helst kan stoppe programmet og se på de faktiske verdiene som blir overført.

Vi koder all dataen som sendes over nettverket i JSON-formatet kombinert med UTF-8-koding fra streng til *bytes*, siden vi da kan være sikre på dataen som leses på andre siden og dekodes fra JSON-format er likt som det vi sendte over.

Motion JPEG (MJPEG) MJPEG er et format for komprimert video. MJPEG-formatet bruker JPEG-koding på hvert bilde. Da komprimeres hvert bilde i videoen individuelt i motsetning til for eksempel H.264, som er en standard for videokomprimering som bruker et komplett bilde kalt en *keyframe* der de kun beskriver endringene i områder i bildet for å vise fram bevegelse helt til neste nye *keyframe* blir laget.

Vi bruker MJPEG som videoformat hovedsaklig fordi **Maskinsyn-gruppen** [1.5.1] henter ut video fra kameraene som enkeltbilder slik at **Maskinsyn-gruppen** kan kjøre bildebehandling på bildene. Siden bildene allerede er gjort om til enkeltbilder krevde det minst prosessering å kode videoen til MJPEG.

Koden for strømming av video er laget av vår gruppe. Koden kan sees på GitHub-en til **Maskinsyn-gruppen** [18].

2.3 Vektorgrafikk

Vektorgrafikk representerer grafikk ved hjelp av vektorer, linjer og kurver i motsetning til punktgrafikk som deler opp grafikken i et rutenett og setter en fargeverdi for hver rute i rutenettet. Dette gjør at grafikken kan skaleres uendelig uten at kvaliteten forverres og brukes ofte til formål der man må skalere grafikken som for eksempel 3D-grafikk og til logo-er som må kunne vises på nettsider, men også store plakater.

Datagrafikk I datagrafikk bruker man spesielle prosessorer som heter *Graphics Processing Unit* (GPU). GPU-er er optimalisert til å gjøre *Single Instruction Multiple Data* (SIMD)-operasjoner som vil si at man definerer en funksjon som regner ut hvordan bildet skal se ut for alle punkter på skjermen. En *shader* er et kompilert program for en GPU. En *fragment shader* brukes til å farge en overflate som allerede er definert, for eksempel kan en *fragment shader* brukes på en side av en kube i en 3D-*scene* [19]. I figur 2.4 kan kildekoden til en enkel *fragment shader* sees.

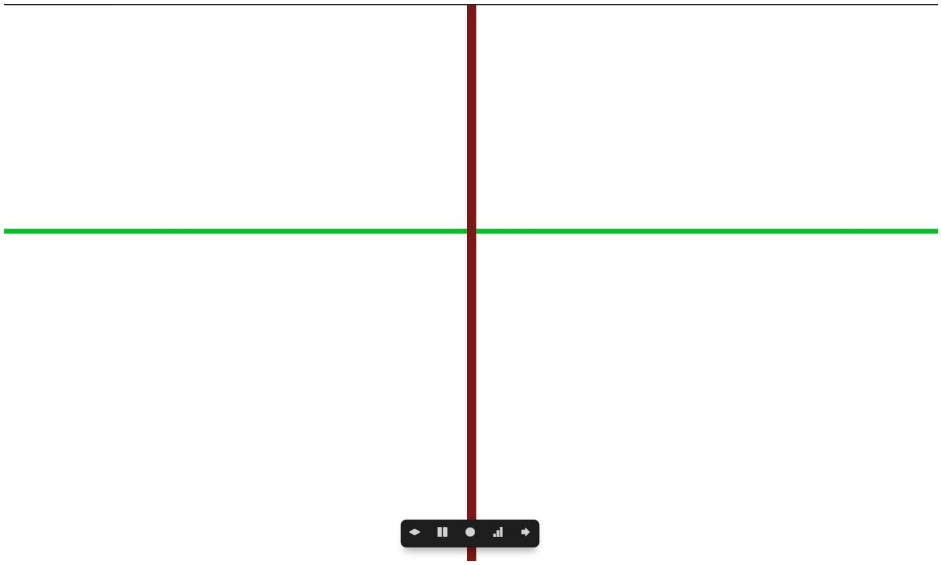
Fragment shaderen i figur 2.4 tar inn posisjonen til musepekeren og oppløsningen til bildet og regner deretter ut posisjonen til den nåværende pikselen som er normalisert mellom 0 og 1.

Deretter sjekker *shaderen* om den nåværende pikselen er mellom posisjonen til musepekeren og posisjonen til musepekeren +1%. *Shaderen* gir da ut en farge som er definert i vektorene på linje 18 og 23 i figur 2.4 og hvis den ikke er innenfor forblir fargen hvit som definert på linje 13.

```
6   uniform vec2 u_resolution;
7   uniform vec2 u_mouse;
8   void main() {
9       vec2 normalized_pixel_pos = gl_FragCoord.xy/u_resolution;
10      vec2 normalized_mouse_pos = u_mouse.xy/u_resolution;
11
12      // standard if outside other parameters
13      gl_FragColor = vec4(1.0, 1.0, 1.0, 1.0);
14
15      // handles y direction
16      if (normalized_pixel_pos.x < normalized_mouse_pos.x &&
17       normalized_pixel_pos.x > normalized_mouse_pos.x-0.01) {
18          gl_FragColor = vec4(0.4941, 0.0902, 0.0902, 1.0);
19          return;
20      }// handles x direction
21      if (normalized_pixel_pos.y < normalized_mouse_pos.y &&
22      normalized_pixel_pos.y > normalized_mouse_pos.y-0.01) {
23          gl_FragColor = vec4(0.0549, 0.7451, 0.1686, 1.0);
24          return;
25      }
26
27  }
28
```

Figur 2.4: Eksempel på en enkel *fragment shader* skrevet i OpenGL.

Resultatet av å kjøre *shaderen* kan sees i figur 2.5



Figur 2.5: Grafikken som produseres av *shaderen*

.

Grunnen til at man bruker en GPU til dette istedenfor en CPU er at oppgavene som skal løses for å lage grafikk er ekstremt parallelle og GPU-er er laget for å prosessere data parallelt med en instruksjon (se kap 2.1.2) [19] (SIMD som nevnt tidligere i kapittelet)

2.4 Utviklingsmetodikk og arbeidsmetode

2.4.1 Git



Figur 2.6: Git [20].

Git er et versjonskontrollsystem. Det vil si at den lagrer alle forandringene i koden i versjoner kalt for *commits*. Fordelen er at man kan gå tilbake til tidligere versjoner for eksempel hvis det er feil i koden eller man ønsker å se hvem som gjorde en forandring. Dette gjør det lettere å jobbe sammen i grupper siden man kan ha forskjellige versjoner og *branches* som gjør at man kan jobbe i det samme prosjektet uten å forstyrre hverandre.

For å ha en versjon som er testet og som er verifisert at funker, brukes det en *branch* hvor det kun *merges* med produksjonskoden etter at denne er testet opp mot ROV-en.

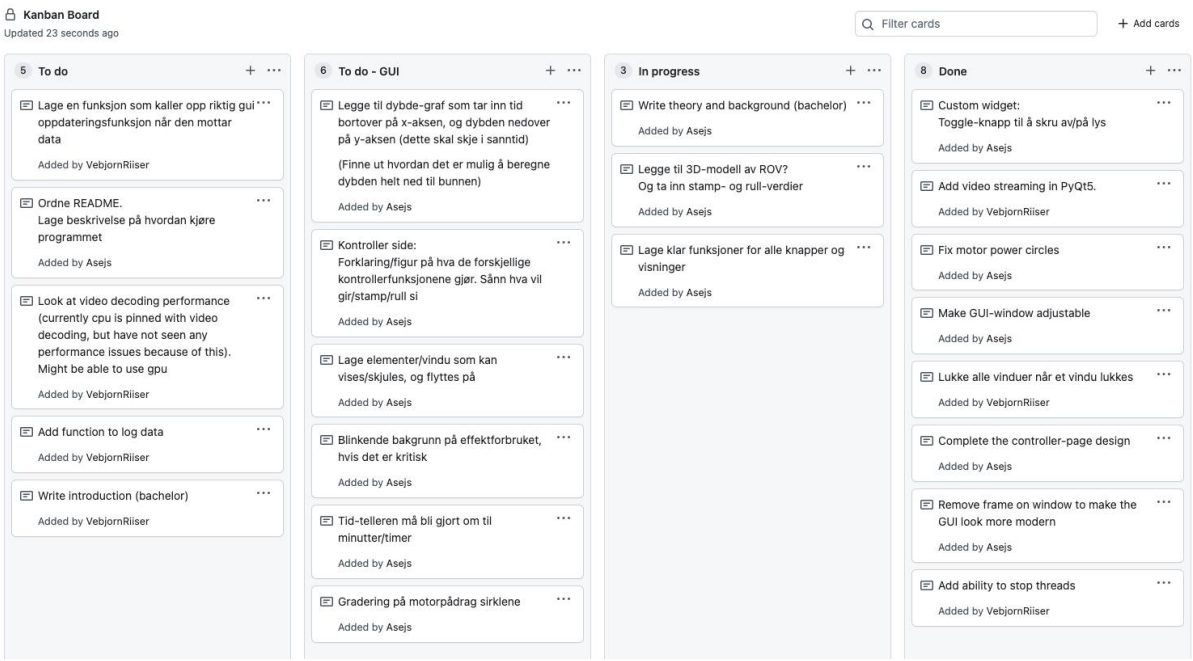
Git blir brukt i flere av gruppene i UiS Subsea. På et tidspunkt ble det gjort forandringer på begge sider av koden som ikke var kompatible med hverandre. Da vi skulle vannteste var det nyttig å kunne gå tilbake til før forandringen var gjort for å kunne fortsette testing av ROV-en for å deretter fikse opp i problemet når andre grupper ikke venter på å få testet sine deler.

2.4.2 Utviklingsmetodikk

Siden gruppen kun består av to medlemmer er det ønskelig med en enkel utviklingsmetodikk. Derfor benyttes Github sin prosjekttavle (se kap. 2.4.3 nedenfor) for å organisere og holde oversikt over hva som må gjøres, hva som er under behandling, og hva som er blitt gjort. Det blir først fokusert på de delene av oppgaven som må være ferdig for at de andre gruppene i UiS Subsea ikke blir forsinket på deres prosjekt. Videre prioriteres de oppgavene som tilfører mest verdi til prosjektet.

2.4.3 Github-prosjekttavle

Github sin prosjekttavle brukes for å organisere arbeidet. Den fungererer på lignende måte som en *kanban-tavle*, hvor gjøremål legges inn i kolonnen **To do**. For å ha oversikt over hvem som tar ansvar for hva, flyttes det kortet du velger over til **In Progress**-kolonnen. Når du blir ferdig med oppgaven legges den i **Done**. Dette gir en god oversikt over alle gjøremålene.



Figur 2.7: Github prosjekttavle over vårt prosjekt fungerte som en *kanban*-tavle.

2.4.4 Møter

I større prosjekt er det nødvendig med god kommunikasjon. Hver mandag deltar minst ett medlem fra hver gruppe på Subsea-rommet. Her blir det gjennomgått hva man hadde gjort siden forrige møte, hvordan man ligger an i forhold til frister, og om det oppstår noen problemer underveis siden sist eller om man trenger noe fra de andre gruppene. Det foregår også skriftlig kommunikasjon over en egen *chatte*-kanal i Discord. I tillegg skal informasjon deles gjennom et Google Sheets-dokument hvor for eksempel en oversikt over alle CAN-pakkene som skal sendes legges inn.

2.4.5 Modularitet i programmet

Boken *The Pragmatic Programmer* [21] som er en generell guide på hvordan lage gode programmer, forklarer blant annet prinsippet DRY (*Don't Repeat Yourself*). Ved å skrive programmet i små generiske deler er det lettere å gjenbruke denne delen flere steder i koden. Dette minimerer sjansen for feil og gjør det lettere å fikse feilene som måtte oppstå siden man kun trenger å fikse feilen et sted.

Et eksempel på dette er **Network**-klassen som brukes både på mini-PC-en og *topside* (se 3.1 for illustrasjon). Programmet i denne oppgaven er delt opp i flere deler som vist i figur 3.3.

Dette gjør at det ikke er nødvendig å vite noe om hvordan ROV-en funker i teorien for å kunne lage GUI og styring så lenge vi får beskjed om hva elektrogruppene trenger.

2.5 Datasikkerhet

I følge Store norske leksikon er datasikkerhet: "*en samlebetegnelse for metoder, tjenester og verktøy som sikrer at digital informasjon og digitale systemer ikke uten eiernes ønske eller forutgående handling, blir utilgjengelig, går tapt, blir lest eller endret.*" [22] I vårt prosjekt er utilgjengelighet, endring og tap av kode de største farene. Siden UiS Subsea jobber åpent ved å legge ut noe av koden tilgjengelig på Github og publiserer blant annet bilder av prosjektarbeidet i sosiale medier er det ikke særlig farlig om for eksempel elektronikkdesign eller kode fra prosjektet blir lest av andre. Hvis et sikkerhetsinnbrudd forandrer, gjør utilgjengelig eller sletter koden vår derimot vil dette forsinke prosjektet og i verste fall gjøre at vi ikke blir ferdig.

Gruppen bruker versjonskontrollsystemet Git (se kap. 2.4.1) med Github som vert for koden. En bieffekt av dette er at alle versjoner av koden er lagret på Github og utviklingsmaskinene til gruppemedlem mene så lenge man har hentet inn de siste forandringene som ikke er gjort av en selv. Dette gjør at selv om en datamaskin og hele Github skulle bli slettet samtidig (noe som er ekstremt usannsynlig), så forsvinner ingen eller minimalt med kode. Dette gjør også at vi har alle versjonene av koden slik at det er mulig å gå tilbake til en tidligere versjon hvis det skulle bli uønskede forandringer i koden.

2.6 Programmeringsspråk

Det ble gjort en vurdering av hvilket programmeringsspråk som er best egnet til å utføre oppgaven. Språkene som var mest aktuelle var Python, Go og C#. Det ble gjennomgått fordeler og ulemper ved de disse språkene. I tillegg ble det sett på hvilke programmeringsspråk som var brukt tidligere i UiS Subsea til å lage *topside*-systemet, og hvilke språk det er vanlig å benytte innen subsea-industrien.

2.6.1 Python

Python er et populært språk som har mange tredjepartsbiblioteker som er enkelt å benytte i koden. Dette vil spare tid ved å bruke ferdiglagde løsninger. For eksempel valgte vi å bruke en modul i Pygame-biblioteket til å hente ut inndata fra Xbox-kontrolleren.



Figur 2.8: Python [23].

En ulempe med Python er at det er tolket (*interpreted*) og har derfor mer *overhead* og krever derfor mer ressurser enn enn kompilerte språk for den samme oppgaven. Gruppen tok en vurdering på om dette var en så stor ulempe at det ikke var verdt å bruke Python, men kom fram til at programmet som skulle lages ikke krevde så mye ressurser fra datamaskinen at dette ville bli et problem.

I tillegg har de fleste ingeniørstudenter i UiS Subsea erfaring med Python fra DAT120 Grunnleggende programmering, og det gjør det lettere for de andre grupper å sette seg inn i programmet.

2.6.2 Go

Go er et relativt nytt programmeringsspråk og har derfor færre tredjepartsbibliotek enn eldre språk. Språket blir ut-



Figur 2.9: Go [24].

viklet hos Google og har flere gode egenskaper som egner seg til prosjektet:

- Go er *cross-platform* som vil si at det støtter flere prossesorarkitekturer og operativsystem.
- Go er et kompilert språk og har bedre ytelse enn fortolkede program som Python.
- Go er en mellomting mellom abstraksjonen i høynivåspråk som Python og lavnivåspråk som C.

2.6.3 C#

C# er et objektorientert programmeringsspråk som kjører på .NET-plattformen. Både C# og .NET er produkter laget av Microsoft. C# har i likhet med Python mange biblioteker. C#-program vil prestere bedre i utvikling av applikasjoner som spill, men vil ikke slå Python sin raske utviklingstid [26].

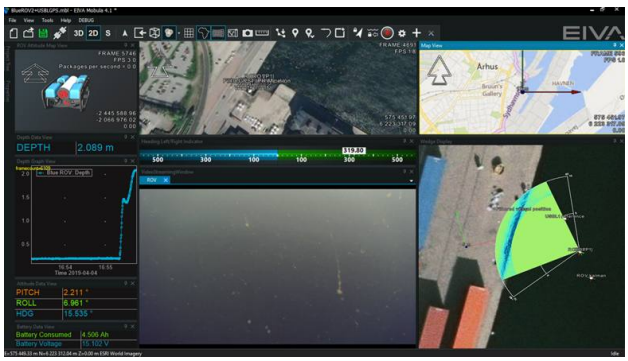


Figur 2.10: C# [25].

I 2014 benyttet UiS Subsea C# og .NET i Microsoft Visual Studio til å programmere styringssystemet for ROV-en deres [27]. Begrunnelsen for valget deres, var blant annet at Microsoft Visual Studio er brukervennlig og tilbyr gode hjelpemidler for å lage det grafiske brukergrensesnitt.

2.6.4 Programmeringsspråk brukt i subsea-industrien

C# er et språk som blir brukt mye av bedrifter til å lage programvare i subsea-industrien. EIVA, som er et dansk ingeniørselskap som hovedsaklig leverer løsninger til offshore, har laget NaviSuite som er en av de mest brukte programvarene i subsea-industrien, og her er C# blitt benyttet i utviklingen [28]. Se figur 2.11b. I tillegg har iCsys, som viste fram sin programvare iCsys Suite på *Subsea dagen*, brukt C# i utviklingen av den, se figur 2.11a.



(a) Programvaren NaviSuite fra EIVA er laget med C#. Hentet fra [29]



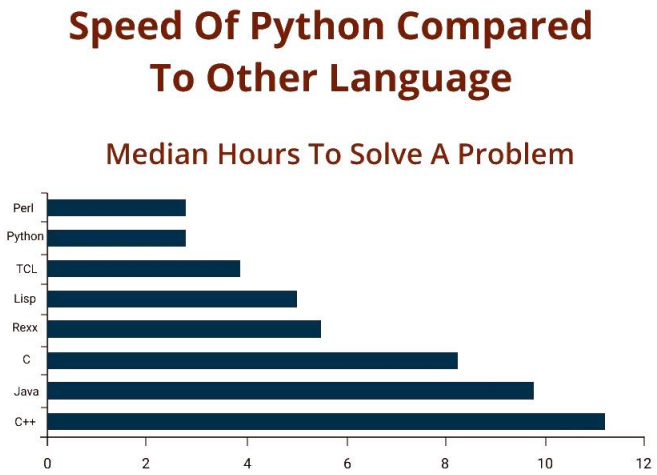
(b) Vidar Haus og Sven Hatteland, som var med å starte opp iCsys, med programvaren iCsys Suite. Bildet er hentet fra [30].

Figur 2.11: Eksempler på at C# er benyttet i subsea-industrien.

2.6.5 Valg av programmeringsspråk

Riktig valg av programmeringsspråk er viktig fordi feil verktøy til jobben kan gjøre oppgaven mer komplisert og at utførelsen av oppgaven tar lengre tid. **I år ble det valgt å programmere i Python**, fordi det går raskt å lage prototyper og i tillegg har Python store, gode bibliotek til

blant annet GUI-design og til å hente inn inndata fra styringsspaker. Python er også et språk begge gruppens medlemmer har mye erfaring med, gjennom skole og personlige prosjekter. I tillegg er Python et programmeringsspråk som det går raskt å lage løsninger med sammenlignet med andre språk. Dette kan sees i figur 2.12 som er lagd av programvareselskapet eSparkInfo [31].



Figur 2.12: eSparkInfo sin oversikt over median programmeringstid for å løse et problem. Hentet fra [31].

Når gruppen skulle bestemme seg for hvilket programmeringsspråk som skulle brukes så ble det sett på tidligere års arbeid i UiS Subsea og løsningene de hadde valgt og utviklet. En oversikt over programmeringsspråkene tidligere brukt, er vist i tabell 2.1.

År	Tidligere brukte programmeringsspråk til *topside*-systemet	Studieretning
2014	C# og .NET	Data
2015	C++	Elektro
2016	Python	Data
2017	—	—
2018	Python (med ReactJS, NodeJS)	Elektro
2019	MATLAB	Elektro
2020	—	—
2021	Python (med Flask, Gunicorn og Nginx, JS) og Go	Elektro

Tabell 2.1: Tidligere programmeringsspråk brukt til å lage *topside*-systemet i UiS Subsea. Flesteparten har vært elektrostudenter. I 2017 og 2020 var det ikke noe prosjekt i UiS Subsea.

Gruppen som hadde ansvar for GUI i 2021, lagde en webløsning med Python og Flask, der de brukte HTML og JavaScript for design av GUI-en [32]. I starten av årets prosjektet, ble det også vurdert å ha en slik webløsning der det kan lages flere brukere og logge inn for å styre ROV-en, men det ble til slutt avgjort å lage et program som kjører lokalt istedenfor en nettside.

Dette hindrer likevel ikke muligheten for å koble seg til utenfra hvis det brukes en *router* med *port forwarding* for SSH (standard er port 22), så kan man kjøre programmet fra hvor som helst i verden så lenge man har nett. Da må det kobles til med kommandoen:

```
ssh -L 6900:routerIP:6900 subsea@routerIP
```

`RouterIP` er routerens eksterne IP-adresse. I tillegg må nettleseren som henter videostrømmene

pekes mot den nye IP-en. Dette er ikke planlagt å teste, men det er avhengig av kort forsinkelse for en god opplevelse og dette er ikke forenlig med internett over lange avstander.

Grunnen til at vi endte vi opp med å lage GUI-en med **Qt** var hovedsaklig fordi det var mer hensiktsmessig å ha et program enn en nettside når det kun trengs å kjøres lokalt da dette minimerer kompleksiteten. En annen fordel ved å bruke PyQt er at flere elektrostudenter i UiS Subsea hadde erfaring med det fra et prosjekt i faget ELE340 Datamaskinkonstruksjon.

2.7 Rammeverk brukt i operatørgrensesnittet

2.7.1 Qt



Figur 2.13: Qt [33].

Qt et rammeverk skrevet i C++ [34]. Qt blir brukt av titusenvis av bedrifter i mer enn 70 industrier og i de fleste regioner av verden [35]. Det blir brukt til å lage *infotainment*-systemer i biler, lokasjon- og navigasjonssystemer, brukergrensesnitt for forbrukerelektronikk, mobilapper, operativsystemer og mye annet. Noen velkjente applikasjoner som er laget med Qt er KDE, Opera og Google Earth. Linux-versjonen av Skype pleide å være basert på Qt [36].

Qt brukes for å lage grafiske brukergrensesnitt, som gjør det enkelt å hente inndata gjennom knapper og andre elementer som slidere. Dette kan senere brukes til å styre andre systemer som for eksempel undervannsroboter som i denne oppgaven. I Qt finnes det mange ferdiglagde *widgets* som knapper, kombibokser, tekstfelter og tabeller. For å bruke Qt anbefales det å bruke QtCreator IDE for tekstredigering eller Qt Designer, som er et dra-og-slipp verktøy for å lage brukergrensesnitt. Qt Designer forklares i delkapittelet 2.7.2.

2.7.2 PyQt



Figur 2.14: PyQt [37].

PyQt er en binding til Qt. Qt brukes for å lage brukergrensesnittet, mens PyQt brukes til å manipulere det etterpå. En binding er et program som oversetter kommandoer fra et språk som i dette tilfellet er Python til C++-kode for Qt-spesifikke kommandoer.

PyQt brukes i denne oppgaven siden det er nødvendig å manipulere brukergrensesnittet mens programmet kjører og utføre kommandoer når brukeren trykker på en knapp eller drar en slider. For eksempel oppdateres all sensordataen med PyQt.

Lage et enkelt PyQt5-applikasjonsvindu

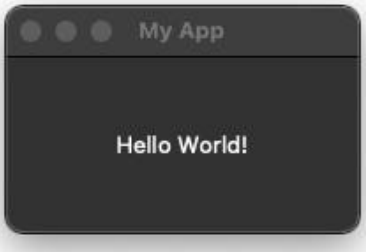
Hvordan det kan lages et enkelt PyQt5-applikasjonsvindu vises i figur 2.15, og nedover forklares hvordan dette gjøres, og det blir referert til denne figuren. Teorien bak er hentet fra [38].

```python
import sys
from PyQt5.QtWidgets import QMainWindow, QLabel, QApplication
from PyQt5.QtCore import Qt
from PyQt5.QtGui import *

# MainWindow er en underklasse av QMainWindow
# - Brukes til å tilpasse applikasjonsens hovedvindu
class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        # Setter vinduets overskrift
        self.setWindowTitle("My App")
        # Lager en ny QLabel, som viser teksten: "Hello World!"
        label = QLabel("Hello World!")
        # Midstiller tekstfeltet
        label.setAlignment(Qt.AlignCenter)
        # Setter tekstfeltet som sentral widget-en
        self.setCentralWidget(label)

# Lager en ny QApplication-instans som tar inn sys.argv
app = QApplication(sys.argv)
# Lager vindue-instansen
window = MainWindow()
# Viser vinduet
window.show()
# Starter hendelsesløkken
app.exec_()
```

(a) Python-filen som lager applikasjonsvinduet.



(b) Hvordan vinduet blir seende ut i macOS.

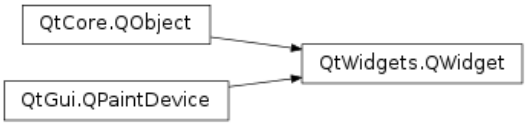
Figur 2.15: Hvordan det lages et enkelt PyQt5-applikasjonsvindu.

Qt-moduler

Qt API-et består av flere moduler, tre vanlige brukte moduler er:

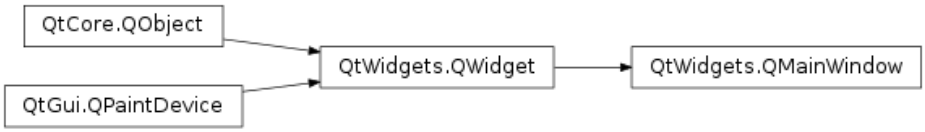
- `QtCore`-modulen er som navnet tilsier hovedmodulen i Qt. Den legger til mulighet for *signals* som er en *callback*-funksjon som sender oppdateringer til andre funksjoner. Denne er nødvendig for å oppdatere GUI-en. Den legger også til funksjonalitet for å arbeide med filer og mapper.
- `QtGui`-modulen legger til et vindu-system og mulighet for 2D-grafikk, bilder og forskjellige skrifttyper.
- `QtWidgets`-modulen legger til mange ferdiglagde GUI-elementer som lett kan bli tatt i bruk. For eksempel kan `QLabel` brukes til å vise frem tekst og en `QDateEdit` kan brukes til å velge dato.

I figur 2.16, ser man hvordan klassen `QtWidgets.QtWidget` arver fra klassene `QtCore.QObject` og `QtGui.QPaintDevice`.



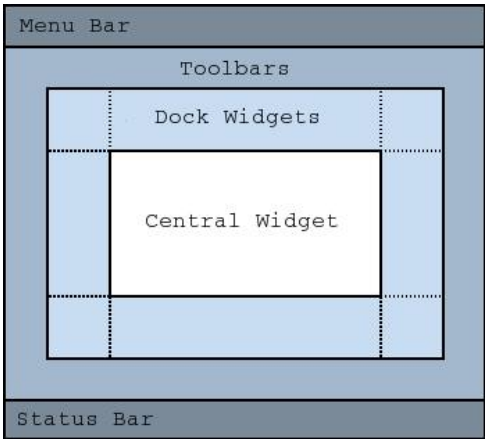
Figur 2.16: Hvordan `QtWidgets.QWidget` arver fra de de to andre klassene [39].

Hovedvinduet i applikasjonen lages ved å bruke en underklasse som arver fra `QMainWindow`-klassen fra `QWidgets`, se figur 2.17. Dette gjør at `QMainWindow` får tilgang til alle metodene og attributene som `QWidgets` har.



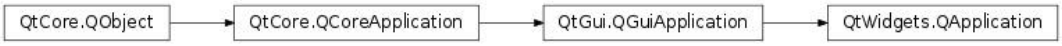
Figur 2.17: Hvordan `QMainWindow` arver fra PyQt-modulene. [40].

`QMainWindow` består av elementene vist i figur 2.18.



Figur 2.18: Hovedvinduet består av en menybar, verktøylinje, *dock-widgets* og en sentral-*widget* [39].

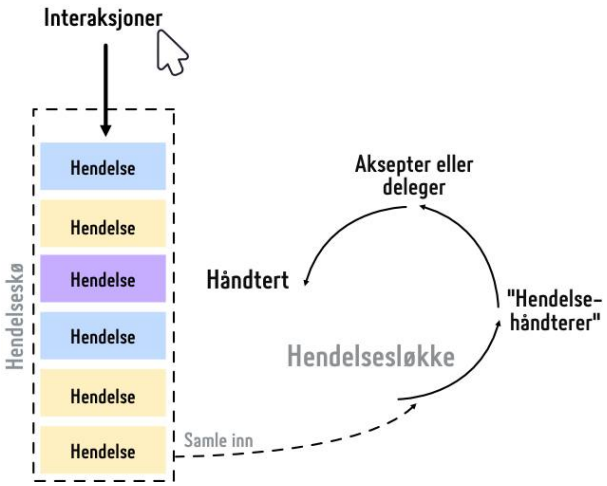
Videre i `__init__`-blokken fra kodeutdraget (linje 8, figur 2.15a), oppgis oppsettet til vinduet, og hvilke elementer vinduet skal innholde. For å sette opp selve applikasjonen må det lages en instans av klassen `QApplication` fra `QtWidgets`-modulen, se figur 2.19.



Figur 2.19: Hvordan `QApplication` arver fra PyQt-modulene. Hentet fra [41].

Hendelsesløkke

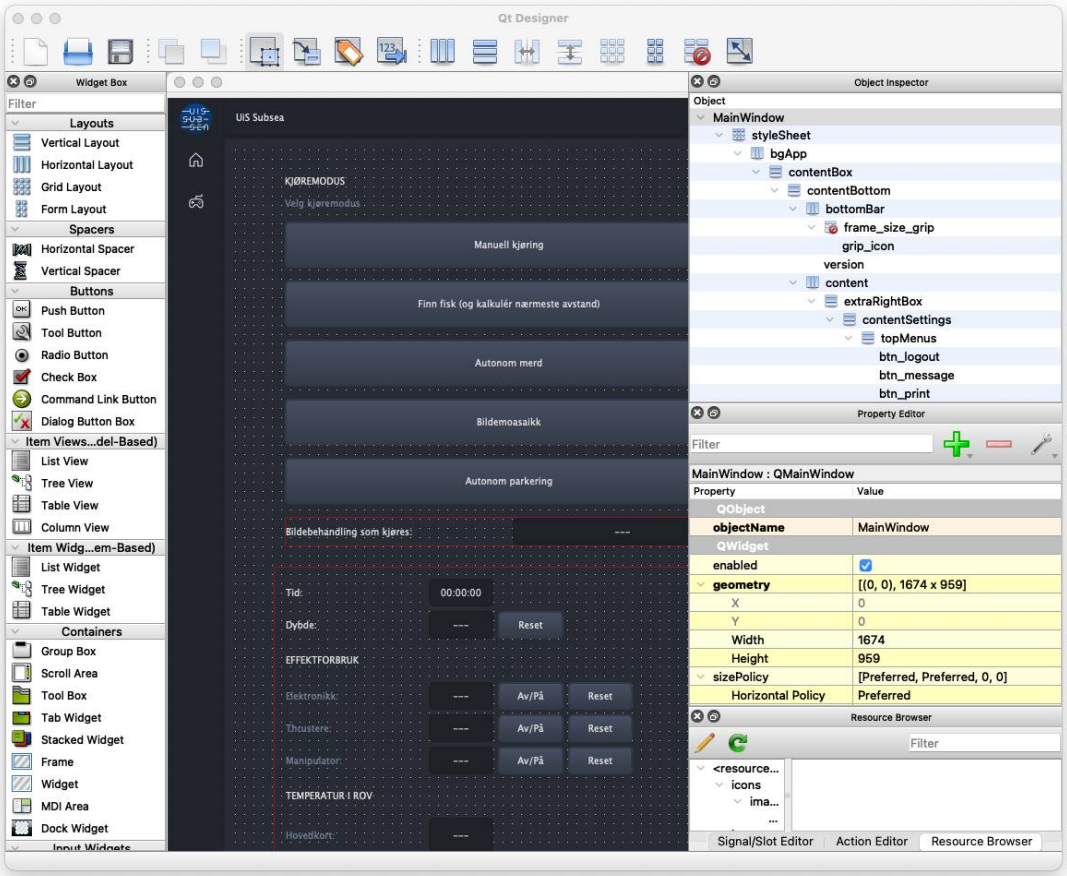
All interaksjon med applikasjonen som for eksempel knappetrykk eller at en slider flyttes på, fører til at hendelsen blir lagt i en hendelseskø. Køen sjekkes ved hver iterasjon og hvis har en eller flere ventende hendelser, blir den sendt videre til en "hendelseshåndterer", som tar seg av hendelsen. Etterpå sjekkes hendelseskøen på nytt igjen. Det finnes bare en hendelsesløkke per applikasjon. `app.exec_()` fra kodeutdraget (linje 27) starter hendelsesløkken. Hvordan hendelsene blir håndtert er illustrert i figur 2.20. Hvis *widget*-en som blir interagert med har en *callback* funksjon registrert blir denne kjørt.



Figur 2.20: Hendelsesløkken i applikasjonen. Hentet fra [38].

Qt Designer

Qt Designer er et Qt-verktøy for å designe og bygge grafiske brukergrensesnitt. Det man gjør i Qt Designer blir lagret i en UI (*User Interface*)-fil. Den inneholder XML-kodet data som definerer elementene lagt i Qt Designer. Den letteste måten å redigere UI-filer til Qt, er å bruke Qt Designer [42]. Hvordan Qt Designer-applikasjonen ser ut kan sees i figur 2.21.



Figur 2.21: Hvordan verktøyet Qt Designer ser ut.

Layouts I Qt finnes det *layouts* for å definere plasseringen av elementene. De forskjellige *Layout*-klassene er *QHBoxLayout*, *QVBoxLayout*, *QGridLayout* og *QFormLayout*. Ved å bruke *layouts* unngår man å plassere elementer manuelt og dette gjør at de plasseres forutsigbart. For eksempel plasseres elementer inne i en *QHBoxLayout* ut horisontalt helt til det ikke er mer plass og da flyttes de til neste linje.

UI-fil For å konvertere en .ui-fil til en .py-fil brukes `pyuic5`-verktøyet. Vår UI-fil er kalt for: `SUBSEAGUI.ui`, og terminal-kommandoen som da brukes til å konvertere .ui- til .py-fil ser slik ut:

```
pyuic5 -x SUBSEAGUI.ui -o SUBSEAGUI.py
```

Her er står `-x` for *execute* og `SUBSEAGUI.ui` er filen som skal gjøres om til en Python-fil. Argumentet `-o` forteller programmet hva den genererte Python-filen skal hete.

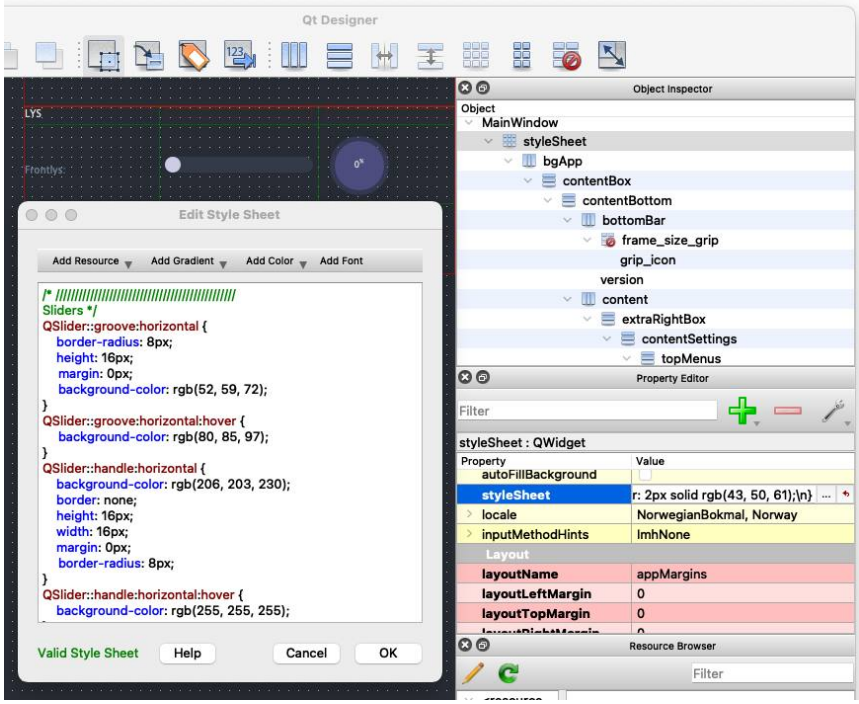
***Resource*-fil** En *resource*-fil gjør om binære filer til en Python-fil som lagrer disse i hexadesimalt-format. Dette brukes til å lagre bilder/ikoner i denne filen i stedet for i det lokale filsystemet, slik at alle som åpner UI-filen har tilgang til bildene i *resource*-filen.

Ved å bruke `pyrcc`, blir en Qt *resource*-fil (.qrc) konvertert til en Python-modul, som kan bli importert til en PyQt5-applikasjon. Alle filer som er lastet inn i Qt som har en kolon som prefiks lastes inn fra *resources* Python-filen. Kommandoen som brukes for å konvertere .qrc til .py, ser slik ut:

```
pyrcc5 resources.qrc -o resources_rc.py
```

Qt Style Sheets Qt Style Sheets gjør at man kan endre utseendet til *widgets* og deler mange likhetstrekk med *Cascading Style Sheets* (CSS) [43].

Qt Style Sheets kan i likhet med CSS også settes på hele applikasjonen, som vil bli arvet videre til komponentene som legges inni hvis dette er spesifisert. I oppgaven vår brukes en `QWidget` kalt **styleSheet**, som har egenskapen **styleSheet** hvor stilene til de forskjellige *widgets*-ene i applikasjonen er spesifisert. Et eksempel på stilen til `QSlider`, kan sees i figur 2.22. Om det er noen enkelttilfeller av knapper som skal ha sin egen farge og form, så blir stilen spesifisert direkte i knappenes sine egenskaper.



Figur 2.22: Eksempel på hvordan *Qt Style Sheet*-syntaksen ser ut for justering av stilen på den horisontale slideren.

2.8 Datamaskin og oppbevaringsløsning

Siden målet er å delta i MATE ROV-konkurransen, så ønskes det en løsning som er enkel å frakte med seg og som fort kan rigges opp på konkurranseområdet.

I 2018 ble det brukt en styrestasjon med dataskjermer montertert i en kasseløsning, som vist i figur 2.23. Den ble også brukt i 2019, da denne hadde oppsettet som trengtes for oppkobling til datidens ROV med blant annet en switch, samt at de brukte en egen mikrokontroller som mellomledd for å kommunisere over CAN [44, side 11].



Figur 2.23: Styrestasjon som ble brukt i 2018 og gjenbrukt i 2019 [44, side 11].

Siden vi bruker en USB til CAN-adapter fra mini-PC-en i ROV-en til CAN-bussen, som mikrokontrollerene snakker sammen på, og kameraene kobles direkte til mini-PC-en, trenger vi ikke en ekstra mikrokontroller eller switch. Avgjørelsen om hvilken overgang som skulle brukes, ble tatt av **Maskinsyn-gruppen** [1.5.1].

Vi vurderte å bruke samme kasseløsning i år, men dette ble for tungvindt. Når vi ikke trengte å ta med like mye utstyr til bruk på *topside*, gjorde dette at vi bestemte oss for å ta med utstyret i den vanlige fly-baggasjen.

2.8.1 Valg av *topside*-datamaskin

Til topside-datamaskin brukes en bærbar datamaskin med en firekjerners Intel i5 11300H-prosessor i tillegg til et Nvidia GTX 1650-grafikkort. Denne kobles opp til to eksterne skjermer. Brukergrensesnittet skal vises på den skjermen til den bærbare datamaskinen, og videostrømmen fra frontkameraet og havbunnskameraet skal vises på de to eksterne skjermene. Oppsettet av den bærbare datamaskin og de to eksterne skjermene, kan sees nedenfor i figur 2.23.

Det ble vurdert å bruke en sterkere stasjonær datamaskin som *topside*-server, men siden **Maskinsyn-gruppen** [1.5.1] valgte å gjøre bildebehandlingen lokalt på ROV-en var ikke dette nødvendig. I tillegg har koronapandemien ført til økte priser på maskinvare som gjør det mindre fornuftig å kjøpe inn deler nå med tanke på at vi har et budsjett å holde.



Figur 2.24: Oppsett av laptop og de to eksterne skjermene.

2.9 Styrespak

Valg av styrespak er viktig for å gjøre styringen av undervannsfarkosten lettest mulig for operatøren. Minimumskriterier for styrespak må blant annet baseres på hvor mange knapper som behøves for å styre undervannsfarkosten.

2.9.1 Valg av styrespak

Vi vurderte i første omgang en kontroller som ofte brukes til flysimulatorer, kalt Logitech Extreme 3D Pro (vist til venstre i figur 2.25), som allerede var brukt på ROV-en som ble lagd i 2014. Selv om denne har flere knapper og styringsmuligheter, som ville passet til styring av ROV-en, ble den valgt bort. Begrunnelsen var at den var tyngre i forhold til en kontroller, som for eksempel en Xbox 360-kontroller (vist midterst i figur 2.25), og en håndkontroller er lettere å holde under testing, hvor man innimellom må flytte seg rundt for å se hva som skjer i bassenget. I tillegg ville det trolig ta lengre tid å lære seg å styre ROV-en siden det er flere knapper som lettere kan blandes med hverandre.



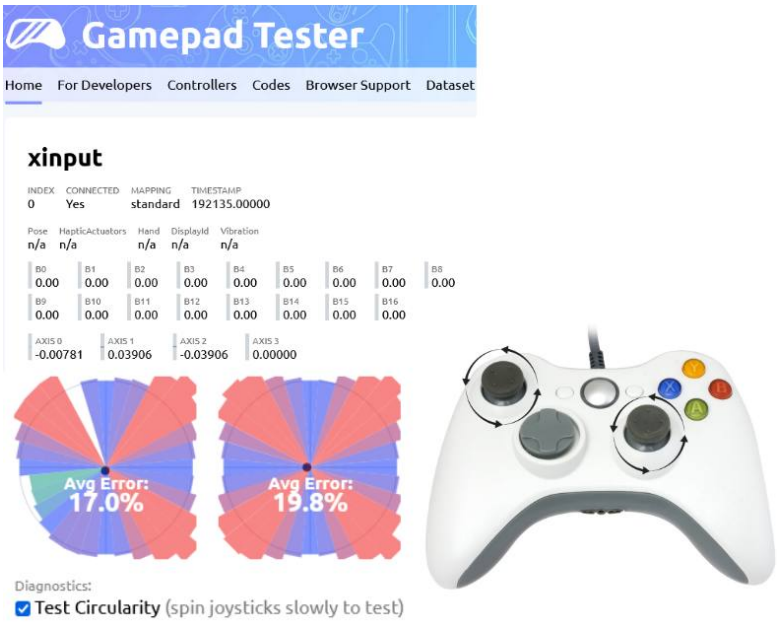
Figur 2.25: Valget av styrespake stod mellom Logitech Extreme 3D Pro [45], Xbox 360-kontroller [46] og Xbox One-kontroller [47].

Etter testing, (se delkapittel 2.9.2), fant vi ut at Xbox 360-kontrolleren var lite nøyaktig. Derfor falt valget heller på en kablet Xbox One-kontroller (vist til høyre i figur 2.25), fordi resultatet fra testen viste at den var mer nøyaktig. I tillegg har den passende antall knapper og styrespaker. Hver knapp skal ha sin funksjon. Hvis det i framtiden trengs flere funksjoner enn det er knapper, er det mulig å bruke en knapp på kontrollen til å bytte modus slik at en knapp eller en av de to styrespakene på kontrollen kan brukes til flere ting.

Xbox One-kontrolleren kan brukes trådløst, men det er ønskelig at den er kablet under MATE ROV-konkurransen, for å minimere risikoen for at den kobles ut i et kritisk øyeblikk. En trådløs kontroller kan dessuten gå tom for batteri og er utsatt for elektromagnetisk stråling. Men det at kontrolleren har mulighet til trådløs oppkobling kan være en fordel dersom operatøren, ønsker å stå ved bassengkanten og styre ROV-en, for eksempel under testing. I konkurransen derimot er det et krav at operatøren må sitte med ryggen til bassenget.

2.9.2 Test av styrespak

Ved å bruke nettsiden, `www.gamepad-tester.com`, er det mulig å teste nøyaktigheten til kontrollere som er tilkoblet til datamaskinen. Der testet vi en Xbox One-kontroller mot en uoriginal Xbox 360-kontroller. Avviket fra sirkulært verdiområde ble kontrollert ved å snurre styrestikken sakte rundt i en sirkel. Det observerte resultatet kan sees i figur 2.26 og 2.27.



Figur 2.26: Gjennomsnittlig avvik for den uoriginale Xbox 360-kontrolleren sin venstre og høyre *joystick* er henholdsvis 17,0% og 19,8% [48].



Figur 2.27: Gjennomsnittlig avvik for Xbox One-kontrolleren sin venstre og høyre *joystick* er henholdsvis 7,4% og 8,2%. Noe som er betydelig mindre sammenlignet med den uoriginal kontrolleren [48].

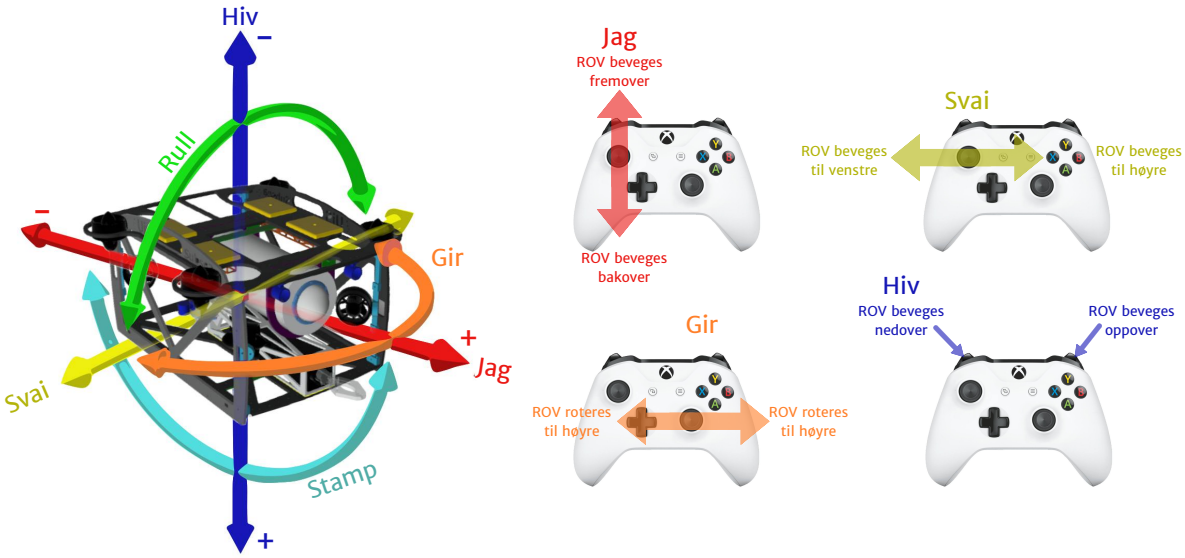
Resultatet viste at Xbox One-kontrollen har et mindre gjennomsnittlig avvik fra et perfekt sirkulært verdiområde og mindre dødsone, som vil si hvor langt man må bevege kontrollen før

man får se endringen i inndataen. Dette varierer på de forskjellige kontrollene basert på merke og modell, men også innen en spesifikk modell kan det være forskjeller.

Resultatet av denne testen ble avgjørende for at vi bestemte oss for å bruke Xbox One-kontrolleren som var mer nøyaktig.

2.10 Styring av ROV-en

Hvordan de ulike frihetsgradene påvirker ROV-en er presentert i figur 2.28. Figuren viser også hvordan **jag**, **svai**, **gir** og **hiv** skal kunne styres fra kontrolleren.



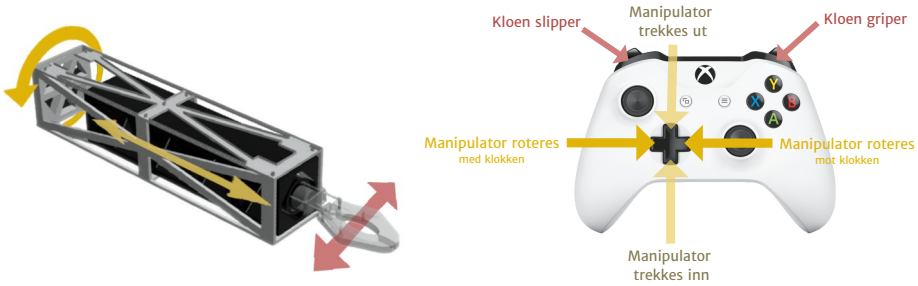
Figur 2.28: Hvordan de seks frihetsgradene påvirker ROV-en, og hvordan de fire frihetsgradene skal kunne styres fra kontrolleren.

Reguleringsgruppen [1.5.1] har valgt at de to resterende frihetsgradene **rull** og **stamp** ikke skal prioriteres å kunne styres fra kontrolleren. Det er likevel satt av plass for dette i CAN-pakken[1] som sendes til **Reguleringsgruppen**, slik at det vil være mulig å implementere dette i senere tid.

[1]Denne CAN-pakken har ID 70, og vises i oversikten over CAN-pakkene, se figur 3.5. Hvordan styredataen blir sendt kan sees i kapittel 3.5.

2.10.1 Styring av manipulatoren

Manipulatoren, som er på undersiden av ROV-en, skal ha tre motorer og en klo til å gripe. Den skal ha en motor til å trekke manipulator-armen inn og ut, en motor til å rotere manipulator-armen, og en siste motor til å kunne åpne og lukke kloa. Figur 2.29 viser hvordan manipulatoren skal kunne beveges. Design av manipulatoren og virkemåte er bestemt av **Manipulatorgruppen** [1.5.2]. Dette vil ikke bli gått videre inn på, siden dette er utenfor vårt ansvarsområde. Kommandoene til manipulatoren sendes ned sammen med resten av styredataen til reguleringsgruppen og kan sees på linje 415 i figur 3.9.



Figur 2.29: Pilene i figuren illustrerer at manipulatoren kan trekkes inn og ut, roteres, og at kloen kan åpnes og lukkes.

2.11 Xbox-kontrolleren

Xbox-kontrolleren består av 12 trykk-knapper og to styrespaker. Trykk-knappene er: X-, Y-, B-, A-knappene, to skulderknapper, to triggerknapper, to knapper på selve styrespakene, og til slutt visning- og meny-knappen. Både venstre og høyre styrespak har en x-akse og en y-akse.

2.11.1 Kontroll-funksjoner

Hvilke funksjoner som skal styres av kontrolleren, når det gjelder ROV-en, kameraet og manipulatoren, er listet opp:

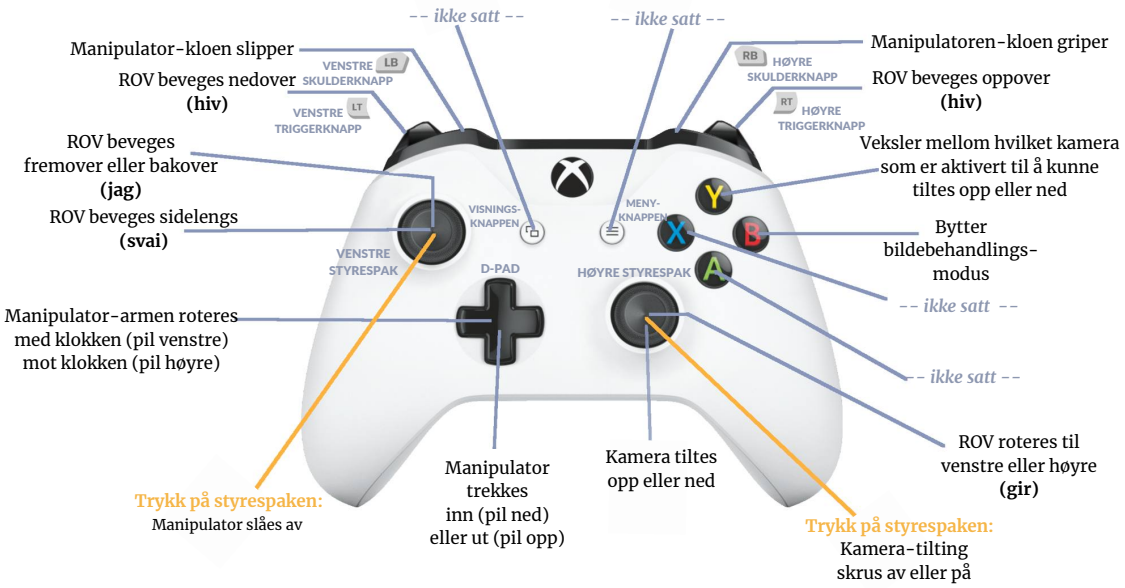
- Styring av fire frihetsgrader: jag, svai, gir og hiv (forklaring av frihetgradene vises i figur 2.28).
- Skru manipulatoren av og på.
- Bevege manipulatoren inn og ut.
- Gripe med manipulator-kloen og slippe grepet.
- Skifte mellom hvilket av de to kameraene som er aktivt, slik at kameravinkelen på valgt kamera kan justeres.
- Justering av kameravinkel, opp eller ned i 30 grader.

2.11.2 Standardoppsett

Det er tenkt å ha et standardoppsett av hvilke funksjoner hver knapp har, som skal være det valgte oppsettet når man starter programmet. Standardoppsettet kan endres, ved at man lager

en ny profil med eget ønske av knappefunksjoner. Dette kan være nyttig dersom ulike ROV-operatører har forskjellige preferanser på oppsettet, og ønsker at en knapp skal gjøre noe annet enn en annen operatør ønsker.

Hvilken funksjon de ulike knappene eller styrespakene er satt til kan sees i figur 2.30.



Figur 2.30: Figuren viser Xbox-kontrolleren og knappenes ulike funksjoner. Bildet av Xbox-kontrolleren er hentet fra [49].

Forklaring av standardoppsettet av knappefunksjonene til kontrolleren:

- **Venstre styrespake (framover/bakover):** ROV beveges fremover eller bakover **(jag)**.
- **Venstre styrespake, (venstre/høyre):** ROV forskyves sideveis til venstre eller høyre **(svai)**.
- **Høyre styrespake (venstre/høyre):** ROV roteres mot venstre eller høyre **(gir)**.
- **Høyre styrespake (framover/bakover):** Kamera tiltes opp eller ned, fra 0 til 30 grader.
- **Venstre styrespake-klikk:** Manipulatoren skrus av, eller skrus på igjen ved å trykke to ganger.
- **Høyre styrespakeklikk:** Tilting av kamera skrus av eller på.
- **Venstre og høyre skulderknapper (LB, RB):** Manipulatorkloen åpnes eller lukkes.
- **Venstre og høyre triggerknapper (LT, RT):** ROV beveges nedover eller oppover **(hiv)**.
- **D-pad (venstre/høyre):** Manipulator roteres mot klokken eller med klokken.
- **D-pad (framover/bakover):** Manipulator trekkes ut eller inn.

2.11.3 De analoge styrespakene

Det er ikke mulig å forandre de analoge styrespakene. Dette ville tatt mer tid og ville vært mer komplekst, siden det må settes dødsone på hver av aksene og det er vanskeligere å vise dette på en god visuell måte.

Dette er en av funksjonene som kan videreutvikles etter innlevering av bacheloren, men før MATE ROV-konkurransen. Eventuelt er dette en funksjon som kan lages av neste års lag dersom de benytter vårt system.

2.11.4 Forskjellen på ønsket pådrag fra kontrolleren og faktisk pådrag til motorene

Motorpådraget til ROV-en blir påvirket av pådraget på kontrolleren, men det er ikke et én-til-én-forhold mellom pådrag fra kontrolleren og pådraget fra motoren. Dette er fordi det utføres regulering på ROV-en for at den skal holde seg stille i vannet slik at den skal være enkel å manøvrere og at den holder seg rolig når det ikke er inndata fra kontrolleren. I tillegg må inndata fra kontrolleren behandles siden den ikke er perfekt sirkulær som vist i figur 2.27. Dette betyr at inndataen må skaleres slik at den høyeste verdien på utslaget blir 100 % og ikke mer. Dette håndteres ved at alle verdiene skaleres i prosent fra det høyeste pådraget.

2.11.5 Bibliotek til håndtering av kontrollerinndata

PyGame er et bibliotek for å lage spill og GUI i Python [50], og skal brukes til å hente inn data fra Xbox One-kontrolleren.



Figur 2.31: PyGame [50].

Det at PyGame skal benyttes gjør at det lett å lage programmet modulært, slik at det vil være enkelt å bytte ut en Xbox One-kontroller med for eksempel en stikke som brukes i flysimulatorer, som Logitech G Extreme 3D Pro *joystick*-en.

2.12 Kommunikasjon mellom ROV og *topside*

2.12.1 Kommunikasjonsmedium

Kommunikasjonsmedium vil si det fysiske stoffet som transporterer signalet som brukes for å kommunisere. Dette er relevant fordi det påvirker virkemåten til ROV-en og også vår oppgave med tanke på hva som er mulig. For eksempel hvis det ikke hadde vært nok overføringshastighet kunne det ikke ha blitt vist fram *live*-video i GUI-en.

Under vil vi gå gjennom mulige kommunikasjonsmedium som kunne bli brukt for dataoverføringen mellom *topside* og ROV-en. Dette kan være kjent stoff for datastudenter, men vi velger å gå igjennom det, dersom neste års gruppe ikke har kjennskap til kommunikasjonsmedium.

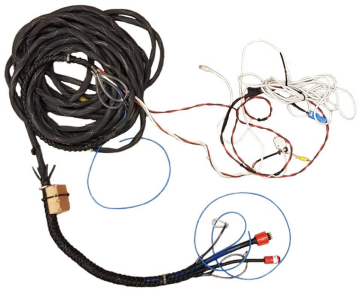
Fiberoptisk kommunikasjon Dataoverføring blir sendt som lys, noe som gjør kommunikasjonen rask og at den fungerer godt over lengre avstander. Dette er som regel infrarødt lys som ikke er synlig [51]. En fordel er at fiber ikke blir påvirket av elektromagnetisk støy fra kretser. En ulempe er at fiberkabelen er skjør.

Kommunikasjon gjennom kobberkabel Kommunikasjon gjennom en kobberkabel skjer ved at siden som sender data varierer spenningen i kabelen og siden som mottar tolker denne spenningen etter en fastsatt koding. Ulempen med kobberkabel er at den er tyngre og har lavere overføringshastighet enn en fiberkabel, og kan også bli mer påvirket av elektromagnetisk støy. Til gjengeld tåler kobberkabelen mer og den er en del billigere.

Trådløs dataoverføring ved hjelp av lyd og lys Det finnes også muligheter for å overføre data trådløst gjennom vann ved hjelp av lydsignaler og lyssignaler[2], men dette er uaktuelt i vårt tilfelle siden disse løsningene som regel er dyrere, mer kompliserte og tregere enn løsninger som benytter kabel. ROV-en trenger uansett en tether i MATE ROV konkurransen, da dette er et krav.

2.12.2 Valg av kommunikasjonsmedium

På fjorårets ROV ble det brukt en fiberkabel som kommunikasjonsoverføring [52, side 31], se figur 2.32. **Teknisk ansvarlig** i UiS Subsea har kommet fram til at den samme løsningen skal brukes i år. Dette er blant annet fordi fiberkabelen er lett, noe som er viktig på grunn av vektbegrensning av navlestrengen og ROV-en i MATE ROV-konkurransen.



Figur 2.32: Bildet viser kabelen som er overføringsmediumet fra overflate til ROV, og kalles for ROV-en sin "navlestreng" eller tether. Denne ble også brukt i 2018 og 2019 [53, side 12], og skal også brukes i år.

2.12.3 Test av fiberkabelens overføringskapasitet

Det blir gjort en test av fiberkabelens overføringskapasitet, og den ble utført ved hjelp av programmet iperf3 [54] som vist i figur 2.33.

```
subsea@subsea-MINIPC-PN51-E1:~$ iperf3 -c 10.0.0.1
Connecting to host 10.0.0.1, port 5201
[  5] local 10.0.0.2 port 42608 connected to 10.0.0.1 port 5201
[ ID] Interval           Transfer     Bitrate         Retr  Cwnd
[  5]   0.00-1.00   sec   113 MBytes   952 Mbits/sec    0    210 KBytes
[  5]   1.00-2.00   sec   111 MBytes   930 Mbits/sec    0    210 KBytes
[  5]   2.00-3.00   sec   112 MBytes   937 Mbits/sec    0    210 KBytes
[  5]   3.00-4.00   sec   113 MBytes   946 Mbits/sec    0    210 KBytes
[  5]   4.00-5.00   sec   113 MBytes   950 Mbits/sec    0    210 KBytes
[  5]   5.00-6.00   sec   113 MBytes   950 Mbits/sec    0    210 KBytes
[  5]   6.00-7.00   sec   111 MBytes   927 Mbits/sec    0    210 KBytes
[  5]   7.00-8.00   sec   111 MBytes   931 Mbits/sec    0    210 KBytes
[  5]   8.00-9.00   sec   109 MBytes   912 Mbits/sec    0    210 KBytes
[  5]   9.00-10.00  sec   113 MBytes   950 Mbits/sec    0    240 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.00  sec  1.09 GBytes   938 Mbits/sec    0             sender
[  5]   0.00-10.00  sec  1.09 GBytes   938 Mbits/sec                  receiver

iperf Done.
```

Figur 2.33: Test av overføringshastigheten direkte mellom mini-PC-en og *topside* gjennom fiber.

[2]Dette ble vist fram av Innova under et bedriftsbesøk. Innova leverer produkter og løsninger til undervannsindustrien.

Testen viser at vi får en overføringshastighet på rundt 1 gigabit. Dette er nesten den teoretiske makshastigheten til medie-konverteren som er på 1 gigabit. Dette er forventet siden noe av overføringshastigheten går med til *overhead* i TCP som forklart i kapittel 2.1.1. Fiberkabelen gir mer enn nok overføringskapasitet for flere videostrømmer og sensordata opp og ned.

2.12.4 Mini-PC i ROV-en

I elektronikkhuset til ROV-en skal det plasseres en mini-PC. Denne skal ta inn sensordata fra CAN-bussen og mottar videostrømmen fra kameraene over USB. Dette ordner gruppen som har ansvar for maskinsyn og interkommunikasjonen [1.5.1]. Mini-PC-en skal på kommando fra brukergrensesnittet utføre bildebehandling. Da tegnes det på videoen fra kameraet før den sendes opp til *topside*.

Det å sende videostrøm og sensordata fra mini-PC-en til *topside*, skjer i samarbeid med gruppen, som har ansvar for maskinsyn og kommunikasjon [1.5.1], og vår gruppe. Vår oppgave er å lage en løsning for å sende videostrømmen opp og vise den sammen med sensordataen.



Figur 2.34: Mini-PC-en, ASUS Mini PC PN51 [55], skal bli plassert nedi ROV-en.

Nettverks-*socket*

Til å kommunisere mellom *topside*-serveren og mini-PC-en brukes nettverks-*sockets* [2.1.1]. Server-siden av *socket*-en har et eget portnummer som brukes til å opprette kontakten mellom denne og klienten. Klienten tar kontakt med serveren ved hjelp av dens IP-adresse og porten. Porten gjør at brannmuren og operativsystemet vet hvor pakkene skal sendes.

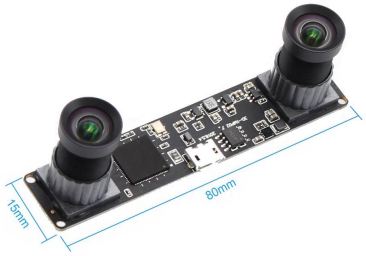
To populære typer protokoller som brukes i en *socket* er TCP og UDP [56]. *Datagram-socket* kommuniserer via UDP. Dette planla vi å bruke til strømming av video, men endte opp med å heller bruke MJPEG-strømming over HTTP som benytter TCP-kommunikasjon. *Stream-socket* benytter seg av TCP-kommunikasjon og skal brukes for å sende data til og fra ROV-en.

2.13 ROV-ens kamera

Valget av kamera ble tatt utifra hva de ulike gruppene hadde behov for. Stereokamera var nødvendig for å utføre bildebehandling, siden **Maskinsyn-gruppen** [1.5.1] trengte å sammenligne bildene for å kunne se tredimensjonalt. Dette blir blant annet brukt til å beregne avstanden til bunn, avstanden til nærmeste objekt, og det blir også brukt i forbindelse med en av oppgavene i MATE-konkurransen som går ut på å måle lengden av tre fisker og beregne gjennomsnittslengden av dem [10]. HD-kvalitet var også praktisk for gruppen som har ansvar for bildebehandling.

Det var **Maskinsyn-gruppen** som tok ansvaret for innkjøp av kamera, de kjøpte inn to USB-

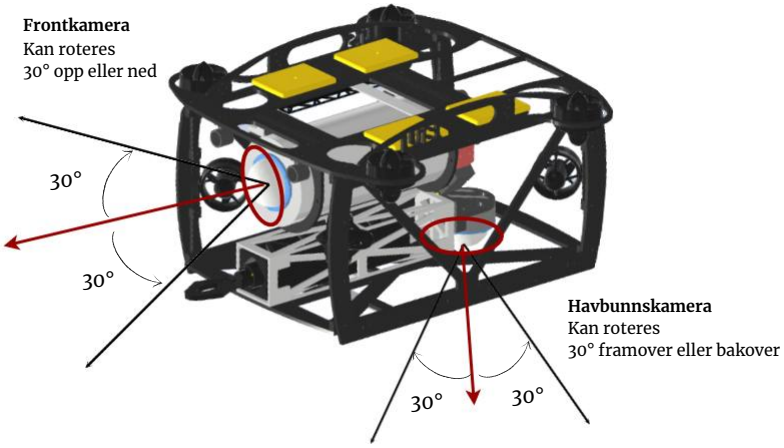
stereokamera, som vises i figur 2.35. Et skal brukes som frontkamera og det andre brukes som havbunnskamera.



Figur 2.35: To stk USB-stereokamera ble kjøpt fra AliExpress [57], for å plasseres på ROV-en.

2.13.1 Plassering og manøvrering av kameraene

Kameraene skal monteres i to akryl kupler, for å ha utsikt og holde seg vanntett. Begge kameraene skal monteres slik at de kan beveges i en vinkel på 30° opp og ned (se figur 2.36). Dette gjør at manipulatoren kan sees av operatøren når ROV-en skal gripe et objekt. **Maskinsyn-gruppen** står for plassering og montering av kamera i tillegg til innhenting av videostrømmen.



Figur 2.36: Figuren viser hvor kameraene på ROV-en skal festes, og viser hvordan de kan tiltes.

2.13.2 Belysning

Ved kameraene skal det plasseres LED-lys, dette kan være nødvendig i dypere vann og grumsete vann for å få en klar videostrøm. **Kraftforsyningskort-gruppen** [1.5.1] tar seg av innkjøp og montering av lys, men styrken styres fra en slider på GUI-en ved at programmet vårt sender ned en melding til kraftforsyningskortet.

2.13.3 Lyd fra motorene

Kameraene tar ikke opp lyd. I 2016 mente ROV-operatøren at det var vanskelig å vurdere hvordan ROV-en reagerte på pådrag uten lyd [58, side 47]. Men i år skal vi ha en figur som viser motorpådraget. Gruppen som har ansvar for styrings- og reguleringssystemet [1.5.1] regulerer pådraget basert på data de mottar fra Xbox kontrolleren og sensorkortet. De sender deretter dette pådraget opp til oss gjennom mini-PC-en.

2.14 Logging av sensordata

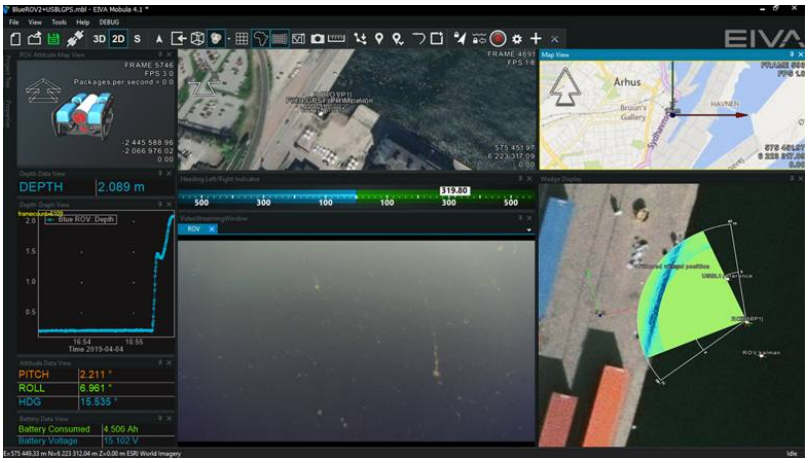
Logging vil si å samle inn og lagre relevant informasjon. Dette kan senere brukes til å finne ut hva som gikk galt, forstå hvordan man kan optimalisere og til å simulere systemet ved å spille tilbake dataen. Logging av data kan i tillegg gjøre det raskere og lettere å teste og utvikle, siden flere kan jobbe med samme fysiske system bare at man bruker dataene man har lagret istedenfor å koble direkte til systemet.

Planen er å lagre alle pakkene, som blir sendt opp og ned fra ROV-en i en tekstfil med tidsstempel på.

2.15 Brukergrensesnitt i subsea-industrien

Siden manualen til MATE ikke spesifiserer noen spesielle behov eller krav til brukergresesnittet, har vi prøvd å undersøke hvordan typiske brukergresesnitt i ROV-industrien ser ut.

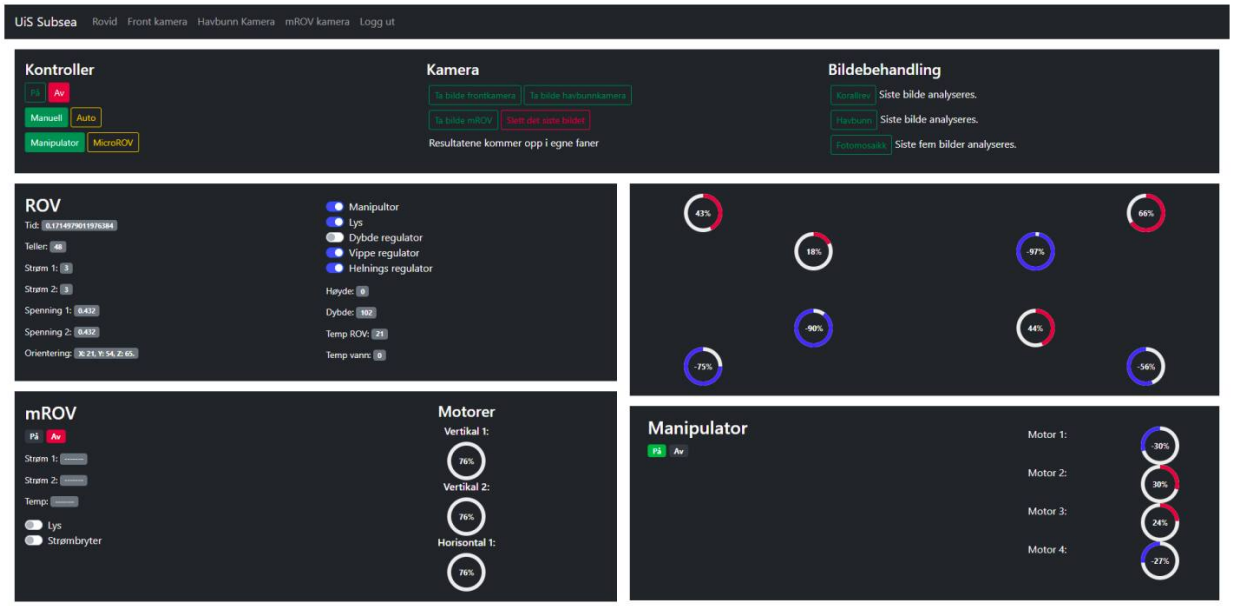
Det ble sett på EIVA sin styringspromgramvare for ROV og NaviSuite Mobula. i Disse kan man bestemme hva som skal vises fram basert på hva man trenger. Noen eksempler på dette elementer som kan vises legges til er en 3D-figur som illustrerer vinkel på enten rull eller stamp, skanning fra sonar og laser, dybdegraf og satellittkart over ROV-ens plassering [59]. Programvaren vises i figur 2.37 nedenfor.



Figur 2.37: Bildet viser EIVA NaviSuite Mobula ROV Steering Software [29], som ble sett på for å se hva som fantes av software i subsea-industrien.

2.16 Fjorårets brukergrensesnitt

Selve oppsettet av system er forskjellig fra i fjor, mens designmessig har det blitt tatt noe utgangspunkt i hvordan brukergrensesnittet så ut i fjor, vist i figur 2.38.



Figur 2.38: GUI-en til gruppen som hadde lagde det i fjor. Hentet fra [32, side 165].

Endringer av fjorårets brukergrensesnitt

En del designmessige endringer skal gjøres og noe kan beholdes. Vi skal fortsatt kunne velge mellom manuell og automatisk kjøring, kunne skru av og på manipulatoren, ta bilde med frontkamera og havbunnskamera, og kunne velge mellom forskjellige bildebehandlinger. Motorpådrag fra ROV-en sine thrustere og manipulatorens motorer er fortsatt relevant å ha med. Temperatur i ROV er fortsatt nødvendig å ha med. Siden det ikke var spesifisert i MATE ROV-konkurransen at vi må ha med temperaturmåling i vannet, valgte **Sensorkort-gruppen** [1.5.1] å ikke prioritere å sende opp sensordata på dette, men dette er noe som er mulig å implementere i ettertid.

I fjor brukte de bildebehandlingene; korallrev, havbunn, fotomoasaikk. I år er det andre bildebehandlingsoppgaver som skal utføres, gitt fra MATE ROV-konkurransen.

I år brukes det ikke mikro-ROV (se feltet for **mROV** i figuren 2.38, nederst til venstre), så det blir heller ikke relevant å ha med i år.

Kapittel 3

Implementasjon

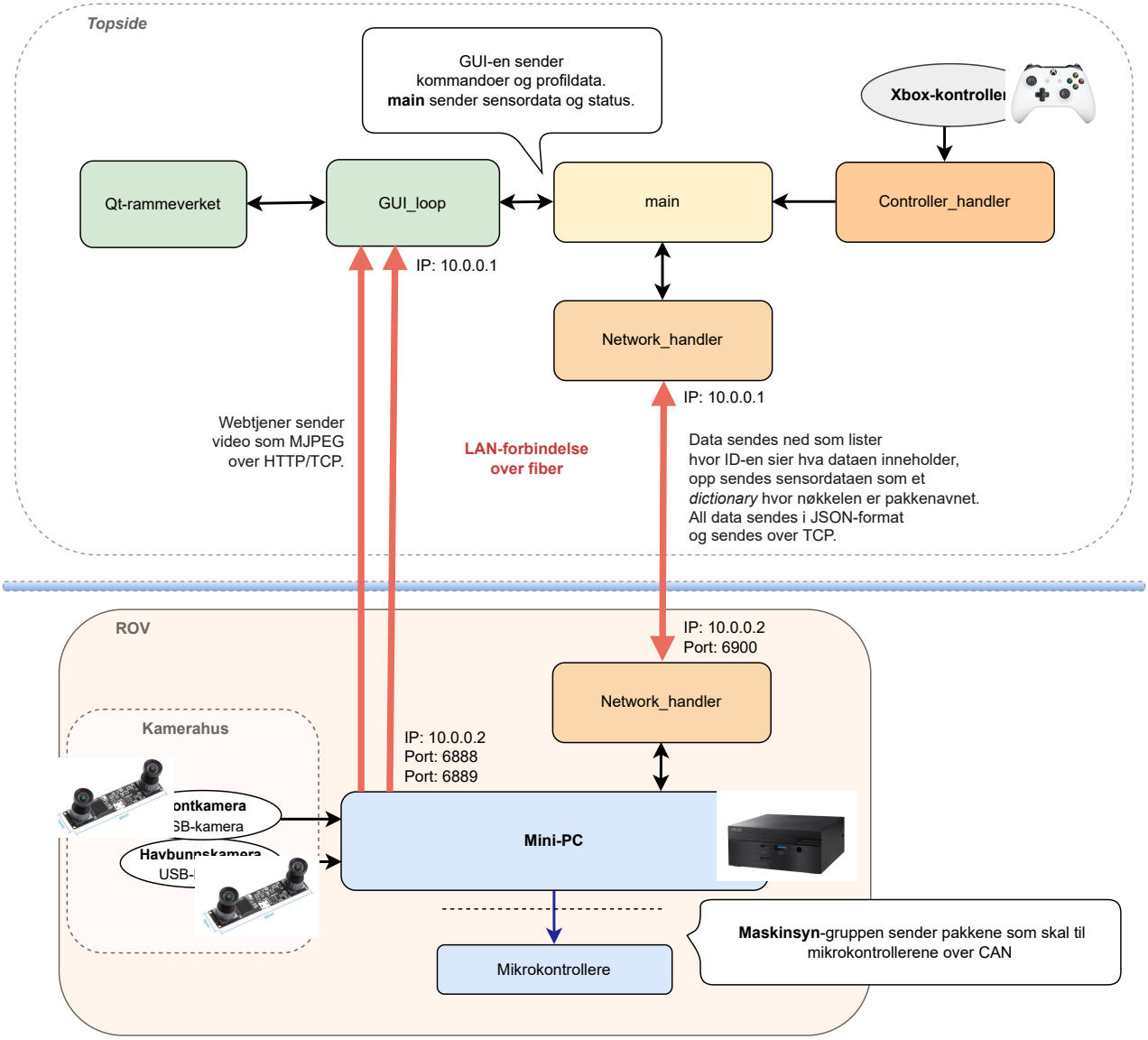
Kapitteloversikt

3.1 Oversikt over kommunikasjon og programflyt 53
3.2 Programflyt 54
3.3 Informasjonsoverføring mellom topside og mini-PC 55
3.4 Oversikt over data i GUI 56
3.5 Kontrolleren 57
3.6 Kontroll over et flertrådet program 60
3.7 Sanntidsvisning av videostrøm 61
3.7.1 Hvordan sendes videostrømmen? 61
3.8 Brukergrensesnittet 63
3.8.1 Tidlig utvikling 63
3.8.2 Brukergrensesnittet i år 63
3.8.3 Hovedsiden 64
3.8.4 Kontroller-siden 74
3.8.5 Moderne utseende 74

Dette kapittelet viser oversikt over kommunikasjon og programflyt. I tillegg blir det gjennomgått implementasjon av kommunikasjonen, kontroller-programmet, sanntidsvisning av videostrømmingen, og til slutt brukergrensesnittet.

3.1 Oversikt over kommunikasjon og programflyt

I figur 3.1 kan *topside*-kommunikasjonen i UiS Subsea-prosjektet sees.



Figur 3.1: *Topside*-systemet.

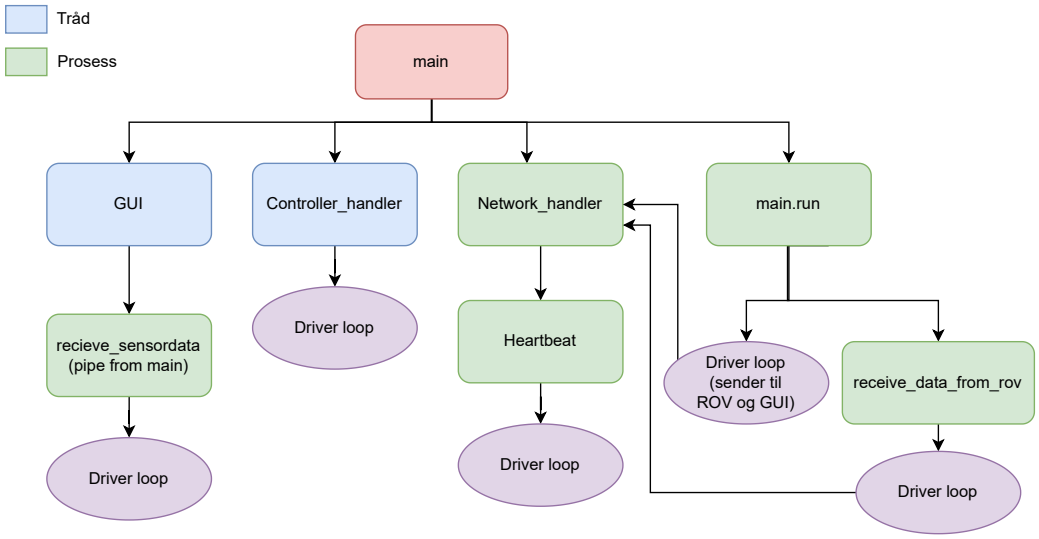
I tabellen, vist i figur 3.2, kan man se oversikten over de forskjellige portene som brukes og IP-adresse til mini-PC-en (som også kan sees i figur 3.1 av *topside*-systemet)

Ethernet					
Port	Protokoll	IP	Data	Mottaker	Sender
6898	TCP	10.0.0.2	Bilde kamera bak	Topside	ROV
6899	TCP	10.0.0.2	Bilde kamera front	Topside	ROV
6900	TCP	10.0.0.2		Topside, ROV	Topside, ROV

Figur 3.2: Oversikt over nettverkskommunikasjonen i prosjektet.

3.2 Programflyt

Hvordan programflyten foregår kan sees i figur 3.3.



Figur 3.3: Diagram over programflyten.

Programmet startes ved å kjøre **main.py**. Deretter starter **main** opp en rekke prosesser og tråder.

Først startes GUI-en i en prosess. GUI-en går da inn i en *loop* som oppdaterer elementene i GUI-en, men det er ikke tatt med i diagrammet siden dette blir håndtert av Qt-rammeverket. Det neste som skjer i GUI-en er at det startes opp en ny tråd som kontinuerlig henter ut data ved å gå inn i en *while*-løkke illustrert som **Driver loop** i diagrammet. Denne henter data fra en *pipe* som kommer fra **main** så lenge avslutt-signalet fra **Threadwatcher**-klassen ikke er sendt ut. **Threadwatcher** og avslutt-signalet forklares nærmere i kap. 3.6.

Controller_handler startes også som en prosess rett etter GUI-en. **Controller_handler** kobler til kontrolleren og går deretter inn i en *while*-løkke som henter ut data fra kontrolleren og sender det med en *pipe* (lagt opp som en FIFO-kø som flere kan sende inn i), helt til den mottar avslutt-signalet fra **Threadwatcher**-klassen. **Controller_handler** setter opp kontrolleren på nytt hvis denne skulle koble seg fra.

Deretter startes **Network_handler** som en tråd. **Network_handler** skal ta seg av TCP-koblingen opp mot ROV-en og starter derfor opp en ny tråd som sender *heartbeat*-pakker, som sjekker at det fortsatt er en TCP-kobling mellom mini-PC-en og *topside*-PC-en. *Heartbeat*-pakkene sendes med et *delay* på 300 *ms*. Hvis enheten er satt som server-siden starter **Network_handler** opp en ny tråd som venter på en ny tilkobling og erstatter den gamle i tilfelle koblingen faller ut.

Videre startes **run**-funksjonen i **main**-programmet som en tråd. Funksjonen **run** har som oppgave å koble alle de separate delene sammen. **run** starter først opp en ny tråd som skal ta imot nettverkstrafikk fra mini-PC-en (**recieve_data_from_rov**). Deretter går **run** inn i sin **Driver loop**, som gjør følgende:

- Henter ut data fra *pipe*-en som kontrolleren og GUI-en sender til.
- Bestemmer hvilken funksjon som skal kjøres når den mottar kommando fra GUI.
- Oppdaterer kontroller-profiler basert på data fra GUI.
- Oppdaterer statusen til ROV-en i **Rov_state**-klassen.
- Lager kommandoene som skal til mini-PC-en basert på data fra kontrollen og GUI-en.
- Sender inn sensordata som genereres i **main**. For eksempel tid, vinkel på kamera og om manipulatoren kjører.
- Sender kommandoer til mini-PC-en over TCP ved å bruke **Network_handler**.

Tråden som kjører funksjonen **recieve_data_from_rov** skal ta imot nettverkstrafikk fra mini-PC-en. Funksjonen er rimelig enkel. Den går inn i en *while*-løkke, hvor den tar imot nettverkstrafikk gjennom klassen **Network_handler** sin **receive**-funksjon.

3.3 Informasjonsoverføring mellom topside og mini-PC

Det mottas sensordata fra fire kretskort i ROV-ens elektronikkhus. Kretskortene er de følgende:

- Styrekort
- Sensorkort
- Hovedkort
- Kraftkort

Figur 3.4 viser hvilke CAN-pakkene som sendes til *topside*. Oversikten viser hvilket kretskort det sendes fra, navnet på CAN-pakken som blir sendt og hva som sendes i CAN-pakkene.

Maskinsyn-gruppen [1.5.1] mottar pakkene fra CAN-bussen på mini-PC-en, konverterer de til Python-datatyper og setter navn på pakkene. Øverst i figur 3.4 kan man se *gyro*, som er nøkkelen i *dictionary*-en og verdien er sensordata som mottas fra sensorene. Tilsvarende blir det for de andre pakkene *accel*, *power_temp*, *lekk_temp* og *thrust*.

Beskrivelse CAN-pakker til topside

Fra	CAN-pakke navn	Hva	can_id
Sensorkort	gyro	{"gyro": [rull, stamp, hiv]}	80
Sensorkort	accel	{"accel": [accel_x, accel_y, accel_z]}	81
Kraftkort	power_consumption	{"power_consumption": [uint16_power_consumption_0_i_watt, uint16_power_consumption_1_i_watt, uint16_power_consumption_2_i_watt]}	90
Sensorkort	lekk_temp	Pakke fra sensorkort med et dictonary hvor verdiene er en byte som sier hvilke av lekkasje sensorene er på. 0 for ingen. Deretter 3 forskjellige temperatursensorer {"lekk_temp": [lekk, temp1, temp2, temp3]}	140
Styrekort	thrust	{"thrust": [int8 * 8]} Rekkefølge: hhf, hhb, hvb, hvf, vhf, vhb, vvb, vvf. horisontal/vertikal, høyre/venstre, framme/bak	170

Figur 3.4: Hvordan CAN-pakkene ser ut etter omforming fra **Maskinsyn**-gruppen.

En utsnitt av oversikten over CAN-pakkene som blir sendt, vises i figur 3.5. I tabellen (se figur 3.5) vises CAN-ID-en, hvilket kretskort som sender og kortet som mottar, i tillegg til innholdet i pakken.

CAN-pakker

ID (Desimal)	Sender	Innhold	Mottar
64	Hovedkort	oppstart ("start\n")	Styrekort
70	Hovedkort	Styredata int8 (X ,Y, Z, rotasjon, manipulator, fri, fri, throttling)	Styrekort
71	Hovedkort	oppdatere param (uint32 <ID>, float <parameterverdi>)	Styrekort
80	Sensorkort	Regpakke int16 [#1-2Byte: hiv(mm), #3-4: rull*10, #5-6: stamp*10] (hiv/rull/stamp = int16)	Styrekort
81	Sensorkort	Akselerometer int16 [#1-2Byte: x-aks*100, #3-4: y-aks*100, #5-6: z-aks*100] (X/Y/Z = int16)	Styrekort
90	Strømforsyning	Strømforbruk [(12v 1300w forbruk i watt) uint16, Strømforbruk (12v 240w forbruk i watt) uint16, Strømforbruk (5v 65w forbruk i watt) uint16] alle verdiene ganges med 10 slik at vi får desimaler og må gjøres om til float av mini-PC-en	Styrekort

....

Figur 3.5: ID med sender, innhold og hvilket kort som mottar. Se resten av tabellen i vedlegg B.

3.4 Oversikt over data i GUI

Figur 3.6 viser navn på *widget*-ene i brukgergrensesnittet og hvilken CAN-pakke som tilhører *widget*-en.

Oversikt over sensordata som skal til GUI		
Fullt navn på widget	CAN-pakke navn	Beskrivelse
label_effekt_elektronikk	power_consumption	index 2 i power consumption i watt. 5V 65W
label_effekt_manipulator	power_consumption	index 0 i power consumption i watt. 12V 1300W
label_effekt_thrustere	power_consumption	index 1 i power consumption i watt. 12V 240W
label_temp_ROV_hovedkort	lekk_temp	temperatur ved hovedkort
label_temp_ROV_kraftkort	lekk_temp	temperatur ved kraftkort
label_temp_ROV_sensorkort	lekk_temp	temperatur ved sensorkort
label_gjsnitt_temp_ROV	lokalt	gjennomsnittet regnes ut
label_HHF_percentage	thrust	hhf - horisontal høyre foran - horisontal/vertikal, høyre/venstre, framme/bak
label_HHB_percentage	thrust	hhb - horisontal høyre bak
label_HVB_percentage	thrust	hvb - horisontal venstre bak
label_HVF_percentage	thrust	hvf - horisontal venstre foran
label_VHF_percentage	thrust	vhf - vertikal høyre foran
label_VHB_percentage	thrust	vhb - vertikal høyre bak
label_VVB_percentage	thrust	vvb - vertikal venstre bak
label_VVF_percentage	thrust	vvf - vertikal venstre foran
mani_kraft_inn_ut	lokalt	Hvor stor kraft vi kjører manipulator inn og ut, ut er positive verdier, alltid 100%
mani_kraft_rotasjon	lokalt	Hvor stor kraft vi roterer manipulator, med klokka er positiv, alltid 100%
mani_kraft_gripe_slippe	lokalt	Hvor stor kraft gripemotor får, gripe er positive verdier, alltid 100%

.....

Figur 3.6: Oversikt over sensordataen i GUI-en. Se resten av tabellen i vedlegg B.

Oversikten gjorde det lettere å legge til sensordataen i GUI-en siden all informasjon som trengtes var lett tilgjengelig.

3.5 Kontrolleren

Henting av kontrollerdata

Kontrollerdata ble hentet ut fra Xbox One-kontrolleren ved hjelp av PyGame. Det ble lagd en klasse for å modellere statusen på knappene i kontrolleren og til å holde hjelpemetoder.

Først initialiseres PyGame sine moduler på linje 42 og deretter kontrolleren på linje 43 med metoden **wait_for_kontroller**, som kan sees i figur 3.7, og til slutt initialiseres PyGame sin klokke. Klokkens funksjon vil bli gått inn på nærmere senere i dette kapittelet.

```
42        pygame.init()
43        self.wait_for_controller()
44        self.clock = pygame.time.Clock()
45
```

Figur 3.7: Initialiseringsprosess for klassen.

Sjekk om kontrolleren er initialisert

wait_for_kontroller sjekker om kontrolleren allerede er initialisert for å kunne resette slik at en ny kontroller kan initialiseres. Dette kan sees i figur 3.8 på linje 59 og 60. Deretter kobles kontrolleren til PyGame-biblioteket i linje 64 til 68.

```
55      def wait_for_controller(self):
56          """wait_for_controller will attempt to connect until it finds a controller."""
57          while self.t_watch.should_run(self.id):
58              try:
59                  if not self.first_run:
60                      pygame.joystick.quit()
61                  else:
62                      # clear_screen()
63                      print("Attempting to Connect to controller")
64                  pygame.joystick.init()
65                  global joystick
66                  if pygame.joystick.get_count() > 1:
67                      print("Found several controllers. Connecting to controller 0 only")
68                  joystick = pygame.joystick.Joystick(0)
69                  print(f"Connected to {joystick.get_name()}")
70                  break
71              except:
72                  self.first_run = False
73                  for sec in range(5,0,-1):
74                          sys.stdout.write("\r" + f"Could not find controller. if it is a
75                          time.sleep(1)
76                          sys.stdout.flush()
77          joystick.init()
```

Figur 3.8: Metoden for initialisering av kontrolleren.

Hente *events* fra kontrolleren

I figur 3.10 er det et utdrag fra koden, som viser metoden **get_events_loop**. Denne skal som navnet tilsier, hente ut hendelser eller *events*.

Styredata

Figur 3.9 viser et utdrag av hvordan styredata-pakken blir lagd.

```
403     def build_styredata(self):
404         #  X, Y, Z, rotasjon, (m.teleskop, m.vri, m.klype, aktiv), rull(ikke implementert), stamp(ikke implementert), struping
405         if self.data == {}:
406             return
407         styredata = []
408         styredata.append(self.data["joysticks"][X_axis])
409         styredata.append(self.data["joysticks"][Y_axis])
410         styredata.append(-self.data["joysticks"][Z_axis]) # Nedover er definert som positiv retning
411         if not self.camera_tilt_control_active:
412             styredata.append(self.data["joysticks"][ROTATION_axis])
413         else:
414             styredata.append(0)
415         styredata.append(self.build_manipulator_byte())
416         styredata.append(0)
417         styredata.append(0)
418         styredata.append(self.thruster_struping)
419         self.packets_to_send.append([70, styredata])
```

Figur 3.9: Utsnitt av hvordan styredataen sendes.

Threadwatcher - stopping av tråder

Det er ønskelig at man kan stoppe trådene og det sendes derfor inn et objekt av klassen **Threadwatcher**, som forklares nærmere i **Kontroll over et flertrådet program (kap. 3.6)**.

På linje 127 i figur 3.10, kan man se bruken av **Threadwatcher**-klassen som gir ut en boolsk verdi. Verdien fra metoden forteller tråden om den skal avsluttes eller fortsette å kjøre. Videre sjekkes det at det er en kontroller koblet til, på linje 128, og hvis det ikke er det, så kjøres initialiseringen på nytt som tidligere forklart i **Sjekk om kontrolleren er initialisert (kap. 3.5)**

```
126     def get_events_loop(self, t_watch: Threadwatcher, id: int, debug=False, debug_all=False):
127         while t_watch.should_run(id):
128             if pygame.joystick.get_count() < 1:
129                 self.wait_for_controller()
130             self.duration = self.clock.tick(20)
131             # print(duration)
132             for event in pygame.event.get():
133                 # print("entered event check")
134                 if event.type == DPAD: #dpad (both up and down)
135                     self.dpad = event.value
136                     # self.dpad = [val*100 for val in event.value]
137 
138                 if event.type == BUTTON_DOWN: #button down
139                     self.buttons[event.button] = 1
```

Figur 3.10: Metoden for henting av kontrollerdata.

Til slutt går metoden **get_events_loop** gjennom hendelsene som metoden **pygame.event.get** gir ut, som vist på linje 132-139 i figur 3.10. Den samler inn knappene, *directional pad* (**Dpad**) og *joysticks* (alle analoge knapper) i 3 forskjellige variabler. Knappene samles i en liste med verdier hvor **1** betyr at knappen er trykket ned og **0** for knappene som ikke er trykket ned. De analoge knappene gir ut rasjonale verdier fra **-1 til 1**. Disse lagres i en liste, men først er de innom en funksjon som normaliserer variabelene basert på hva disse skal brukes til og setter verdien til **0**, hvis denne er mindre enn dødsoneverdien som er satt.

Tick-metoden

PyGame-metoden **tick** til klokke-objektet **pygame.time.Clock** kjøres på linje 130, se figur 3.11. Denne sørger for at programmet henter ut verdier med jevne mellomrom. Metoden **clock** har en tidtaker som resettes og returnerer tiden siden sist gang **tick**-metoden ble kalt.

```
130             self.duration = self.clock.tick(20)
```

Figur 3.11: Tick-metoden.

Hvis tiden siden sist gang **tick**-metoden ble kalt er mindre enn $\frac{1s}{Hz}$, så venter **tick**-metoden til den spesifiserte tiden har gått. I dette tilfellet, så er $Hz = 20$. Vi får da likning 3.1.

$$\frac{1s}{20Hz} * 1000ms = 50ms \tag{3.1}$$

Resultatet fra likning 3.1 viser 50 *ms*, som vil si at dersom tiden siden **tick**-metoden ble kalt, er mindre enn 50 *ms*, så venter metoden til det har gått 50 *ms*. *Hz* kommer fra parameteren som sendes inn i **tick**-metoden. I PyGame-terminologi brukes *fps*, bilder per sekund, men siden GUI-en sin oppdateringsfrekvens ikke er avhengig av PyGame blir det riktigere å bruke *Hz* som vil si oppdatering per sekund. Tiden som har gått blir lagret i en **duration**-variabel (linje 130), og brukes til å regne ut forandring i helningsvinkelen på kameraet.

Oppdatering av kamera-tilt

Tiden som er lagret **duration**-variabelen, brukes til å regne ut forandring i helningsvinkelen på kameraet. Dette kan sees på linje 349 i figur 3.12. Her integreres forandringen i helningvinkel, med hensyn på tiden mellom oppdateringene som er lagret i verdien for *dictionary*-en kalt **data**, med nøkkelen **time_between_updates** (se linje 349).

```
338     def update_camera_tilt(self):
339         # print("camera tilt update func")
340         if self.camera_tilt_control_active and self.camera_tilt_allowed[self.active_camera]:
341             camera_movement = self.data.get('camera_movement')
342             if camera_movement is None:
343                 return
344             if abs(camera_movement) > 0:
345                 old_tilt = self.camera_tilt[self.active_camera]
346                 tilt_time_sec = 2  # time in seconds for the camera to move from one side to the other
347                 total_degrees = 80
348                 tilt_per_ms = total_degrees/(tilt_time_sec*1000)
349                 self.camera_tilt[self.active_camera] += (camera_movement/100)*self.data["time_between_updates"]*tilt_per_ms
```

Figur 3.12: Utregning av ny helningsvinkel på kameraet.

3.6 Kontroll over et flertrådet program

For å ha kontroll over trådene i programmet brukes flere metoder. En av disse er klassen **Threadwatcher** som forteller trådene om de skal kjøre med metoden **should_run** som returnerer en boolsk verdi basert på ID-en til klassen. Metoden **should_run** kan sees i figur 3.14 fra linje 11-14. Alle trådene får sendt inn et objekt av typen **Threadwatcher** og en ID som kommer fra **add_thread**-metoden som kan sees fra linje 6-9 i figur 3.14. Deretter sjekker trådene at **should_run**-metoden gir *True* og avslutter hvis ikke. For at **Threadwatcher** skal fungere er det viktig at trådene får inn oppdateringene som skjer i andre tråder. For å dele oppdateringene i objektet mellom tråder settes disse som *daemon*-tråder. At trådene har *daemon*-parameteren satt til *True* har også sideeffekten av at tråden vil avsluttes når hovedtråden i programmet avsluttes. Dette er ønskelig siden trådene kun utfører hjelpeoppgaver som ikke lenger er nødvendig når programmet avsluttes. Dette kan være et problem for eksempel hvis programmet skriver til en fil, mens det avsluttes. Da kan filen bli ødelagt, eventuelt kan filen bli stående åpen, noe som ikke er anbefalt. Eksempel på starting av en tråd med innsending av både ID og et objekt kalt **t_watch** av typen **Threadwatcher**, kan sees i figur 3.13.

```
655
656         id = t_watch.add_thread()
657         main_driver_loop = threading.Thread(target=run, args=(network, t_watch, id, queue_for_rov, gui_parent_pipe), daemon=True)
658         main_driver_loop.start()
```

Figur 3.13: Eksempel på bruk av **Thread**-klassen i Python sitt innebygde *threading*-bibliotek

Også den andre metoden for sending av data mellom tråder som brukes i programmet vises i figur 3.13, hvor det sendes inn en *Queue* fra Python sitt innebygde *multiprocessing*-bibliotek

som i dette tilfellet kalles **queue_for_rov**. Denne *queue*-en fungerer slik at flere tråder kan sende inn, mens en tråd tar ut. *Queue*-en er av typen FIFO (First In First Out). Både **Controller_handler** og GUI-en sender inn data som skal brukes av tråden. **Controller_handler** sender inn hvilke knapper og *dpad*-er som er trykket inn i tillegg til de analoge knappene, mens GUI-en sender inn oppdatering av hvilke knapper som hører til hvilke funksjoner på kontrolleren og justering av variabler som skal sendes ned til ROV-en som for eksempel lysstyrke på LED-lysene ROV-en har under og foran.

```
1  class Threadwatcher:
2      def __init__(self) -> None:
3          self.id = -1
4          self.threads = {}
5
6      def add_thread(self) -> int:
7          self.id += 1
8          self.threads[self.id] = True
9          return self.id
10
11     def should_run(self, id)-> bool:
12         if id in self.threads:
13             return self.threads[id]
14         return True
15
16     def stop_thread(self, id):
17         self.threads[id] = False
18
19     def stop_all_threads(self):
20         print("ThreadWatch: attempting to stop all threads")
21         for i in range(len(self.threads)):
22             self.stop_thread(i)
23         print(self.threads)
```

Figur 3.14: Threadwatcher-klassen som brukes for å signalisere til trådene om de skal kjøre.

3.7 Sanntidsvisning av videostrøm

3.7.1 Hvordan sendes videostrømmen?

Sending av videostrømmen skjer ved at mini-PC-en starter en entrådet webserver som leverer en MJPEG-bildestrøm. Hvordan MJPEG fungerer er forklart i kapittel 2.2.

MJPEG i nettleseren fungerer ved at det sendes en spesiell header som gjør at nettleseren erstatter JPEG-bildet som er tidligere vist. Headeren, som kan sees på linje 20 i figur 3.15, har `boundary` satt til **''**`frame`**''**, det vil si at nettleseren erstatter det tidligere bildet når den kommer til **''**`-frame`**''** (linje 28), som markerer starten på et nytt bilde.

Metoden **wfile** til **BaseHTTPRequestHandler**-klassen, brukes på linje 32 og 28, og den skriver til den åpne koblingen mellom nettleseren og webserveren.

```
16 def do_GET(self):
17     """Handles the get requests to the website based on the file extension"""
18     if self.path.endswith('.mjpg'):
19         self.send_response(200)
20         self.send_header('Content-type','multipart/x-mixed-replace; boundary=frame')
21         self.end_headers()
22         while True:
23             try:
24 
25                 img = self.server.pipe.recv()
26 
27                 _, jpg = cv2.imencode(".jpg", img)
28                 self.wfile.write(b"--frame\n")
29                 self.send_header('Content-type','image/jpeg')
30                 self.send_header('Content-length',str(len(jpg)))
31                 self.end_headers()
32                 self.wfile.write(bytes(jpg))
33                 # time.sleep(0.016)
34             except KeyboardInterrupt:
35                     break
36         return
```

Figur 3.15: Klassen som sender MJPEG-streamen over HTTP.

Koden er en modifisert versjon av kildekode funnet på Github [60]. Koden ble skrevet om fra Python 2 til 3 og mottar bildedataen fra **Maskinsyn-gruppen** etter at de har prosessert den og eventuelt tegnet på bildet. Videre måtte programmet gjøre slik at det bytter port basert på hvilket kamera det mottar video fra.

I den første prototype-løsningen tok mini-PC-en inn video fra kameraene og sendte opp rådata over UDP, men det viste seg fort å kreve for mye båndbredde.

I den første prototype-løsningen ble videoen sendt som rådata over UDP, men dette viste seg fort å kreve for mye båndbredde[1], men siden det var lettere å starte en nettleser i Qt og bruke denne til å se på en videostrøm over HTTP i tillegg, gikk vi for løsningen som ble forklart i kap. 3.7.1. Dette løste også et problem hvor det var streker med ødelagt data nederst i videoen.

Videre unngås det å sende videostrømmen gjennom **main** (kap 3.2) som hadde krevd mer prosessorkraft enn å bare ta imot direkte i GUI-en. Om det hadde blitt laget en ny kobling mellom GUI-en og mini-PC-en ville dette økt kompleksiteten i GUI-en ytteligere.

[1] ca 2,6 gigabit på en 1 gigabit link $2560x720x24x60$

3.8 Brukergrensesnittet

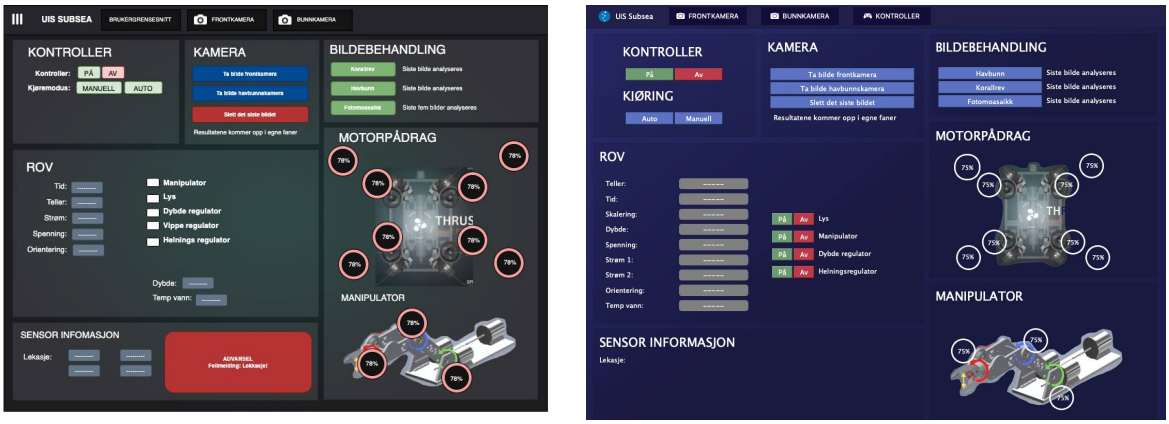
Det blir gjennomgått tidlig utvikling av brukergrensesnittet og videre implementasjonen av det. Det blir og gjennomgått de forskjellige komponentene det består av.

3.8.1 Tidlig utvikling

Skisse som ble laget helt i starten av prosjektet vises i figur 3.16a. Skissen ble laget i grafteg-ningsverktøyet `www.diagrams.net`. Fordelen med å lage en skisse fra start er for å kunne avklare hvordan informasjonen skal vises og hva slags funksjonalitet brukegrensesnittet skal ha.

I tillegg kan en skisse brukes som et hjelpemiddel i samarbeidet mellom gruppene i UiS Subsea, slik at gruppene som jobber med sensordataen fra ROV-en, får oversikt hva slags informasjon de skal sende opp og vise fram i brukergrensesnittet.

Figur 3.16b viser hvordan GUI-en ble seende ut etter å ha blitt konstruert i PyQt. Dette var et av de første stadiene i utviklingsfasen.



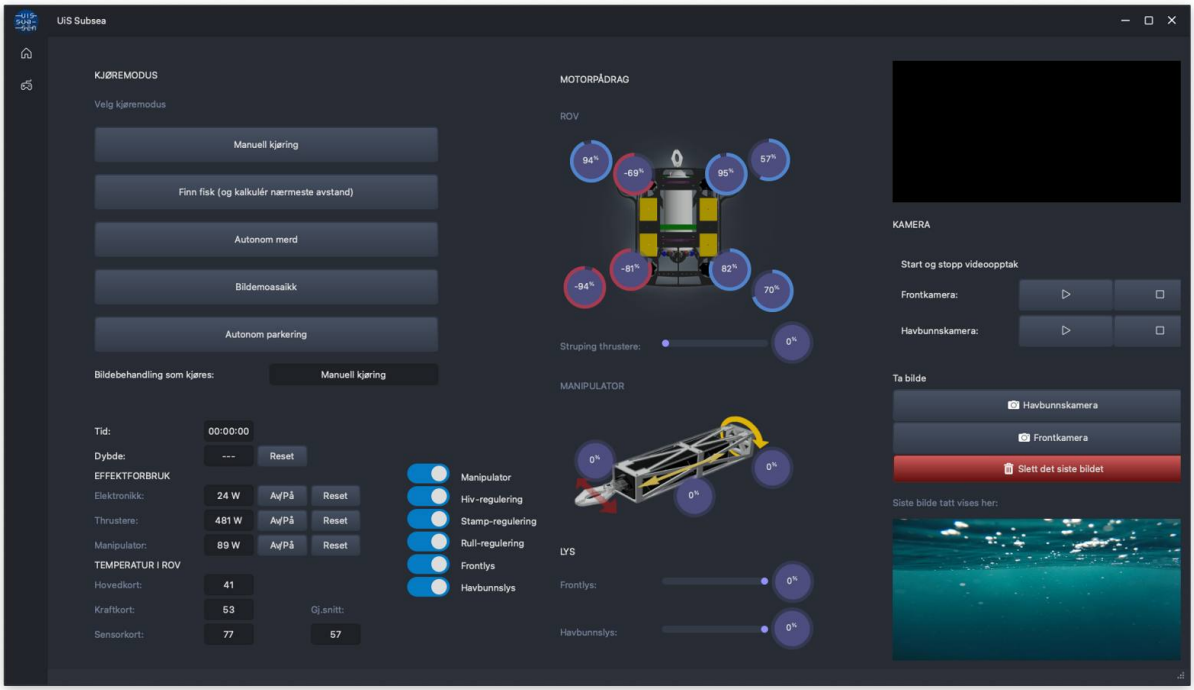
(a) Tidlig skisse av GUI.

(b) Tidlig versjon av GUI-en laget i Qt.

Figur 3.16: Startfasen av GUI-utviklingen.

3.8.2 Brukergrensesnittet i år

Det er blitt gjort en god del utseendemessige endringer fra fjorårets GUI, som kan sees i figur 3.17. Det er også blitt lagt til andre funksjonaliteter. Det var ønskelig å gjøre det med et mer moderne preg og bedre visualisert med figurer av ROV og manipulator, i tillegg til en 3D-modell som viser stamp og rull, når ROV-en styres.

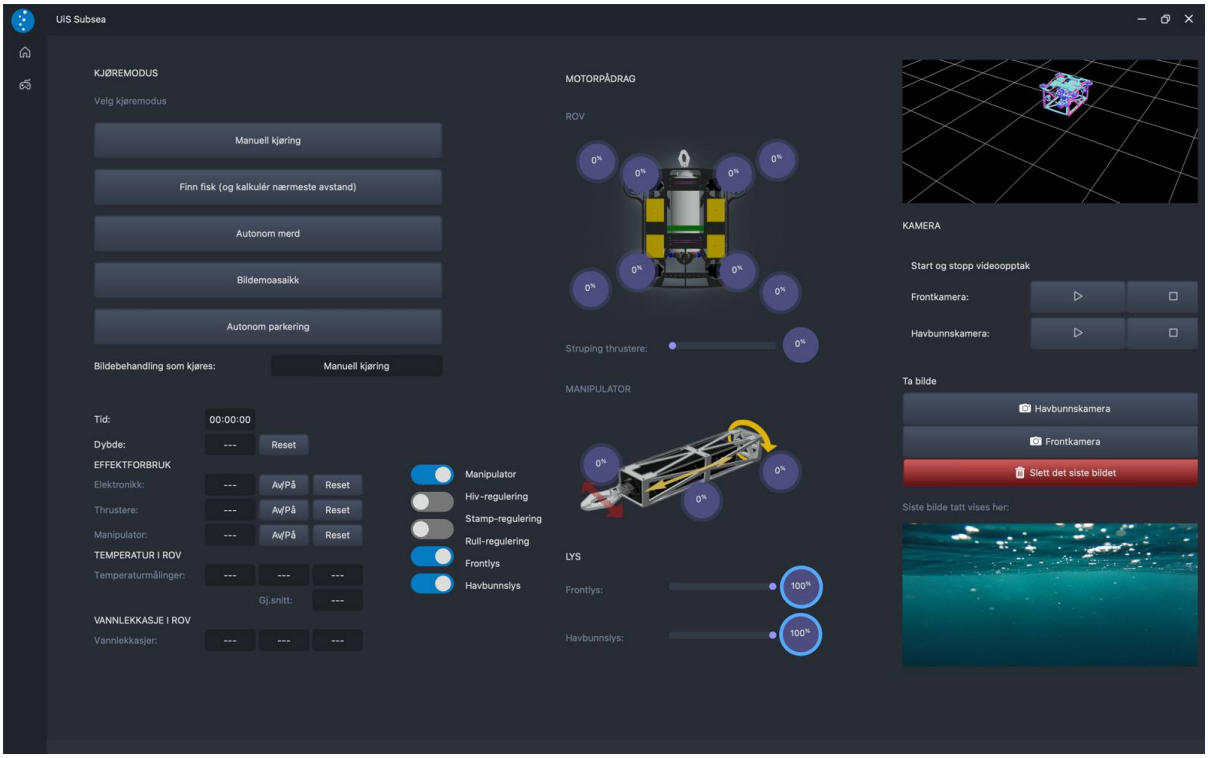


Figur 3.17: GUI som skal brukes i år.

Det er lagt til sidemeny på venstre side. Det er en knapp for hjemmesiden med all sensorinformasjonen, og en knapp for å komme til kontroller-siden hvor er det mulig å endre oppsettet til kontrolleren, i tillegg gir figuren informasjon om hva de ulike knappene på kontrolleren er satt til. Brukergrensesnittet blir forklart mer i kapittel 3.8.

3.8.3 Hovedsiden

Hovedsiden består av tre kolonner bortover, som du kan se i figur 3.18. I første kolonne vises kjøremodus og informasjon om sensordata. I andre kolonne vises motorpådraget til ROV og manipulator, og slidere til lys. I tredje kolonne vises 3D-modellen og knapper relatert til kamera. Vi går igjennom de forskjellige komponentene hovedsiden består av mer inngående i de neste kapitlene.



Figur 3.18: Hvordan hovedsiden ser ut. (Oppdateres når vi er ferdige)

Fremvisning av sensordata

I figur 3.19, kan fremvisningen av data vi mottar fra ROV-ens sensorer sees. Det som vises fram er dybden til ROV-en, effektforbruket og temperaturer inni ROV-en. I tillegg vises tiden fra ROV-en blir startet, og en stoppeklokke som kan brukes til å ta tiden.

Dybden til ROV-en måles fra nullpunktet, som settes ved å trykke på **Reset**-knappen og ved oppstart av programmet.

Effektforbruket vises for elektronikken, thrusterene og manipulatoren i ROV-en. Bakgrunnen for verdiene skiftes til rød for å varsle, dersom effektforbruket til elektronikken overstiger 65 W, thrustere overstiger 1300 W og manipulatoren overstiger 240 W. Grensen for når rød bakgrunn skulle vises fram ble satt av kraftforsyningsgruppen

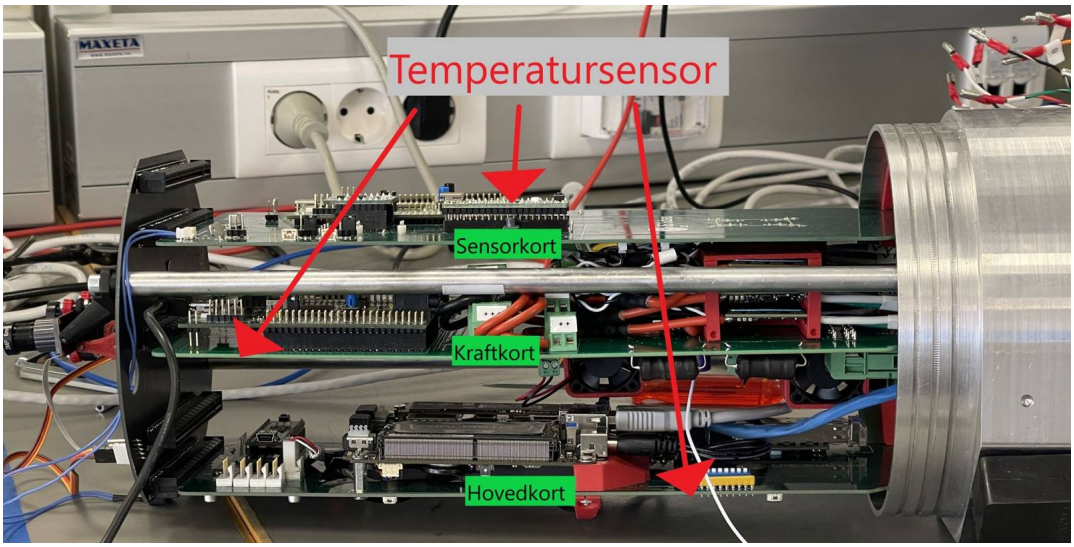


Figur 3.19: Framvisning av sensordataen.

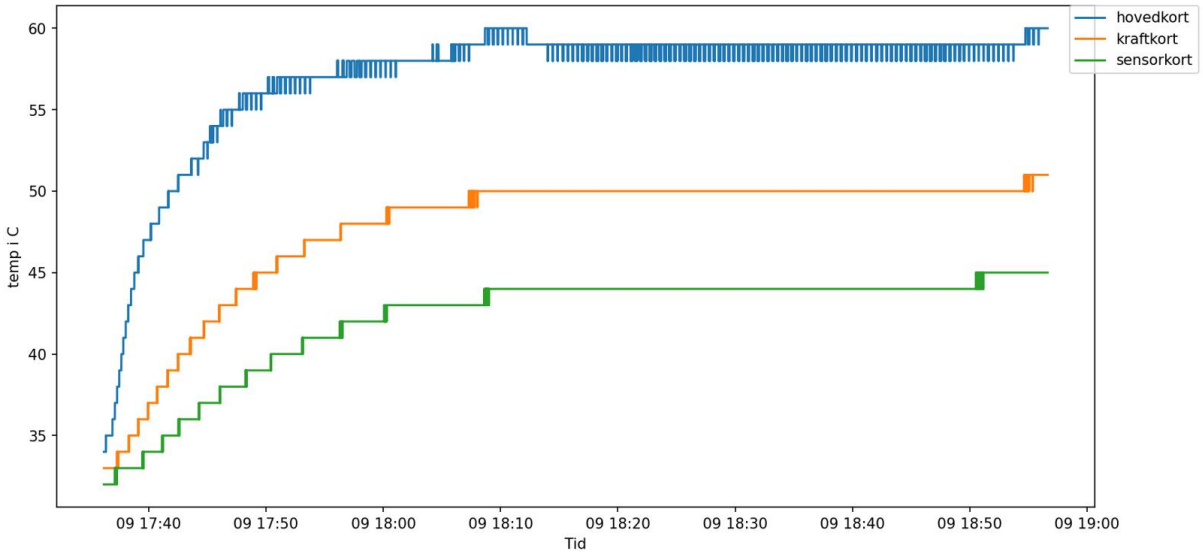
Hvor temperatursensorene er plassert kan sees i figur 3.20. Temperatur inni ROV måles rundt hovedkortet, kraftkortet og sensorkortet, i tillegg til at gjennomsnittet av de tre temperaturene regnes ut.

I figur 3.21 kan temperaturen under kjøring i basseng ved 30 grader sees. Ut i fra denne grafen og observasjoner fra tidligere kjøringer ble det valgt at bakgrunnen til henholdsvis hovedkortet, kraftkortet og sensorkortet sin temperatur

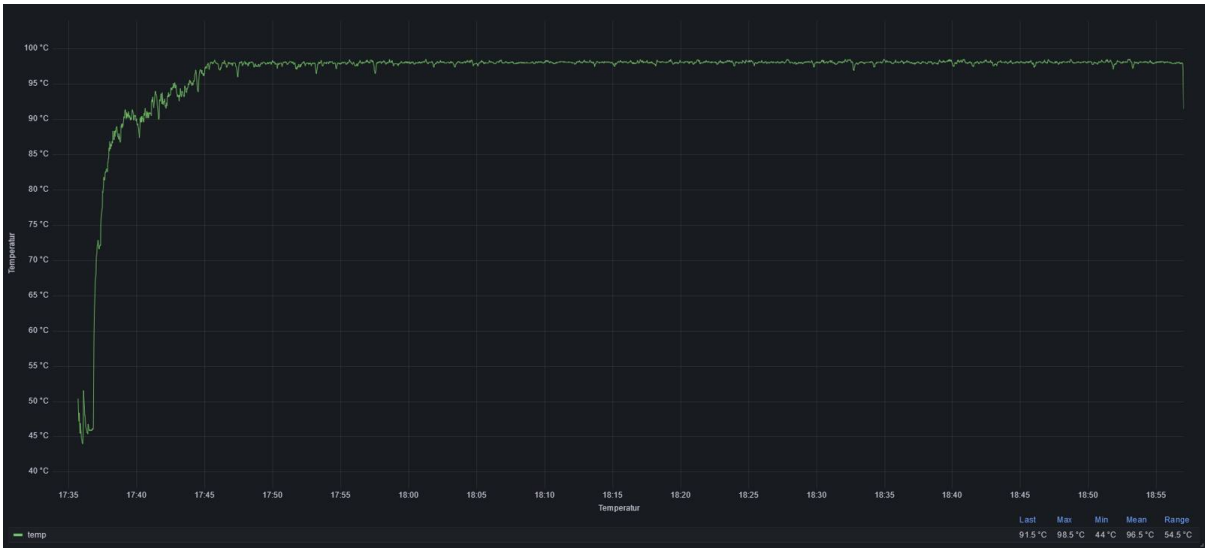
skulle bli rød ved 62, 52 og 47 grader. Dette for å varsle oss om at temperaturen er høyere enn normalt.



Figur 3.20: Temperatursensorene inne i elektronikkhuset er plassert ved de røde pilene. Bildet og illustrasjonene er laget av sensorkort-gruppen



Figur 3.21: Temperatur under vanntesting av ROV. Elektronikkhuset blir varmet opp før det gjøres tett slik at trykket skal forandre seg mindre under kjøring



Figur 3.22: CPU Temperatur på mini-Pc-en under vanntesting av ROV. Utklipp av grafen og logging av CPU temperatur er gjort av Maskinsyn-gruppen

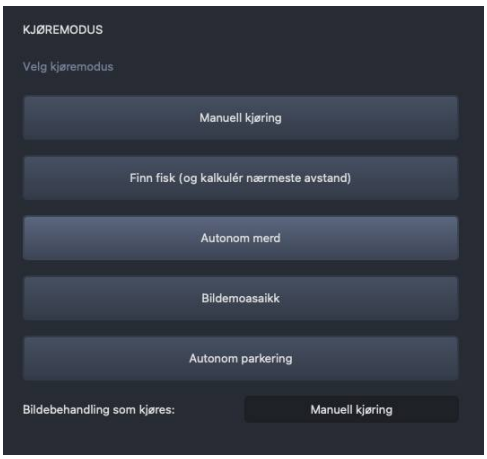
Prosessoren på mini-PC-en har en *TJ-max* på 105 grader [61], som vil si det punktet prosessoren starter å kraftig redusere spenning til prosessoren, men grafen i figur 3.22 tyder på at CPU-en *thermal throttler* slik at temperaturen holder seg på 98 grader [61]. Denne teorien støttes også av at mini-PC-en sliter med å hente bilder i tide fra kameraet som peker nedover som gjør at det sendes opp et testbilde istedenfor faktiske bilder fra kameraet. Dette kan utbedres ved å optimalisere programmene som kjører på mini-PC-en eller å øke kjølevnen, men dette ligger utenfor vårt område og vil bli tatt hånd om av Maskinsyngruppen etter at bacheloroppgaven er levert inn, men før konkurransen.

Kjøremodus

Det er mulig å velge mellom de ulike kjøre- og bildebehandlingsmodusene, som vist i figur 3.23.

De forskjellige bildebehandlingsmodusene; **finn fisk**, **autonom merd**, **bildemoasaikk** og **autonom parkering**, er de som det ble planlagt å legge til i kapittel 1.6.5. Manuell kjøring er den modusen ROV-en starter med.

Nederst vises det hvilken bildebehandlingsmodus som er valgt, denne endrer seg dersom man trykker på en knapp for en annen bildebehandlingsmodus. Bildebehandlingen foregår nede på mini-PC-en.

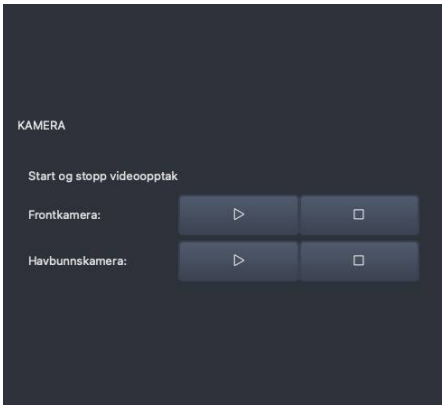


Figur 3.23: De forskjellige bildebehandlingsmodusene.

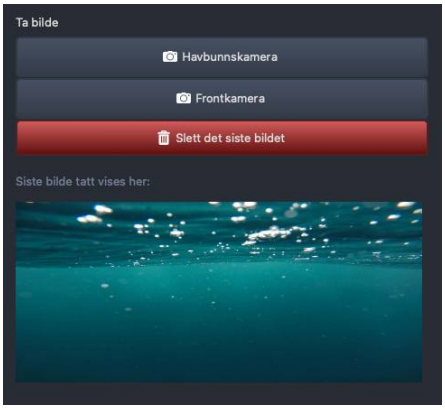
Videoopptak og bildetaking

Det er mulig å starte og stoppe videoopptaket fra frontkamera og havbunnskamera. En mulighet hadde vært å tatt opp video hele tiden, men på grunn av lagringsplass og den ekstra forsinkelsen dette ville medført tas det kun opp når det trengs.

Videre er det mulig å ta bilde med havbunnskamera eller frontkamera. Bildetaking brukes blant annet for å lage fotomosaikk av 8 bilder, som beskrevet i oppgavebeskrivelsen i kap. 1.3. Det var planlagt å ha mulighet til å slette bilder, og vise disse fram i GUI-en. Orginalt var planen fra vår side at maskinsyn gruppen skulle sende dette opp over nettet, men de valgte å lagre bildene lokalt istedenfor. Dette gjorde det vanskeligere å vise frem bildene i GUI-en, men det planlegges å legge til denne funksjonaliteten etter bachelorfristen, men likvel før MATE ROV konkurransen. Planen er å bruke *SSH* til å overføre bildene og deretter vise frem bildet med den siste modifikasjonsdatoen. At bildene ble lagret lokalt på ROV-en gjorde det også vanskeligere å slette bildene. Det planlegges at "slett siste bilde" skal slette det siste bildet som ligger inne til å lage fotomosaikk av, etter innlevering av oppgaven.



(a) Start og stopp av videoopptak.



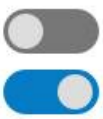
(b) Knapp for å ta bilde med havbunnskamera eller frontkamera, og til å slette det siste bildet. Siste bildet tatt vises også.

Figur 3.24: Kamera-innstillingene.

Toggle-buttons

Toggle-button er en type knapp som veksler mellom to tilstander; på eller av (se figur 3.25). Fordi det ikke finnes en ferdiglagd *toggle-button* i Qt-biblioteket, trengtes det å lage en egendefinert *widget* ved å bruke en `QCheckbutton`. For å lage den egendefinerte og animerte *toggle-button*-en, ble koden fra youtube-video [62] benyttet.

Bryterene lages, ved å først opprette klassen **PyToggle**, som arver fra QCheckBox. Størrelsen på `QCheckBox` og trykkområdet settes større. Det lages en animasjon med `QPropertyAnimation` som varer i 500ms. Rektangel blir tegnet med `QRect`, og sirkel inni blir tegnet med `QPainter` i et PaintEvent, som lager bakgrunnen (rektangel med avrundet kanter) og tegner sirkelen/ellipsen som er inni. Sjekker når `QCheckBox` er avkrysset, så plasseres sirkelen i venstre hjørne i stedet for høyre.



Figur 3.25: Hvordan en *toggle button* ser ut. Blå er den aktive fargen som indikerer at knappen er skrudd på.

Disse bryterene, vist i figur 3.26, brukes i GUI-en til å skru på eller av manipulatoren, hiv-, stamp- og rull-reguleringen. I tillegg til frontlys og havbunnslys

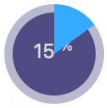


Figur 3.26: Hvordan *toggle-buttons* blir benyttet i GUI-en.

Sirkulære motorpådriftssindikatorer

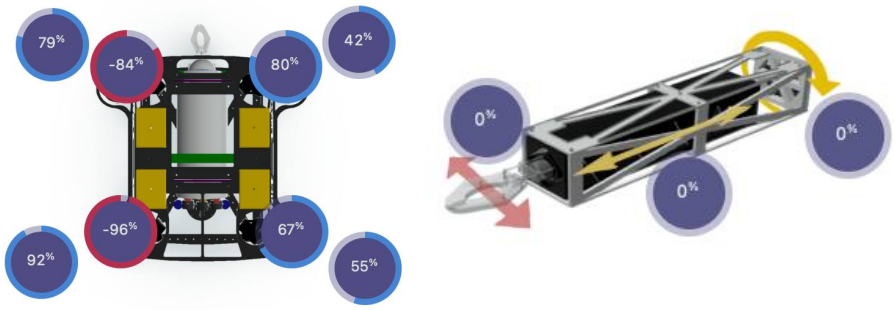
Hvordan de sirkulære motorpådragsindikatorene, vist i figur 3.27, ble lagt, blir beskrevet.

Det ble det brukt tre `QFrame` formet som sirkler, ved å bruke *CSS border-radius* *CSS gradient* ble brukt for å fylle opp den ene sirkelen avhengig av hvilken verdi fra motorpådraget som ble mottatt. Se figur 3.27 for hvordan bakgrunnsfargen på sirkelen fyltes opp med å bruke *gradient*, bestående av en blåfarge og en gjennomsiktig farge. Denne ble plassert bak den lilla sirkelen, slik at resultatet ser ut som en sirkulær fremdriftsindikator som viser fram motorpådraget. Dette er med på å gjøre brukergrensesnittet intuitivt, og operatøren får informasjon om hvordan thrusterene og manipulatorens motorer reagerer på trykk på kontrolleren.



Figur 3.27: Her vises den blå sirkelen, med *CSS gradient* fylt opp til 15%. Her i figuren er den blå sirkelen plassert fremst foran de andre sirkelene for illustrasjonen sin skyld, men skal egentlig være skjult bak den midterste, lilla sirkelen.

Figur 3.28 viser fram motorpådraget, som går fra $0 - 100$ %, til de åtte forskjellige thrusterene, og motorpådraget til manipulatorens tre motorer (viser enten 0 % eller 100 %). Akkurat nå er de satt med eksempelverdier, og ikke noe som ville vært et realistisk tilfelle.



Figur 3.28: Motordriftssindikatorene til ROV-en og til manipulatoren.

Koden ble funnet på et av Wanderson M. Pimenta sine GitHub-prosjekter [63], og hvordan dette ble gjort i Qt Designer kan sees i en tilhørende YouTube-video [64]. Funksjonene fra

[65], `setValue` og `progressBarValue`, ble brukt, men funksjonene ble skrevet til én funksjon, `update_round_percent_visualizer`, vist i figur 3.29. Funksjonen brukes til å oppdatere *CSS Gradient*-en i *StyleSheet*-en til sirkelen. Det er blitt gjort visse modifikasjoner på koden slik at prosenten også viser negative verdier og fargen skiftes til rød. Videre ser man at `htmlText` oppdateres for å vise endringen av **value** som er prosentverdien. `setText` metoden brukes for å oppdatere teksten til et tekstelement i PyQt.

Det var en utfording å skille mellom de vertikale og horisontale thrusterene, men etter at vi satt pådragsprosenten på toppen av motoren mente piloten at det var lettere å differensiere mellom motorene.

```
809     def update_round_percent_visualizer(self, value, text_label, round_frame):
810         """This function will update the various circles with a progress bar around and percent in the middle"""
811         blue = "rgba(85, 170, 255, 255)"
812         red = "rgb(226, 47, 53)"
813         progress = (100 - value) / 100.0
814         stop_1 = str(progress - 0.001)
815         stop_2 = str(progress)
816         color = blue
817         if value < 0:
818             stop_1 = str(progress - 1)
819             stop_2 = str(progress - 0.001 -1)
820             color = red
821         if value == 100:
822             stop_1 = "1.000"
823             stop_2 = "1.000"
824         htmlText = f"""<p align="center"><span style=" font-size:9pt;">{value}
825         </span><span style=" font-size:9pt; vertical-align:super;">%</span></p>"""
826         text_label.setText(htmlText)
827         styleSheet = """QFrame{ border-radius: 30px; background-color: qconicalgradient(cx:0.5, cy:0.5, angle:90,
828         stop:{STOP_1} rgba(255, 0, 127, 0), stop:{STOP_2} {COLOR}); }"""
829         styleSheet = styleSheet.replace("{STOP_1}", str(stop_1)).replace("{STOP_2}", str(stop_2)).replace("{COLOR}", color)
830         round_frame.setStyleSheet(styleSheet)
```

Figur 3.29: Funksjonen `update_round_percent_visualizer` som brukes for å oppdatere prosenten og *CSS gradienten* til de sirkulære motorpådragsindikatorene.

Slidere

Slidere brukes for å stille styrken på frontlys og havbunnslys, se figur 3.30.



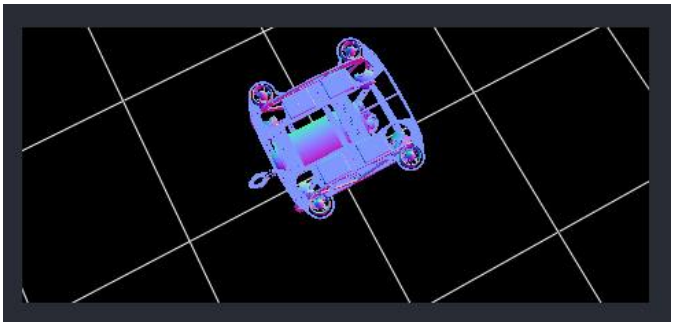
Figur 3.30: Slidere for å stille lysstyrken i prosent.

Videre brukes det en slider for å stille på strupingen av thrustere. Det vil si at man reduserer pådraget fra kontrolleren med strupeprosenten. Dette påvirker ikke reguleringen som skal holde ROV-en rett i vannet. Se slideren i figur 3.31.



Figur 3.31: Slider for å stille på struping.

3D-modell STL-fil av ROV kommer fra **ROV design**-gruppen [1.5.2], og modellen fra STL-filen kan sees i figur 3.32. Det ble også brukt et kodeutdrag fra **Maskinsyn-gruppen** for å vise frem 3D-modellen, også funnet på GitHub: `https://github.com/Be4rR/STLViewer/blob/master/stl-viewer.py`.



Figur 3.32: 3D-modell. (Oppdater med bedre bilde av figuren)

PyQtGraph For å vise frem 3D-modellen av ROV-en, brukes PyQtGraph. PyQtGraph bruker Open Graphics Library (OpenGL) for å vise fram 3D-grafikk. OpenGL er et API som brukes for å fremstille 3D-vektor grafikk. Det brukes også en Python-binding av OpenGL, kalt PyOpenGL. For teorien rundt fremstilling av vektorgrafikk se kap 2.3. For å vise fram 3D-objektet lages det en ny *view widget* ved å bruke `GLViewWidget()` fra `pyqtgraph.opengl` (fortkortet til `gl`):

```
self.viewer = gl.GLViewWidget()
```

I GUI-en er det lagt inn et vertikalt layout, `QVBoxLayout`. Metoden `.addWidget()` brukes for å legge *viewer*-en inn i GUI-en:

```
self.layout.addWidget(self.viewer, 1)
```

Til slutt vises *viewer*-en fram med:

```
self.viewer.show()
```

Hele kodeutdraget kan man se i figur 3.33. Utdraget viser hvordan `ROV.STL`-filen leses inn fra `current working directory`, ved å bruke Numpy-STL som lager en *mesh* av filen (linje 115). `GlMeshItem` framviser den i PyQtGraph (linje 121). Og *shader*-en eller skyggeleggingen til figuren er satt til `'viewNormalColor'`, som gir 3D-modellen av ROV-en den fargerike fargen (linje 122).

```
110         # STL viewer for ROV 3D model
111         self.viewer = gl.GLViewWidget()
112         self.viewer.opts['distance'] = 250
113 
114         cwd = os.getcwd() # Current working directory
115         self.stl_mesh = mesh.Mesh.from_file(f'{cwd}/ROV.STL') # Imported STL file
116         shape = self.stl_mesh.points.shape
117         points = self.stl_mesh.points.reshape(-1, 3)
118         faces = np.arange(points.shape[0]).reshape(-1, 3)
119 
120         self.meshdata = gl.MeshData(vertexes=points, faces=faces)
121         self.meshitem = gl.GLMeshItem(meshdata=self.meshdata, smooth=False, drawFaces=True,
122         shader='viewNormalColor', glOptions='opaque', color=(0,0,0,0), drawEdges=False, edgeColor=(0,0,0,0))
123         self.viewer.addItem(self.meshitem)
124 
125         self.meshitem.rotate(0, 0, 0, 0)
126         self.layout = self.QVBoxLayout
127         self.layout.addWidget(self.viewer, 1)
128         self.viewer.setCameraPosition(distance=350)
129         self.meshitem.translate(0,0, 60)
130 
131         g = gl.GLGridItem()
132         g.setSize(1000, 1000)
133         g.setSpacing(100, 100)
134         color = QColor("#ffffff")
135         g.setColor(color)
136         self.viewer.addItem(g)
137 
138         self.rotation = [0, 0, 0]
139 
140         self.viewer.show()
```

Figur 3.33: Kodeutdrag for å fremvise STL-3D-modellen.

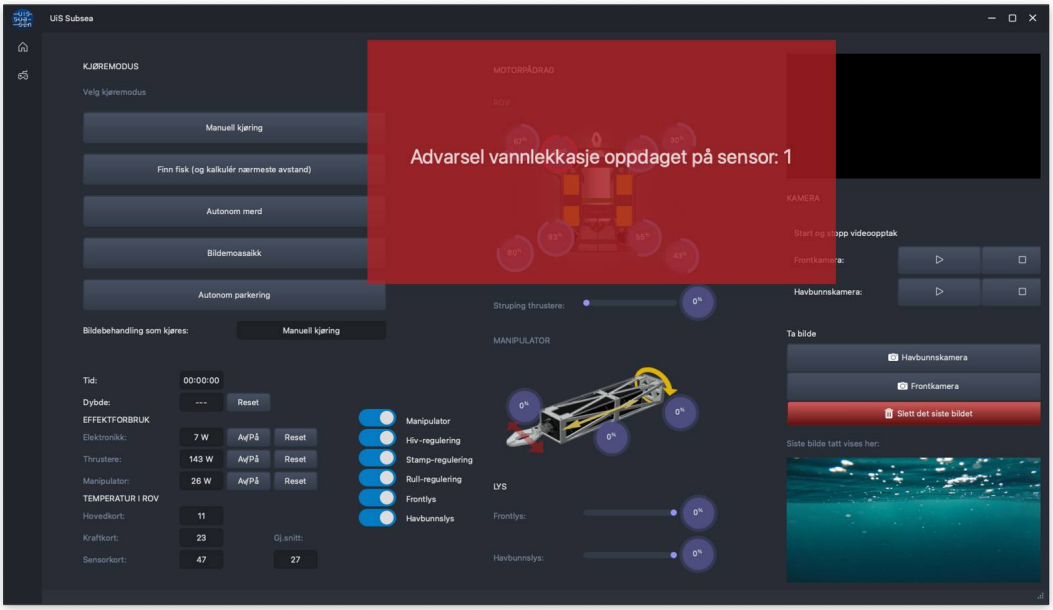
Rotasjon av ROV-3D-modellen Rotasjon av 3D-modellen blir utført i to operasjoner for å slippe å regne ut en rotasjonsvektor som tar både stamp og rull i en rotasjon. Dette er matematikk som ikke er gjenomgått på studiet og dette var ikke en prioritet, siden det funket greit å rotere i to omganger. En stor ulempe med dette er at det ved høyere utslag i rullvinkelen (siden denne roteres først) vil føre til større og større feil i stampvinkelen. Dette er fordi sensorene regner ut ifra det lokale koordinatsystemet, mens måten modellen blir rotert på er ifølge det globale koordinatsystem. I verste tilfelle som er 90 grader rullvinkel vil stampvinkelen bli påført vinkelrett på den egentlige rotasjonen. Dette er likevel ikke et stort problem siden ROV-en i første omgang er planlagt å skulle kjøres med regulering som holder stamp og rullvinkelen så nærme null som mulig. Ved små rullvinkler vil det bli små feil i modellen som vises fram og ved å så gjennom log filen fra en kjøring hvor det ble testet uten regulering og med herjing for å se hvordan ROV-en oppførte seg, var den største rullvinkelen 48 grader.

```
835     def gui_gyro_update(self, sensordata):
836         # Removes the previous rotation. We do not have yaw rotation
837         # so it is not necesarry to reset or rotate it
838         # print(f"{sensordata = }")
839         # hiv, rull, stamp
840         print(f"gyro update inside gui = {sensordata = }")
841         self.meshitem.rotate(-self.rov_3d_coordinates[1], 0, 1, 0, local=True)
842         self.meshitem.rotate(-self.rov_3d_coordinates[2], 1, 0, 0, local=True)
843 
844         self.meshitem.rotate(sensordata[2], 1, 0, 0, local=True)
845         self.meshitem.rotate(sensordata[1], 0, 1, 0, local=True)
846 
847         # height translation
848         height_diff = sensordata[0]-self.rov_3d_coordinates[0]
849         self.meshitem.translate(0,0, height_diff)
850 
851         self.rov_3d_coordinates = sensordata
852 
853         self.label_dybde.setText(str(round(self.rov_3d_coordinates[0]))+ " cm")
```

Figur 3.34: Funksjonen `gui_gyro_update` som roterer ROV-3D-modellen ettersom hva man mottar fra gyroskopet som **Sensorkort-gruppen** [1.5.1] har ansvar for.

Vannlekkasje varsel

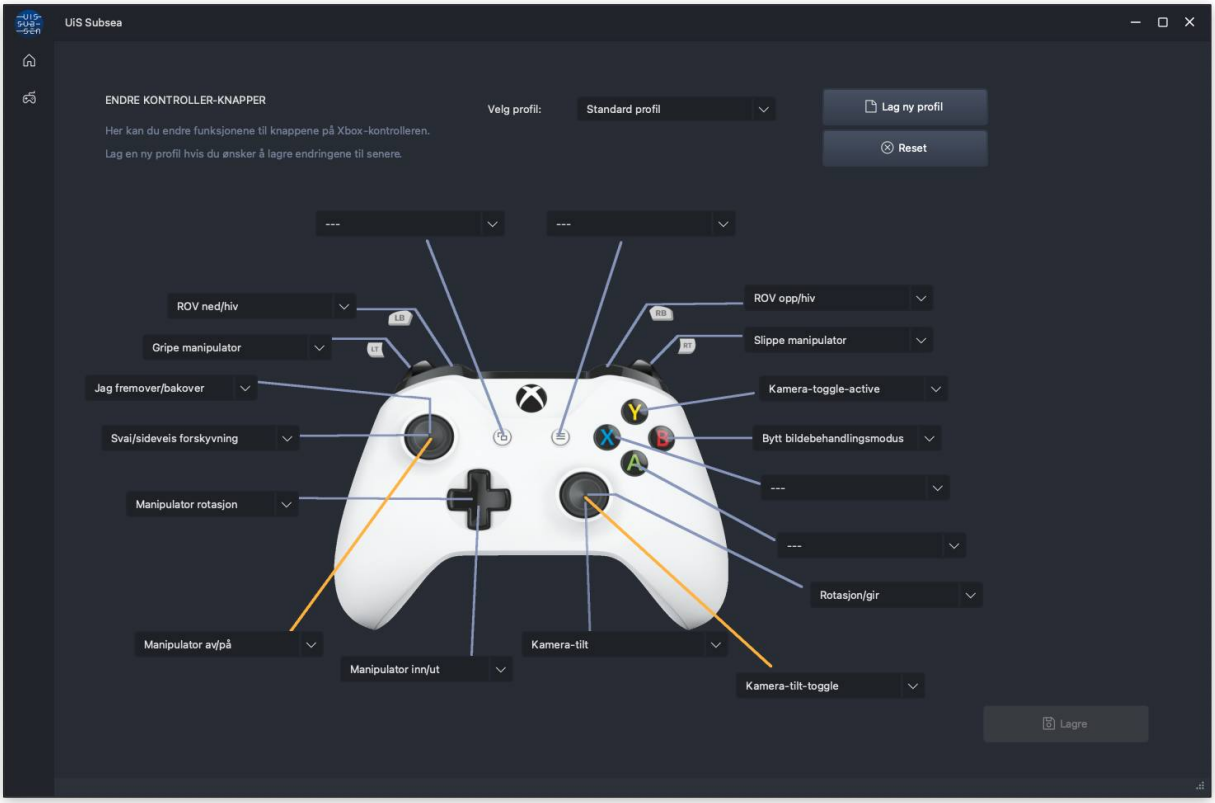
Varsel som kommer opp dersom vannlekkasje i ROV blir oppdaget av sensorkortet, kan sees i figur 3.35. Dette er kritisk, og det er derfor viktig at denne varselen synes, slik at man kan få ROV-en så fort som mulig opp av vannet. I tillegg blir en alarmlyd spilt av, for at man lettere skal oppdage varslingen. Hver gang ved oppstart av *topside*-programmet og ROV, vises lekkasjevarselet, slik at man er sikker på at det fungerer som det skal.



Figur 3.35: Varsel som kommer opp dersom vannlekkasje i ROV blir oppdaget av sensorkortet.

3.8.4 Kontroller-siden

Det ble brukt combobox'er til å endre funksjonen av kontrollerkappene. I tillegg fungerer det som en oversikt over hva de forskjellige knappene gjør.



Figur 3.36: Kontroller-siden.

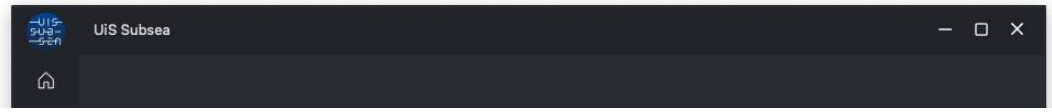
3.8.5 Moderne utseende

For å få et moderne utseende på PyQt-vinduet, ble koden brukt fra GitHub [66]. Dette gjør at Windows-vindustilen blir erstattet av en egendefinert stil. Koden fra *repository*-et ble skrevet om fra PyQt6 til PyQt5.

Ønsket en mørk modus, for dette kan være lettere å se på i sollys. Vi kunne også ha eller at vi burde ha hvitt modus også som valgmulighet, kunne også hatt mulighet til å endre modus.



Figur 3.37: Justering av vindu



Figur 3.38: Ikoner til å minimere, maksimere og lukke vinduet.

Kapittel 4

Økonomisk oversikt og miljørapport

Økonomisk oversikt over faktisk forbrukt materiell

Alle gruppene fikk et budsjett til å kjøpe inn komponenter og utstyr for, men det har ikke vært behov for å kjøpe inn utstyr, siden det var mulig å bruke materiell som allerede var kjøpt inn av oss eller UiS Subsea. Vi setter likevel opp et anslått budsjett basert på prisjakt.no, kvitteringer og anslag.

Vi har brukt 2 portable PC-skjermer, en laptop, kabler, en adapter, en Xbox-kontroller og en media konverterer.

UiS Subsea hadde allerede en Xbox One-kontroller, men prisjakt oppgir at en ny kontroller kan kjøpes for ca 700 kr. Utenom kontrolleren er resten av de fysiske gjenstandene lånt fra medlemmer i UiS Subsea eller vår gruppe. De 2 Lenovo Thinkvision 14/14t PC-skjermene ville kostet ca 6000 kr nye. Laptopen kostet 12 000 kr, men en billigere og svakere laptop eller en billigere stasjonær PC klarer å kjøre programmet helt fint. Fiber-mediekonverteren som brukes har en anslått pris på rundt 1000 kr med *SFP*-modulen. SFP står for *Small Form-factor Pluggable Transceiver* og er en kontakt for fiberkabler. HDMI til USB-C adapteren kostet ca. 500 kr.

Materiell	**Pris**
Lenovo Thinkvision 14	3000 kr
Lenovo Thinkvision 14t	3400 kr
Acer Nitro 5 AN515-56-56HA	12000 kr
Xbox One kontroller	700 kr
UGREEN HDMI til USB-C adapter	600 kr
DiabloCable fiber media konverter	1200 kr
Totalt	**20900 kr**

Tabell 4.1: Oversikt over brukt materiell.

Om vi ikke hadde hatt portable skjermer selv kunne vi brukt fjorårets løsning med en koffert som hadde 2 skjermer montert på seg (se figur 2.23, kap. 2.8). Vi kunne kjøpt en brukt datamaskin, eller spurt IT-avdelingen på Universitetet i Stavanger om å få låne en.

4.1 Miljøregnskap

Dataprogrammet vårt krever ikke store ressurser i form av datakraft eller strømforbruk. Programmet kunne vært lagd mer effektivt ved å bruke et mer effektivt programmeringsspråk som f.eks. er kompilert, men siden programmet kun skal kjøre på en datamaskin som kun skal styre en undervannsrobot en relativt kort periode så vil dette gi minimalt med besparelse. Skulle programmet kjørt på mange datamaskiner samtidig over lengre tid ville innsparingen vært mye større.

Innvirkning på miljøet

UiS Subsea som helhet jobber mot å konkurrere i *MATE ROV Competition* der det i år er fokus på FNs havforskningstiår som beskrevet i kapittel 1.3. Årets oppgaver skal simulere arbeid på offshore havvind, vedlikehold på fiskemerder og overvåkning av havet for sunn havhelse.

Ved å delta og finne gode løsninger på problemer hjelper UiS Subsea til å gjøre disse produktene mer attraktive, siden investorer og andre kan se at det er gode og billige løsninger på problemer som for eksempel vedlikehold av havvind.

Vi har ikke kjøpt inn noe nytt datautstyr, for vår oppgave og ved å gjenbruke datautstyr sparer vi både penger og miljøet ved at mindre elektronikk produseres. ROV-design gruppen har tatt ansvar for å redusere miljøpåvirkningen ved produksjon av hele ROV-en utenom komponentene i elektronikkhuset (se kapittel 1.5.2).

Tetheren som ble brukt i år er gjenbrukt fra fjorårets undervannsrobot og kan brukes videre av nye grupper uten store modifiseringer.

Mini-PC-en er også en god kandidat for gjenbruk på videre prosjekter i UiS Subsea.

Kapittel 5

Resultater og diskusjon

5.1 Testing av ROV

5.1.1 Enhetstesting av ROV

Testene ble hovedsaklig utført i samarbeid med elektroingeniør-gruppene, og spesielt mye testing ble utført med **Maskinsyn**-gruppen [1.5.1].

Topside og mini-PC-en ble koblet sammen for å teste alle funksjonene gradvis etter som de ble utviklet. En forbedring som kunne vært gjort her er å lage enhets- og integrasjonstester som automatisk testet at funksjonene returnerte de korrekte verdiene. Dette hadde vært nyttig både første gang man lager en funksjon og når man gjør små oppdateringer siden det da oppdages umiddelbart hvis man har gjort en feil.

Før alle kretskortene var ferdig sendte **Maskinsyn-gruppen** opp eksempeldata som simulerte sensordataen som kretskortene senere skulle sende. Det ble testet at pakkene var riktig formatert og at de oppdaterte seg i GUI-en. Det ble og gjort en test av data fra gyroskopet opp mot 3D-modell, i tillegg til at det ble prøvd å styre 3D-modellen med kontrollen.

Videre ble det testet opp mot ROV-en, når nye kretskort var klare til å kobles opp mot resten av systemet. Dette funket fint siden man da fikk lagt til én og én ting om gangen som gjorde at det var lettere å finne feil.

En prototype av manipulatoren ble testet sammen med **Manipulatorgruppen**, som sto for design og bygging av manipulatoren, og **Reguleringsgruppen** som hadde ansvar for å styre manipulatoren basert på signalene vi sender ned, se kapittel 2.10.1. Denne testen viste at en del av manipulatoren som var 3D-printet var for svak og knakk.

Stillingen av lysstyrke har ikke blitt testet sammen med hele systemet, siden lysene ikke var klare, men det er likevel verifisert at det sendes ned pakker i riktig format når slideren stilles på og når av/på-knappen trykkes på.

5.1.2 Test av videooverføring

Det ble gjort en test for å sjekke forsinkelsen, også kalt *latency*, fra kamera til *topside*-datamaskinen. Dette ble gjort uten bildebehandling, så mer forsinkelse må regnes med når bildebehandlingen tas i bruk. Testen ble utført etter inspirasjon fra fjorårets test [52]. Testoppsettet er illustrert i figur 5.1.

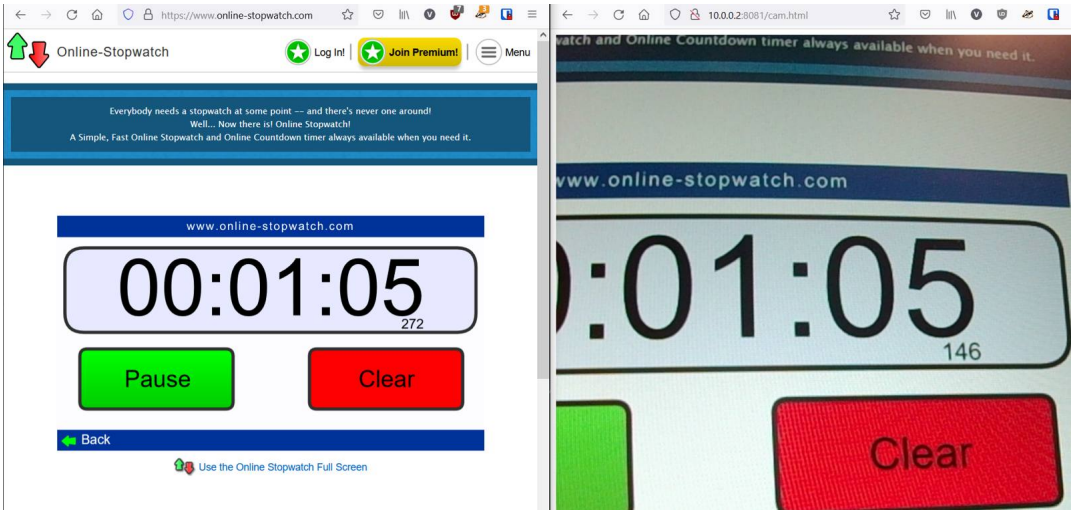
I testen ble det brukt en stoppeklokke, funnet på nettsiden `www.online-stopwatch.com`, for å ta tiden på datamaskinen. Videostrømmen fra kamera vises i nettleseren med IP-adressen til mini-PC-en: `10.0.0.2`, porten til webserveren: `8081`[1], og *path*: `cam.html`. Ett kamera filmer stoppeklokken på datamaskinskjermen, samtidig som videostrømmen fra kameraet vises på den andre halvdelen av skjermen.



Figur 5.1: Oppsettet for test av videooverføring.

Videoforsinkelsen måles, ved at det blir tatt et skjermbilde av både stoppeklokken og videostrømmen, som vises side-om-side på skjermen. Se figur 5.1 for testoppsettet.

De ble gjort en test av forsinkelsen over fiber, vist i figur 5.2, og en test av forsinkelsen over et trådløst nettverk, vist i figur 5.3, for å se hvor stor innvirkning det hadde å bruke trådløs kommunikasjon. Det skal ikke benyttes trådløs kommunikasjon når ROV-en kjøres i vannet. Men under tidlig testing av ROV-en benyttet vi oss av trådløs kommunikasjon gjennom en *router*, fordi det var mer praktisk.



Figur 5.2: Test av forsinkelse over fiber. Stoppeklokken viser 00:01:05 og **272 ms**, mens videostrømmen, viser 00:01:05 og **146 ms**. Dette gir en forsinkelse på **126 ms**.

[1]Dette var tidlig testing der det ble brukt port 8081, senere ble portene 6889 og 6888 brukt - en for hvert kamera.



Figur 5.3: Test av forsinkelse over trådløst nettverk. Stoppeklokken viser 00:01:56 og **395 ms**, mens videostrømmen viser 00:01:56 og **256 ms**. Dette gir en forsinkelse på **139 ms**.

Ved å se på tidsforskjellen mellom stoppeklokken på venstre side og videostrømmen fra kameraet, får vi forsinkelsen. Resultatet av de to testene er oppgitt i tabell 5.1

Test av forsinkelse ved videostrømming	
Test 1: Forsinkelse over fiber	$272\ ms - 146\ ms = 126\ ms$ (se figur 5.2)
Test 2: Forsinkelse over trådløst nettverk	$395\ ms - 256\ ms = 139\ ms$ (se figur 5.3)

Tabell 5.1: Test av forsinkelse til videostrøm, over fiber og trådløst nettverk.

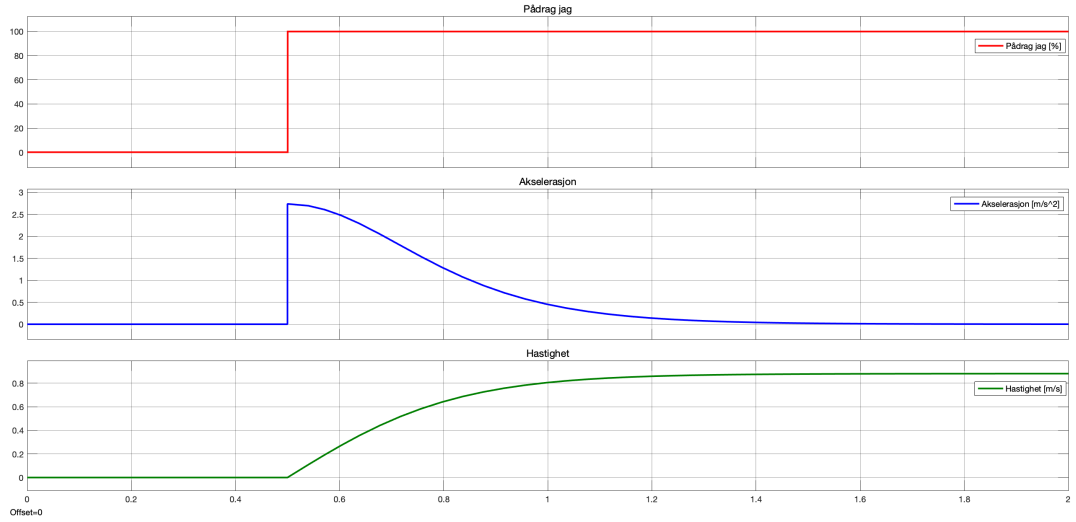
Forsinkelsen over trådløst nettverk er på 139 ms, som er 13 ms mer forsinkelse enn over fiber. Grunnen til at testingen over trådløst nettverk ble gjennomført var fordi de tidlige integrasjonstestene med hele systemet ble utført over et trådløst nettverk og det var derfor ønskelig å vite hvordan ytelsen i forhold til den ferdige løsningen kom til å være.

Forsinkelsen over fiber er 126 ms. Sammenlignet med fjorårets rapport [52, side 48], der det var en forsinkelse på 214 ms med innstillingene på IP-kameraet som ble brukt. Dette betyr at fjorårets forsinkelse var 69,8% høyere enn årets.

5.1.3 Betydning av forsinkelse for kjøring av ROV-en

Forsinkelsen fra en hendelse skjer under vann til operatøren kan oppdage den er gitt av en mengde variabler, og er viktig å minimere slik at operatøren klarer oppgaven som skal utføres, og unngår uønskede situasjoner.

Simuleringer utført av **Reguleringsgruppen** [1.5.1], vist i figur 5.4, viser at topphastigheten på ROV-en ligger på ca. $0,9\ m/s$, og etter $0,5\ s$ er man oppe i $0,8\ m/s$.



Figur 5.4: Modellering av hastigheten på ROV-en ved jag-pådrag, utført av **Reguleringsgruppen**.

Testen av forsinkelsen fra kameraet til *topside*-skjermen viser en forsinkelse på 126 ms uten bildebehandling. Hvis ROV-en har nådd toppfarten og holder 100 %-pådrag, vil ROV-en ha bevegd seg distansen, **d**, som er kommet frem til i likning 5.1.

$$\mathbf{d} = \int_{1.6s}^{1.726s} 0.9\ m/s = 0.1134\ m \tag{5.1}$$

Distansen, **d**, som ROV-en har bevegd seg er 11 cm på 126 ms.

Utregningen viste en forsinkelse som virket akseptabel før testing under vann. Da undervannsroboten senere ble vanntestet (se kapittel 5.1.4) viste det seg at dette stemte. Operatøren klarte fint å styre og plukke opp gjenstander med denne forsinkelsen og UiS Subsea klarte å kvalifisere seg til *MATE ROV World Championship*.

5.1.4 Vanntesting av ROV

Når de viktigste delene var tørrtestet og klare, var det tid for vanntesting. Det viste seg at elektronikkhuset ikke var tett, men dette gikk fint siden vi kjapt fikk trekt opp ROV-en etter at sensorkortet sendte data om vannlekkasjen og GUI-en varslet med lyd og et blinkende advarsel. Etter lekkasjen ble ROV-en tørket godt og elektronikkhuset fikk ligge over natten uten tilkobling til strøm.

Etter at kilden til lekkasjen ble funnet fikk vi testet hele systemet utenom manipulatoren som ikke var klar.

Den første testen uten lekkasje viste at **Reguleringsgruppen** måtte begrense hvor fort man kunne forandre pådrag siden sikringen gikk når man utførte brå bevegelser med kontrollspakene. I tillegg viste testen at vi hadde definert positiv retning som opp, mens **Reguleringsgruppen** hadde definert positiv retning nedover. Videre ble det oppdaget at kameraene måtte fokuseres for undervannskjøring. Det var likevel greit å se hva som foregikk på kameraet og piloten klarte fint å kjøre med den lille forsinkelsen på kameraet.

5.2 Resultat av ROV

ROV-en er blitt vanntestet og kjørt i svømmebassenget. Piloten klarte å utføre alle oppgavene noe som gjorde at UiS Subsea kvalifiserte seg. Det ble kjørt en testrunde for å prøve oppgavene som skulle løses og deretter ble tiden tatt og utførelsen filmet for å kvalifisere seg til konkurransen. Oppgavene ble utført på litt over 10 minutter, men vi hadde 15 minutter til rådighet. Dette er et resultat vi er godt fornøyde med og viser tydelig at det viktigste målet som er at ROV-en skal være lett å kjøre er oppnådd. Det fungerte å sende ned kontrollerdataen for å styre ROV-en, brukergrensesnittet viste fram sensordataen. Resultatet er en funksjonell ROV.

5.3 Overflatesystemet

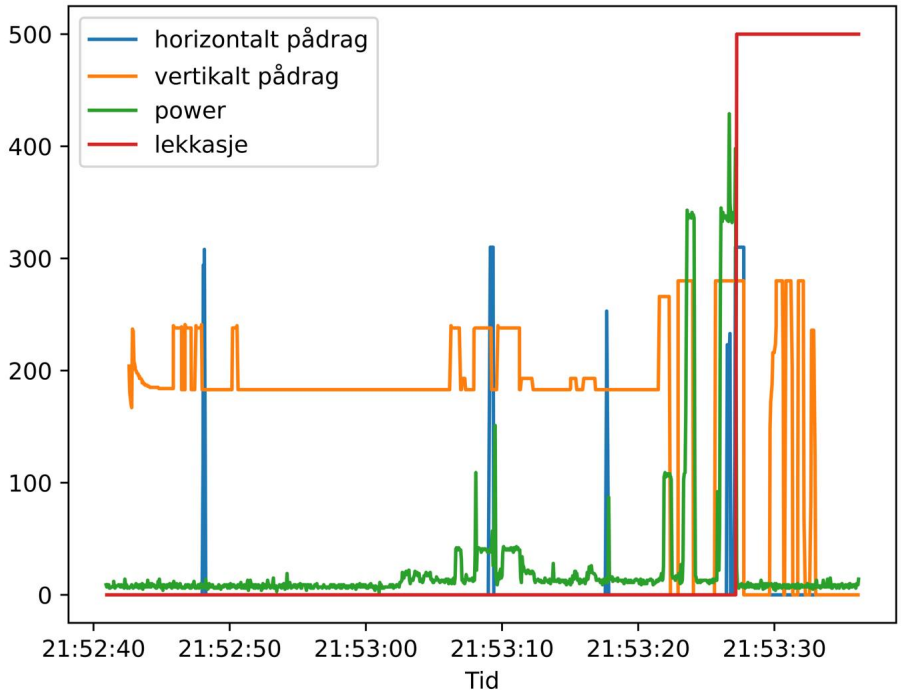
Hvordan overflatesystemet fungerte beskrives her. Det blir sett på resultat av hvordan logging av data, brukergresesnittet og brukervennligheten fungerte.

5.3.1 Logging av data

Logging av data i en tekstfil fungerte fint og ble logget uten feil. Det å logge sensordata viste seg å være nyttig. Loggfilene ble blant annet brukt av de andre gruppene for å finne problemer med sikringer som gikk, til verifisering av regulering, og til å sammenligne effektforbruk opp mot temperatur i elektronikkhuset.

Under den første vanntesten oppsto det en liten vannlekkasje på grunn av en pakning til elektronikkhuset som ikke var tett. Det ble også oppdaget at elektromagnetisk interferens var årsaken til at et kretskort ikke funket. Siden alle pakker var logget i en tekstfil med **Logger**-klassen, var det lett å skrive et skript for å lage grafer til sensordataen.

Figur 5.5 viser et eksempel på grafer som representerer sensordataen som ble logget.



Figur 5.5: Graf som ble laget for å undersøke hvorfor sikringen gikk under kjøringen.

I figuren ser man grafene for **horisontalt** og **vertikalt pådrag**, **effekt**, og **lekkasjevarsel**. Grafen til lekkasjevarselet går opp til **1** dersom det oppdages vannlekkasje i ROV-en. (Her i figuren går grafen til **500** for at det skal være mulig å se).

Rett før tidspunktet `21:53:30` i grafen, detekteres det vannlekkasje. Man kan se at grafen for effekt går ned, som konsekvens av at sikringen er gått. Logging av sensordata gjorde at det var lettere å se når sikringen gikk for videre testing.

I et annet tilfelle, var det mulig å se at kretskortet sluttet å sende opp sensordata. Det ble oppdaget at kretskortet sluttet å sende data rett etter at motorene ble startet, men de andre kortene fortsatte å sende opp data som vanlig. Fordi loggingen viste at det ene kretskortet ikke sendte opp noe data, var det mulig å oppdage at det interferens som stoppet kretskortet fra å sende data opp.

5.3.2 Kommunikasjon

Kommunikasjonen fungerte som den skulle. Det var god nok overføringshastighet fra videostrømmingen og selve kommunikasjonen bidrar minimalt til forsinkelsen av videostrømmen.

Vi forsøkte å lage TCP-kommunikasjonen slik at man kunne koble seg opp igjen uten å starte den andre siden på nytt. Gjenoppkoblingen fungerte delvis, men det var ustabilt. Litt av grunnen til at gjenoppkoblingen er ustabil er at TCP ikke er ment til å kunne bruke den samme koblingen om igjen. Dette har likevel ikke vært et problem siden omstart av programmet ikke er en omfattende prosess.

5.3.3 Brukergrensesnittet

Brukergrensesnittet fungerer som det skal og piloten, som styrte ROV-en, var fornøyd. Det viste seg under testing at det var lurt å ha en *co-pilot* som benytter seg av informasjonen i brukergrensesnittet, slik at piloten kan konsentrere seg fullt om å styre ROV-en. Det ble også lagt opp i GUI-en for at *co-piloten* kan justere kamera-tilt, men dette er ikke koblet sammen med kode på innleveringstidspunktet. Dette skal være greit å legge til og vil ha høy prioritet på ting som skal legges til fram mot konkurransen.

Brukervennlighet

Brukergresnesnittet er brukervennlig. Dette ble bekreftet av ROV-operatøren, som er den som har styrt ROV-en under testingen. Det var en god visualisering av data, og vi fant ut at det var greit å ha en *co-pilot* som følger med på sensorene, mens operatøren kjører. Det var en utfordring å differensiere mellom vertikale og horisontale thrustere, men ved å plassere pådragsprosenten rett på motoren synes piloten det ble lettere å skille mellom de.

Det er lett å starte opp programmet på en ny datamaskin. Dette ble prøvd ut, og programmet fungerte som det skulle da også. En liten ulempe med programmet er det har blitt programert for en relativt ny *Python* versjon som har funksjoner som for eksempel *type hinting*. Dette betyr at programmet ikke vil fungere på eldre Python-versjoner uten modifikasjon av koden. Dette anses likevel ikke som et stort problem da dette ikke er et program som skal distribueres bredt, og hovedsaklig kun skal kjøres på en datamaskin.

5.4 Utfordringer

5.4.1 Sensordata o.l. fra de andre gruppene

En utfordring var å få kontroll på hvilke sensordata, knapper og funksjoner som skulle inn i GUI-en. Dette var medregnet, men likefult en utfordring. Siden det er vanskelig å vite hvordan det ferdige prosjektet kommer til å bli før det begynner å ta form, og hva som må forandres på ser, vi ikke noen gode løsninger på denne utfordringen utenom å lage et modulært system slik vi lagde for å lett kunne endre på GUI-en.

5.5 Forbedringer i forhold til tidligere år

Fra tidligere år, har vi forbedret forsinkelsen på videostrømmen (se kapittel 5.1.2). Brukergrensesnittet har fått et mer moderne preg. Vi har fått til å vannteste ROV-en innenfor bachelorfristen, og kvalifisere oss til *MATE ROV World Championship*. Dette ble ikke utført i fjor.

5.6 Veien videre fremover

Vi har fått til alle de viktigste funksjonene innenfor bachelor-fristen. ROV-en kan kjøres og kvalifikasjonskravet ble fullført. Siden laget skal delta i konkurransen ca. en måned etter bachelor fristen vil det brukes tid på å forbedre delene av programmet som ikke fungerer optimalt og legge til funksjoner som er nyttige, men som ikke har fått prioritet før fristen. Det finnes alltid noe

mer å fikse på eller implementere.

Piloten trenger en del mengdetrening på styring av ROV-en, for at dette skal gå effektivt i konkurransen. Vi lister opp en del forbedringer som kan gjøres i kapittel 5.7.

5.7 Forbedringer til neste år

5.7.1 Overflatesystemet

Mulige forbedringer:

- Mulighet til å rekonfigurere joystickene på kontrolleren (ikke bare trykk-knappene).
- Teste styring av ROV gjennom en *router*.
- Dele opp main.py filen i flere filer, slik at det blir mer oversiktlig.
- Brukt et kamera med mindre *delay*. Det antas at et kamera til samme pris som ikke er et stereo-kamera vil ha kortere forsinkelse på grunn av prosesseringen som gjøres for å lime sammen bildene i mikrokontrolleren til kameraet.

5.7.2 Brukergrensesnitt

Mulige forbedringer:

- Mulighet til å vise tekst, f.eks. sensordata, over videostrømmen.
- 3D-modellen kunne hatt et gulv som viste bunnen av bassenget, slik at man kan se hvor langt unna ROV-en er fra bunnen. På dette tidspunkt sendes det ikke noen data om dybden der ROV-en kjører. Det betyr at modellen måtte ha vist en forhåndsbestemt dybde, utifra kjent dybde på bassenget. Det gis likevel data om den nåværende dybden til ROV-en.
- Det kunne ha blitt lagd *shaders* og teksturer til 3D-modellen for å få den til å se bedre ut.
- Animere manipulatorens bevegelser.
- Hatt en horisontallinje som viser om ROV-en ligger rett, selv om dette også kan sees i 3D-modellen.
- Lagd en oversikt over alle kjøreturene som kan sees i GUI-en.

Kapittel 6

Konklusjon

Som nevnt i problemstillingen (kap. 1.6.1), var det tre ansvarsområder gruppen skulle ta seg av. Ansvarsområdene gikk ut på å:

- Lage en løsning for kommunikasjonen mellom *topside* og ROV-en.
- Håndtere inndata fra kontrolleren som skal styre ROV-en.
- Lage et brukergrensesnittet til ROV-en.

Det var disse hovedmålene som måtte nås for å kunne utføre oppgaven - å lage et velfungerende *topside*-system for UiS Subsea sin ROV. Disse tre hovedmålene ble nådd. Gruppen har klart å lage et fungerende *topside*-system for UiS Subsea.

De tre hovedområdene vil nå bli gjennomgått. I tillegg blir punktene oppgitt i **Oversikten over hva som skal gjøres** (kap. 1.6.5) sammenlignet med hva som ble utført, for å se om målene ble oppnådd.

Overføring av sensordata og videostrøm

Overføring av sensordata og videostrøm fungerer bra. Det har blitt laget et system med to videostrømmer som har kort forsinkelse som fungerte godt under kjøring. Overføringen av sensordata fungerer også godt og slik det ble presentert i introduksjonen. Det er et modulært system hvor det er lett å legge til ny data og dataen blir logget på en hensiktsmessig måte. Den loggede dataen har vært nyttig og enkel å benytte seg av.

Styrekontroller

Uthentingen av data fra kontrolleren fungerer godt. Dataen blir normalisert og sendt til hovedprogrammet. Det er mulig å koble kontrolleren på til på nytt uten å restarte programmet. Profilsystemet fungerer, men det er noen feil som ikke har blitt rukket å fikse på før fristen. Dette er fordi profilsystemet ikke har blitt benyttet i stor grad. Når det ble oppdaget noe som ikke var optimalt ble dette fikset i standardprofilen og det var derfor ikke nødvendig å lage en ny profil for dette.

Brukergrensesnitt

Brukergrensesnittet viser klart og tydelig informasjonen som er viktig for kjøringen av ROV-en. Den har et moderne utseende og Det ble definert klare oppgaver og alle de viktigste målene ble nådd. Det mangler hastighet og ROV-ens girvinkel, men det er fordi det ikke finnes sensordata som viser dette. Det ble laget en fremvisning av en 3D-modell som klart viser hvilken vinkel undervannsroboten står i. Denne har en feil, men for det foreløpige bruksområdet med lave rull- og stampvinkler fungerer denne fint. Justering av lysstyrken til ROV-ens lys har kun blitt testet lokalt, men fra vår side fungerer det som det skal. Lysene, som **Kraftforsyningsgruppen** hadde ansvar for, har ikke blitt vanntestet med ROV-en enda.

Konklusjonen til slutt er at operatørgrensesnittet og *topside*-kommunikasjonen fungerer som det skal, og dette vises i videoene nedenfor.

Link til video som viser en demonstrasjon av operatørgrensesnittet:
`https://youtu.be/T-6gK8st1M8` *(Videoens varighet: 1:36)*

Link til UiS Subsea sin kvalifikasjonsvideo til MATE ROV-konkurransen:
`https://youtu.be/vSQ23oaLuho` *(Videoens varighet: 15:06)*

Bibliografi

[1] MATE ROV Competition . MATE World Championship Information . Hentet fra: https://materovcompetition.org/worldchampinfo. (Lastet ned: 12.05.2022).

[2] MATE . Hjemmeside MATE ROV Competition . https://materovcompetition.org/. (Lastet ned: 30.01.2022).

[3] Wikipedia . Remotely Operated Underwater Vehicle . https://en.wikipedia.org/wiki/Remotely_operated_underwater_vehicle#Educational_outreach, 2022. (Lastet ned: 31.01.2022).

[4] GO-BGC . Float Technology . https://www.go-bgc.org/floats. (Lastet ned: 01.04.2022).

[5] Mbari . Float profil . https://www.mbari.org/go-bgc-release/, 2020. (Lastet ned: 31.01.2022).

[6] MATE . Hjemmeside MATE Center . https://www.marinetech.org/. (Lastet ned: 30.01.2022).

[7] MATE . *2022 Explorer Competition Manual* , 2021. Konkurransemanual fra MATE.

[8] MATE . Mission_Flythrough_2022_Explorer . https://vimeo.com/679543161, 2022. (Lastet ned: 01.04.2022).

[9] MATE . EXPLORER product demonstration prop building instructions . https://files.materovcompetition.org/2022/EXPLORER_prop_building_instructions_2022.pdf, 2022. (Lastet ned: 01.04.2022).

[10] MATE ROV Competition . 2022 EXPLORER Manual 15 Dec 2021 . Hentet fra: https://files.materovcompetition.org/2022/2022_EXPLORER_Manual_21_JAN_2022.pdf. (Lastet ned: 26.01.2022).

[11] J. Walrand and P. Varaiya. *High-Performance Communication Networks*. Morgan Kaufmann Publishers, 2 edition, 2000.

[12] The Cisco Learning Network . OSI Model Reference Chart . Hentet fra: https://learningnetwork.cisco.com/s/article/osi-model-reference-chart. (Lastet ned: 30.03.2022).

[13] Urmila Samariya . TCP 3-Way Handshake Process . Hentet fra: https://www.tutorialspoint.com/tcp-3-way-handshake-process. (Lastet ned: 28.03.2022).

[14] Naeem Khademi . Faget DAT230 Kommunikasjonsteknologi I . Forelesninger på Universitetet i Stavanger. Våren 2021.

[15] AutoPi . CAN Bus Protocol: The Ultimate Guide (2022) . Hentet fra: https://www.autopi.io/blog/can-bus-explained/. (Lastet ned: 28.03.2022).

[16] Hein Meling. Faget DAT320 Operativsystemer og systemprogrammering. Forelesninger på Universitetet i Stavanger. Våren 2020.

[17] OpenClassrooms . Identify the Advantages of Concurrency and Parallelism . Hentet fra: https://openclassrooms.com/en/courses/5684021-scale-up-your-code-with-java-concurrency/5684028-identify-the-advantages-of-concurrency-and-parallelism. (Lastet ned: 24.03.2022).

[18] Vebjørn Lia Riiser - Orginal av Igor Maculan. mjpeg_stream . Hentet fra: https://github.com/ArcticMeteor246/UiS-subsea-Bildebehandling/commit/d40f1d34b74dfd726c326d5b697c2e907c13600d#diff-0c3f09a36f1856ed6bc4e6e295b386bc1eb783ed25f17bae281f5eb1e6b83c64. (Lastet ned: 08.05.2022).

[19] Patricio Gonzalez Vivo og Jen Lowe. The Book of Shaders, . Hentet fra: https://thebookofshaders.com. (Lastet ned: 27.04.2022).

[20] UiO . Git logo . Hentet fra: https://www.usit.uio.no/om/organisasjon/bnt/kd/gdw/gdw-blogg/2016/git-2.10.2.html. (Lastet ned: 20.01.2022).

[21] A. Hunt and D. Thomas. *The pragmatic programmer*. Pearson, 2 edition, 2019.

[22] Tom Heine Nätt. Datasikkerhet . Hentet fra: https://snl.no/datasikkerhet/. (Lastet ned: 31.03.2022).

[23] Domoticx . PyGame logo . Hentet fra: http://domoticx.com/python-library-pygame/. (Lastet ned: 31.03.2022).

[24] Wikipedia . Golang logo . Hentet fra: https://en.wikipedia.org/wiki/Go_(programming_language). (Lastet ned: 23.02.2022).

[25] PNGegg . C# logo . Hentet fra: https://www.pngegg.com/en/png-eduyx. (Lastet ned: 23.02.2022).

[26] Anthony King Ho . Using Python for Subsea Simulation and Control . Hentet fra: https://control.com/technical-articles/using-python-for-subsea-simulation-and-control/. (Lastet ned: 25.03.2022).

[27] Morten Wærsland, Stian Sandve . *Topside styringssystem for fjernstyrt undervannsfarkost (ROV)* . Bacheloroppgave, UiS Subsea, 2014.

[28] EIVA . Technical software architect – C# . Hentet fra: https://www.eiva.com/about/career/career-opportunities/software-positions/technical-software-architect-c. (Lastet ned: 31.03.2022).

[29] EIVA . EIVA Launches a Complete Software Solution for ROV Steering . Hentet fra: https://www.eiva.com/about/eiva-log/eiva-launches-a-complete-software-solution-for-rov-steering. (Lastet ned: 02.03.2022).

[30] iCsys AS . Bilde fra LinkedIn-innlegg . Hentet fra: https://www.linkedin.com/posts/icsys-as_subseadagen-activity-6917043537830113280-bpdz?utm_source=linkedin_share&utm_medium=member_desktop_web. (Lastet ned: 31.03.2022).

[31] Harikrishna Kundariya. Analyzing The Reasons Why Python Programming Language Is Favorite Of Developers. Hentet fra: `https://www.esparkinfo.com/blog/reason-why-python-programming-language-favorite-with-developers.html/`. (Lastet ned: 31.03.2022).

[32] Bjørnar Størksen Wiik, Jens Trydal . *Bildegjenkjenning og autonom kjøring* . Bacheloroppgave, UiS Subsea, 2021.

[33] Wikipedia . Qt Logo . Hentet fra: `https://no.wikipedia.org/wiki/Qt_Development_Frameworks`. (Lastet ned: 28.02.2022).

[34] Wiki.qt.io . About Qt . Hentet fra: `https://wiki.qt.io/About_Qt`. (Lastet ned: 20.02.2022).

[35] KDAB . Why learn about Qt . Hentet fra: `https://www.kdab.com/software-services/scheduled-training/programming-with-qt/why-learn-qt/`. (Lastet ned: 20.02.2022).

[36] Zetcode . Qt5 tutorial . Hentet fra: `https://zetcode.com/gui/qt5/`. (Lastet ned: 20.02.2022).

[37] Wikipedia . PyQt logo . Hentet fra: `https://en.wikipedia.org/wiki/PyQt#/media/File:Python_and_Qt.svg/`. (Lastet ned: 16.03.2022).

[38] Martin Fitzpatrick . Creating your first app with PyQt5 . Hentet fra: `https://www.pythonguis.com/tutorials/creating-your-first-pyqt-window/`. (Lastet ned: 07.01.2022).

[39] Qt . QWidget . Hentet fra: `https://doc.qt.io/qtforpython-5/PySide2/QtWidgets/QWidget.html`. (Lastet ned: 07.01.2022).

[40] Qt . QMainWindow . Hentet fra: `https://doc.qt.io/qtforpython-5/PySide2/QtWidgets/QMainWindow.html`. (Lastet ned: 07.01.2022).

[41] Qt . QApplication . Hentet fra: `https://doc.qt.io/qtforpython-5/PySide2/QtWidgets/QApplication.html`. (Lastet ned: 07.01.2022).

[42] Qt . Qt Designer Manual . Hentet fra: `https://doc.qt.io/qt-5/qtdesigner-manual.html`. (Lastet ned: 20.04.2022).

[43] Qt . The Style Sheet Syntax . Hentet fra: `https://doc.qt.io/qt-5/stylesheet-syntax.html`. (Lastet ned: 21.04.2022).

[44] Heiki Tveit Henrichsen, Kurt Lukas Vincent Neuenschwander . *Design av regulerings-, styrings- og kommunikasjonssystem for et fjernstyrt undervannsfartøy* . Bacheloroppgave, UiS Subsea, 2019.

[45] Komplett . Logitech G Extreme 3D Pro Joystick . Hentet fra: `https://www.komplett.no/product/775770/gaming/gaming-utstyr/spillkontrollere/joystick/logitech-g-extreme-3d-pro-joystick`. (Lastet ned: 11.04.2022).

[46] CDON . Xbox 360 . Hentet fra: https://cdon.no/elektronikk/handkontroll-till-xbox-360-vit-p37004646?gclid=CjwKCAjwo8-SBhAlEiwAopc9Wz0A6PgvbSJ78jUkkfYX-6e08F70I9CuhCQz70MQ-QDeTF92yknW7BoCJisQAvD_BwE&gclsrc=aw.ds. (Lastet ned: 11.04.2022).

[47] Elkjøp . Handkontroll till Xbox 360 . Hentet fra: https://www.elkjop.no/product/gaming/spillkonsoller-og-tilbehor/xbox/xbox-tilbehor/xbox-one-s-tradlos-kontroll-hvit/XONEWLCONTWH. (Lastet ned: 11.04.2022).

[48] Gamepad-tester . Gamepad Tester . Hentet fra: https://gamepad-tester.com/. (Lastet ned: 01.03.2022).

[49] SubPNG . Xbox One Controller Background Transparent PNG . Hentet fra: https://www.subpng.com/png-0pyr5c/. (Lastet ned: 26.01.2022).

[50] Pygame . Pygame . Hentet fra: https://www.pygame.org/. (Lastet ned: 26.01.2022).

[51] The Fiber Optic Association . Understanding Wavelengths In Fiber Optics . Hentet fra: https://www.thefoa.org/tech/wavelength.htm. (Lastet ned: 23.03.2022).

[52] Oliver Langvik Veland, Martin Vesterøy Hausken . *Kommunikasjon og videostrøm av fjernstyrt vannfartøy* . Bacheloroppgave, UiS Subsea, 2021.

[53] Ihab Ghanim, Marius brataas . *Kommunikasjon, Videostrøm og Regulering av fjernstyrt undervannsfarkost (ROV)* . Bacheloroppgave, UiS Subsea, 2018.

[54] iPerf. iPerf - The Ultimate Speed Test Tool for TCP, UDP and SCTP . Hentet fra: http://www.uis.no/bibliotek/aktuelt/dognapent-biblio-pa-uis-article98438-8346.html. (Lastet ned: 15.01.2022).

[55] ASUS . Mini PC PN51 . Hentet fra: https://www.asus.com/no/Displays-Desktops/Mini-PCs/PN-PB-series/Mini-PC-PN51/. (Lastet ned: 01.03.2022).

[56] GeeksforGeeks . Socket in Computer Network . Hentet fra: https://www.geeksforgeeks.org/socket-in-computer-network/. (Lastet ned: 15.01.2022).

[57] AliExpress . 1.3megapixel 960P stereo USB2.0 camera module . Hentet fra: https://www.aliexpress.com/item/33041252306.html. (Lastet ned: 25.01.2022).

[58] Fredrik Jonsbråten, Philip Johan Brun . *Overflatesystem og grafisk brukergrensesnitt* . Bacheloroppgave, UiS Subsea, 2016.

[59] EIVA . A Complete Software Solution for ROV operations . Hentet fra: https://www.eiva.com/about/what-we-do/rov-steering. (Lastet ned: 02.03.2022).

[60] Igor Maculan. simpleMjpegStreamerHttpServer. https://gist.github.com/n3wtron/4624820#file-simple_mjpeg_streamer_http_server. (Lastet ned: 23.02.2022).

[61] Advanced Micro Devices. AMD Ryzen™ 7 5700U. Hentet fra: https://www.amd.com/en/product/10846. (Lastet ned: 10.05.2022).

[62] Wanderson. TUTORIAL - Animated Toggle Button - [Python, PySide6, Qt Widgets] - MODERN GUI - Custom Widget. https://www.youtube.com/watch?v=NnJFi285s3M&ab_channel=Wanderson.

[63] Wanderson M. Pimenta . Python PySide2 - Circular ProgressBar - Modern GUI . Hentet fra: https://github.com/Wanderson-Magalhaes/Python_PySide2_Circular_ProgressBar_Modern_GUI. (Lastet ned: 15.01.2022).

[64] Wanderson. Modern GUI - Python, Qt Designer and PySide2 - Circular Progress Bar/Preloader (PyQt5 Compatible!). https://www.youtube.com/watch?v=zUnrLHbYmKA&ab_channel=Wanderson.

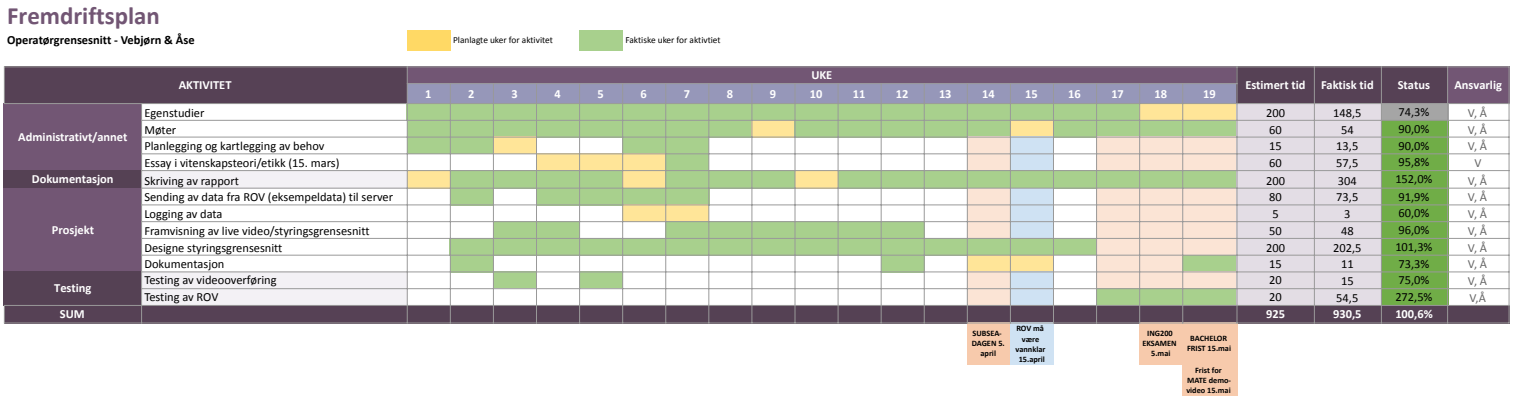
[65] Wanderson M. Pimenta . Python PySide2 - Circular ProgressBar - Modern GUI . Hentet fra: https://github.com/Wanderson-Magalhaes/Python_PySide2_Circular_ProgressBar_Modern_GUI/blob/master/main.py. (Lastet ned: 15.01.2022).

[66] Wanderson M. Pimenta . PyDracula - Modern GUI PySide6 / PyQt6 . Hentet fra: https://github.com/Wanderson-Magalhaes/Modern_GUI_PyDracula_PySide6_or_PyQt6. (Lastet ned: 15.01.2022).

Vedlegg A

Fremdriftsplan

Gruppens fremdriftsplan som viser de forskjellige aktivitetene som ble planlagt å gjennomføre i løpet av perioden. Fremdriftsplanen er vist i figur A.1



Figur A.1: Fremdriftsplanen i løpet av perioden.

Gruppen satte et forventet tidsbruk på de forskjellige oppgavene og loggførte timeantallet som ble brukt på hver aktivitet. Dette ble brukt som et hjelpemiddel på å vite hvordan man ligger an i prosesssen.

Timeantallet som ble logget, vises i figur A.2.

Timelister

Navn: Vebjørn Lia Riiser

AKTIVITET		UKE 1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	Totale timer
Administrativt/annet	Egenstudier	5	6,5	2,5		5,5	4	4		6			1								34,5
	Møter	2	3,5	1		1	1,5	2,5					1,5	1	12				1,5	1,5	29
	Planlegging og kartlegging av behov	1,5	2				1	2					0,5	1							8
	Essay i vitenskapsteori/etikk (15. mars)							0,5		6	35	16									57,5
Dokumentasjon	Skriving av rapport				2	0,5		0,5				3	12	16,5	5	1	15,5	3	19	45	123
Prosjekt	Sending av data fra ROV til server		5,5		5	8	11	9	8				1	4	9	9					69,5
	Logging av data					1							1	1							3
	Framvisning av live video/styringsgrensesnitt			8,5	13	4	2	2	6				2,5	6		2					46
	Designe styringsgrensesnitt		4	19	3	10	4	13	16					4		6					79
	Dokumentasjon		1										1,5	0,5				7			10
Testing	Testing av videooverføring			1		1											13				15
	Testing av ROV																8	17	17		42
SUM		8,5	22,5	32	23	31	23,5	33,5	30	12	35	19	21	34	26	18	36,5	27	37,5	46,5	516,5

Navn: Åse Jortveit Sagebakken

AKTIVITET		UKE 1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	Totale timer
Administrativt/annet	Egenstudier	28	10	6	6	5		5	3	13	10	9	6	4	2	2	5				114
	Møter	2	3,5	1,5	1	1	1	1	1		2	1,5	2	1	1		1	1,5	1,5	1,5	25
	Planlegging og kartlegging av behov	3,5	2																		5,5
Dokumentasjon	Skriving av rapport		0,5	1	3	1		4	1	6		4	13	13,5	8	17	27	12	21	49	181
Prosjekt	Sending av data fra ROV til server		1		1														2		4
	Logging av data																				0
	Framvisning av live video/styringsgrensesnitt				2																2
	Designe styringsgrensesnitt			10	9	20	6	9	22	6	12,5	6	1	2	4	5	1	8	2		123,5
	Dokumentasjon																			1	1
Testing	Testing av videooverføring																				0
	Testing av ROV																	7	5,5		12,5
SUM		33,5	17	18,5	22	27	7	19	27	25	24,5	20,5	22	20,5	15	24	34	28,5	32	51,5	468,5

Navn: Åse Jortveit Sagebakken

UiS Subsea - Markedsføring	Styremøter				2															1,5	3,5
	Planlegging	1	1	1	2					2			2				0,5		2	1	12,5
	Lage flyer			3																	3
	Navnekonkurranse - ROV og float								2	1											3
	Stand/Åpen dag										3										3
	Subsea dagen												3	9	9						21
	Fotografering og redigering															1	1,5	1,5		2	6
																		Timer (utenom bachelorprosjektet):			52

Figur A.2: Timeantall som er blitt logget av studentene på gruppen.

Utenom bachelorprosjektet, ble også noen timer loggført til markedsføring av UiS Subsea, da et av gruppemedlemmene var en av de markedsføringsansvarlige. En del timer gikk til styring av sosiale medier, arrangering og markedsføring av Subsea-dagen.

Vedlegg B

Prosjektfiler

Figur B.1 og B.2 viser tabeller, som ble brukt til informasjonutveksling mellom gruppene i UiS Subsea-prosjektet.

Tilgjengelige kort
Styrekort
Sensorkort
Hovedkort
Strømforsyning

CAN BUS	
Maks	0x7FF
Alle datapakker er maksimalt 8 byte	
Lavere ID = Høyere prioritet	

Mottaker	HEX	DEC	ID-gruppe
ID område styrekort	0x40 - 0x5F	64 - 95	2
ID område sensorkort	0x60 - 0x7F	96 - 127	3
ID område strømkort	0x60 - 0x9F	128 - 159	4
ID område hovedkort	0x80 - 0xBF	160 - 191	5

CAN-pakker

ID (Desimal)	Sender	Innhold	Mottar
64	Hovedkort	oppstart ("start\n")	Styrekort
70	Hovedkort	Styredata int8 (X ,Y, Z, rotasjon, manipulator, fri, fri, throttling)	Styrekort
71	Hovedkort	oppdatere param (uint32 <ID>, float <parameterverdi>)	Styrekort
80	Sensorkort	Regpakke int16 [#1-2Byte: hiv(mm), #3-4: rull*10, #5-6: stamp*10] (hiv/rull/stamp = int16)	Styrekort
81	Sensorkort	Akselerometer int16 [#1-2Byte: x-aks*100, #3-4: y-aks*100, #5-6: z-aks*100] (X/Y/Z = int16)	Styrekort
90	Strømforsyning	Strømforbruk [(12v 1300w forbruk i watt) uint16, Strømforbruk (12v 240w forbruk i watt) uint16, Strømforbruk (5v 65w forbruk i watt) uint16] alle verdiene ganges med 10 slik at vi får desimaler og må gjøres om til float av mini-PC-en	Styrekort
95	Hovedkort	Heartbeat ("ping!\n")	Styrekort
96	Hovedkort	oppstart ("start\n")	Sensorkort
100	Styrekort	Forespør sensordata (80 og 81)	Sensorkort
127	Hovedkort	Heartbeat ("ping!\n")	Sensorkort
128	Hovedkort	oppstart ("start\n")	Strømforsyning
126	Hovedkort	Kommando for å sette nullpunkt til dybdeberegning	Sensorkort
139	Hovedkort	sikringOgRegulatorer [byte 1: bit 1 65W sikring, bit 2 240W sikring, bit 3 1300W sikring, bit 4 regulator 65W sikring, bit 5 240W regulator, bit 6 1300W regulator]	Strømforsyning
140	Sensorkort	Kraftpakke int8 [#1Byte: Lekasje- og kontaktflagg temp, #2Byte: temp på hovedkort, #3Byte: temp på kraftkort, #4 temp på sensorkort](temp = uint8)	Strømforsyning
142	Hovedkort	Lysstyrke uint8[forover, ned]; [69, 42]	Strømforsyning
159	Hovedkort	Heartbeat ("ping!\n")	Strømforsyning
160	Hovedkort	oppstart ("start\n")	Hovedkort
170	Styrekort	Thrusterpådrag 8x int8 (-100 til 100)	Hovedkort
171	Strømforsyning	sikringOgRegulatorerSatt [uint8_t: bit 1 240W sikring, bit 2; 1300W sikring, bit 3; 240W regulator, bit 4; 1300W regulator]	Hovedkort
189	Styrekort	Heartbeat svar ("pong!\n")	Hovedkort
190	Sensorkort	Heartbeat svar ("pong!\n")	Hovedkort
191	Strømforsyning	Heartbeat svar ("pong!\n")	Hovedkort

Figur B.1: Oversikt over ID-område til kretskortene og CAN-pakkene.

Beskrivelse CAN-pakker til *topside*			
Fra	CAN-pakke navn	Hva	can_id
Sensorkort	gyro	{"gyro": [rull, stamp, hiv]}	80
Sensorkort	accel	{"accel": [accel_x, accel_y, accel_z]}	81
Kraftkort	power_consumption	{"power_consumption": [uint16_power_consumption_0_i_watt, uint16_power_consumption_1_i_watt, uint16_power_consumption_2_i_watt]}	90
Sensorkort	lekk_temp	Pakke fra sensorkort med et dictonary hvor verdiene er en byte som sier hvilke av lekkasje sensorene er på. 0 for ingen. Deretter 3 forskjellige temperatursensorer {"lekk_temp": [lekk, temp1, temp2, temp3]}	140
Styrekort	thrust	{"thrust": [int8 * 8]} Rekkefølge: hhf, hhb, hvb, hvf, vhf, vhb, vvb, vvf. horisontal/vertikal, høyre/venstre, framme/bak	170

Oversikt over sensordata som skal til GUI		
Fullt navn på widget	CAN-pakke navn	Beskrivelse
label_bildebehandlingmodus	lokalt	viser hvilken bildebehandlingsmodus som er aktiv
label_tid	lokalt	tid fra ROV-en starter
label_tidtaker	lokalt	tidtaker fra man trykker start
label_dybde	gyro	viser dyben ROV-en beveger seg fra nullpunktet
label_effekt_elektronikk	power_consumption	index 2 i power consumption i watt. 5V 65W
label_effekt_manipulator	power_consumption	index 0 i power consumption i watt. 12V 1300W
label_effekt_thrustere	power_consumption	index 1 i power consumption i watt. 12V 240W
label_temp_ROV_hovedkort	lekk_temp	temperatur ved hovedkort
label_temp_ROV_kraftkort	lekk_temp	temperatur ved kraftkort
label_temp_ROV_sensorkort	lekk_temp	temperatur ved sensorkort
label_gjsnitt_temp_ROV	lokalt	gjennomsnittet regnes ut
label_HHF_percentage	thrust	hhf - horisontal høyre foran - horisontal/vertikal, høyre/venstre, framme/bak
label_HHB_percentage	thrust	hhb - horisontal høyre bak
label_HVB_percentage	thrust	hvb - horisontal venstre bak
label_HVF_percentage	thrust	hvf - horisontal venstre foran
label_VHF_percentage	thrust	vhf - vertikal høyre foran
label_VHB_percentage	thrust	vhb - vertikal høyre bak
label_VVB_percentage	thrust	vvb - vertikal venstre bak
label_VVF_percentage	thrust	vvf - vertikal venstre foran
mani_kraft_inn_ut	lokalt	Hvor stor kraft vi kjører manipulator inn og ut, ut er positive verdier, alltid 100%
mani_kraft_rotasjon	lokalt	Hvor stor kraft vi roterer manipulator, med klokka er positiv, alltid 100%
mani_kraft_gripe_slippe	lokalt	Hvor stor kraft gripemotor får, gripe er positive verdier, alltid 100%
label_percentage_struping	lokalt	prosent til struping
label_percentage_lys_forward	lokalt	mellom 0 og 100. Topside sender ned
label_percentage_lys_down	lokalt	mellom 0 og 100. Topside sender ned
label_percentage_mani_1	lokalt	prosent mani1 (0-100)
label_percentage_mani_2	lokalt	prosent mani2 (0-100)
label_percentage_mani_3	lokalt	prosent mani3 (0-100)
recent_img	fra_ssh	siste bilde som vises
vann_lekk_1	lekk_temp	Varsel om vannlekkasje. i midten på en måte. bit med index 3 er 1
vann_lekk_2	lekk_temp	Varsel om vannlekkasje. framme på en måte. inni elektronikkhuset. framme i nærheten av kamerahuset. bit med index 4 er 1
vann_lekk_3	lekk_temp	Varsel om vannlekkasje. kommer som en byte. Må sjekke hvilke bit som blir satt for å finne ut hvilken sensor. lekkasjesensor kamerahus. bit med index 5 er 1
Kontrollerside:		
comboBox_velg_profil	lokalt	velg kontroller-profil

Figur B.2: Beskrivelse av CAN-pakker til *topside*, og oversikt over sensordataen som skal til GUI-en.

Figur B.3 viser oversikten over navnene på *widget*-ene som ble brukt i GUI-en på *topside*.

Oversikt over widgets i GUI	
Fullt navn på widget	Beskrivelse
btn_manuell	manuell kjøring
btn_finn_fisk	finn fisk og kalkuler nærmeste avstand
btn_autonom_merd	autonom merd
btn_bildemoasaikk	bildemoasaikk
btn_autonom_parkering	autonom parkering
btn_reset_tidtaker	reset tidtaker
btn_start_tidtaker	start tidtaker
btn_reset_nullpunkt	reset-knapp for dybde nullpunkt
btn_regulator_elektronikk	Av/på-knapp - regulator elektronikk
btn_regulator_manipulator	Av/på-knapp - regulator manipulator
btn_regulator_thrustere	Av/på-knapp - regulator thrustere
btn_reset_sikring_elektronikk	reset-sikring for elektronikk
btn_reset_sikring_manipulator	reset-sikring for manipulator
btn_reset_sikring_thrustere	reset-sikring for thrustere
slider_struping_thrustere	slider for struping
label_percentage_struping	prosent til struping
btn_lav_struping	knapp skru ned struping (satt til 30%)
btn_ta_bilde_frontkamera	ta bilde med frontkamera
btn_ta_bilde_havbunn	ta bilde med havbunnskamera
btn_slett_bilde	slett siste bilde
btn_start_video_frontkamera	checked: starter videoopptak på frontkamera. unchecked: stopper videoopptak.
btn_start_video_havbunnskamera	checked: starter videoopptak på havbunnskamera. unchecked: stopper videoopptak.
btn_front_tilt_ned	frontkamera tilt ned
btn_front_tilt_opp	frontkamera tilt opp
btn_havbunn_tilt_ned	bunnkamera tilt ned
btn_havbunn_tilt_opp	bunnkamera tilt opp
Kontrollerside:	
btn_make_new_profile	lage en ny kontroller-profil
btn_reset	reset kontroller-profil
btn_kontroller_info	informasjon om kontrolleren
btn_save_profile	lagre kontroller-profil
comboBox_A_btn, B_btn, LB_btn, LT_btn, RB_btn, RT_btn, X_btn, Y_btn, directional_pad_leftright, directional_pad_updown, left_stick_btn, left_stick_x, left_stick_y, menu_btn, right_stick_btn, right_stick_x, right_stick_y, view_btn	comboBox for å velge funksjoner til de forskjellige kontroller-knappene

Figur B.3: Oversikt over navnene på *widget*-ene som ble brukt i GUI-en.